TECHNICAL INTERVIEW GUIDE

The NLP Engineer's System Design Interview Guide

Preparation + 100 System Design Q&As: Scalability, LLMs, RAG, Transformers, and Production NLP Architecture

BONUS: TOP 35 MAANG NLP SYSTEM DESIGN

AUTHOR

Lamhot Siagian

LINKEDIN NEWSLETTER

AI Engineering Insider

System Design for NLP Interview Preparation

A Comprehensive Guide to Designing Scalable Natural Language Processing Systems

First Edition, 2026

Published independently by the author.

linkedin.com/in/lamhotsiagian

buymeacoffee.com/lamhot

For permissions and inquiries:
lamhotsiagian2025@gmail.com

About This Book

This book bridges two critical skill sets that top tech companies like Meta, Google, Amazon, and other FAANG companies expect from NLP engineers: deep knowledge of Natural Language Processing and the ability to design systems that scale to millions of users.

Each chapter covers a core NLP domain—from foundational mathematics through cutting-edge transformer architectures—and pairs it with 10 system design interview questions that test your ability to build production-grade NLP systems.

The questions are mapped to real-world system design challenges: scalability, high availability, data consistency, latency optimization, and more. Answers include architectural diagrams, Python code snippets, and the kind of trade-off analysis that interviewers expect.

Whether you are preparing for a senior ML engineer role or a staff-level NLP position, this book gives you the vocabulary, patterns, and confidence to tackle any system design question involving language technology.

Lamhot Siagian — AI Engineering Insider

linkedin.com/in/lamhotsiagian

0 Contents

linkedin.com/in/lamhotsiagian

1 Foundations of NLP & System Design

linkedin.com/in/lamhotsiagian

Before building production NLP systems, you need a solid command of the mathematical and computational building blocks. This chapter covers the foundational concepts—linear algebra, probability, statistics, and core machine learning—that underpin every NLP system at scale.

1.1 Chapter Overview

Every NLP system at a company like Meta or Google is built on a stack of mathematical primitives. When an interviewer asks you to design a real-time content moderation system, they expect you to understand not just the ML model, but how linear algebra operations map to GPU compute, how probability distributions inform your sampling strategy, and how gradient descent interacts with distributed training across hundreds of machines.

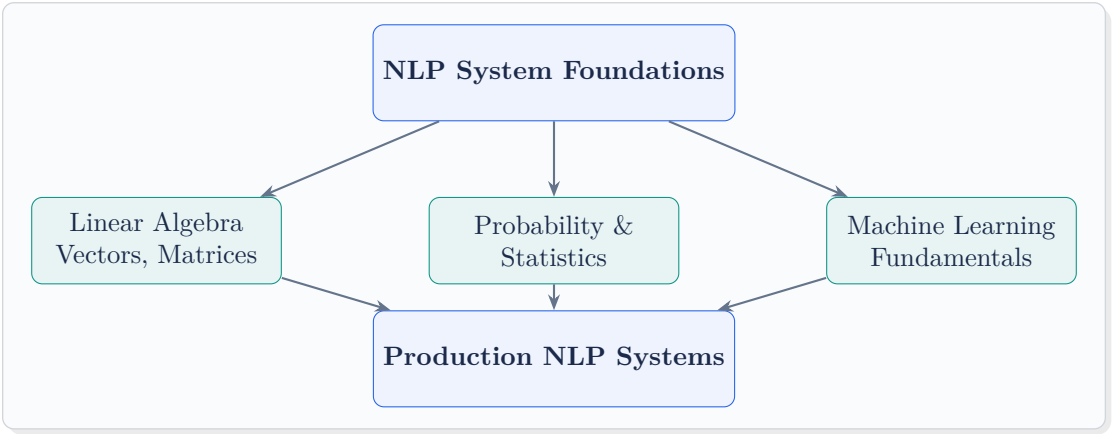
The foundations break down into three pillars:

Linear Algebra powers the core computations. Word embeddings are vectors, attention matrices are matrix multiplications, and dimensionality reduction techniques like PCA and SVD are used to compress representations for efficient storage and retrieval.

Probability & Statistics drives everything from language modeling (predicting the next token is fundamentally a probability distribution over vocabulary) to evaluation metrics, A/B testing your deployed system, and Bayesian approaches to uncertainty estimation.

Machine Learning Fundamentals tie it all together—supervised and unsupervised learning paradigms, optimization via gradient descent, regularization to prevent overfit-

ting, and the bias-variance trade-off that determines whether your model generalizes in production.



Key Insight

At FAANG-level interviews, foundation questions are never asked in isolation. You will be expected to connect math to system decisions: "Why use float16 instead of float32?" is a linear algebra question, a latency question, and a cost management question all at once.

1.2 System Design Interview Questions

Q1: Design a Distributed Model Training Pipeline for a Large NLP Model

System Design Focus: Scalability Cost Management

How would you design a distributed training system capable of training a 10-billion parameter language model across a cluster of GPUs? Walk through the architecture, data parallelism vs. model parallelism, and how you would handle gradient synchronization.

Architecture Overview:

The system requires a multi-node GPU cluster orchestrated by a training coordinator. The key architectural decisions involve choosing between data parallelism (replicate the model across GPUs, split the data) and model parallelism (split the

model across GPUs), or a hybrid approach.

Core Components:

- **Training Orchestrator:** Manages job scheduling, checkpoint management, and fault recovery. Use Kubernetes with custom operators for GPU-aware scheduling.
- **Data Pipeline:** Distributed data loading with prefetching. Store training data in object storage (S3), use a streaming dataloader to avoid memory bottlenecks.
- **Gradient Synchronization:** Use AllReduce (via NCCL) for data parallelism. For 10B parameters, use ZeRO-3 optimization to partition optimizer states, gradients, and parameters across GPUs.
- **Checkpointing Service:** Async checkpointing every N steps to distributed storage, enabling recovery without full restart.

```python
import torch
import torch.distributed as dist
from torch.nn.parallel import DistributedDataParallel as
    DDP

def setup_distributed(rank, world_size):
    dist.init_process_group("nccl", rank=rank,
                            world_size=world_size)
    torch.cuda.set_device(rank)

def train_step(model, optimizer, dataloader):
    model = DDP(model, device_ids=[local_rank])
    for batch in dataloader:
        outputs = model(batch["input_ids"].cuda())
        loss = outputs.loss
        loss.backward()  # Gradients synced via AllReduce
        optimizer.step()
        optimizer.zero_grad()
```

Scalability: Horizontal scaling by adding GPU nodes. Use mixed-precision training (float16) to halve memory and double throughput. Gradient accumulation lets you simulate larger batch sizes without more GPUs.

Cost Management: Use spot/preemptible instances for non-critical training runs. Implement elastic training that can scale down gracefully when spot instances are reclaimed.

Q2: Design a Real-Time Feature Store for NLP Model Serving

System Design Focus: **Latency & Performance** **Data Storage**
Design a feature store that serves precomputed NLP features (embeddings, TF-IDF vectors, entity tags) to ML models in production with sub-10ms latency at 100K QPS.

Architecture:
A two-tier storage architecture separates *offline* feature computation from *online* serving:
Offline Layer: Batch pipelines (Spark/Beam) compute features from raw text and write to a feature registry. Features are versioned and stored in a columnar format (Parquet on S3).
Online Layer: A low-latency serving layer backed by Redis Cluster for hot features and a fallback to DynamoDB for warm features. A consistency daemon syncs offline computations to the online store.
Key Design Decisions:

- **Caching:** L1 cache (in-process LRU, 1ms) and L2 cache (Redis, 3-5ms). Cache hit rate target: ¿95%.
- **Sharding:** Consistent hashing on entity ID across Redis nodes. Each shard handles 20K QPS.
- **Serialization:** Protocol Buffers for compact encoding of embedding vectors. A 768-dim float32 embedding compresses to 3KB with quantization.

```
import redis
import numpy as np
from typing import Optional

class NLPFeatureStore:
    def __init__(self, redis_cluster_nodes):
        self.redis = redis.RedisCluster(
            startup_nodes=redis_cluster_nodes,
            decode_responses=False
        )
        self.local_cache = {}  # LRU cache

    def get_embedding(self, entity_id: str) ->
        Optional[np.ndarray]:
        # L1: in-process cache
        if entity_id in self.local_cache:
```

```
            return self.local_cache[entity_id]
        # L2: Redis cluster
        data = self.redis.get(f"emb:{entity_id}")
        if data:
            embedding = np.frombuffer(data,
                dtype=np.float32)
            self.local_cache[entity_id] = embedding
            return embedding
        return None  # Cache miss -> fallback to DB
```

Performance: P99 latency ¡ 8ms at 100K QPS. Redis Cluster with 6 shards provides sufficient throughput with 2x replication for fault tolerance.

Q3: Design an A/B Testing Framework for NLP Models

System Design Focus: **Data Consistency** **Observability**
How would you design an A/B testing platform that allows data scientists to safely experiment with different NLP models in production while maintaining statistical rigor?

Architecture:
The framework consists of four services: an *Experiment Manager* (defines experiments), a *Traffic Router* (assigns users to variants), a *Metrics Collector* (captures outcomes), and a *Statistical Engine* (analyzes results).
Traffic Assignment: Use deterministic hashing (MurmurHash3 of user ID + experiment ID) to ensure consistent assignment. A user always sees the same variant, even across sessions. This avoids the consistency problem of random assignment.
Guardrails:

- Automatic rollback if error rate exceeds baseline by ¿2%.
- Gradual ramp-up: 1% → 5% → 25% → 50%.
- Isolation between experiments using traffic slicing to avoid interaction effects.

Statistical Rigor: Pre-register hypotheses and sample sizes. Use sequential testing (not fixed-horizon) to allow early stopping. Apply Bonferroni correction for multiple comparisons.

```
import hashlib
```

```python
from scipy import stats

def assign_variant(user_id: str, experiment_id: str,
                   num_variants: int = 2) -> int:
    hash_input = f"{user_id}:{experiment_id}"
    hash_val = int(hashlib.md5(
        hash_input.encode()).hexdigest(), 16)
    return hash_val % num_variants

def check_significance(control, treatment, alpha=0.05):
    t_stat, p_value = stats.ttest_ind(control, treatment)
    return {
        "significant": p_value < alpha,
        "p_value": round(p_value, 4),
        "effect_size": (np.mean(treatment) -
           np.mean(control))
                       / np.std(control)
    }
```

Observability: Every experiment logs assignment events, model predictions, and user interactions to a unified event stream (Kafka). A real-time dashboard shows key metrics per variant with confidence intervals.

Q4: Design a GPU Resource Scheduler for NLP Workloads

System Design Focus: Cost Management Scalability
Design a system that efficiently schedules NLP training and inference jobs across a shared GPU cluster, optimizing for utilization and cost.

Architecture:
Build a priority-based scheduler with three queues: *Critical* (production inference), *Standard* (scheduled training), and *Best-Effort* (experiments, fine-tuning). The scheduler implements bin-packing to maximize GPU utilization.
Key Components:

- **Resource Broker:** Tracks GPU memory, compute utilization, and network bandwidth across all nodes. Maintains a real-time resource graph.
- **Job Profiler:** Estimates resource requirements based on model architecture. A 1B parameter model in float16 needs 2GB GPU memory for inference; training

needs 6x that.

- **Preemption Manager:** Best-effort jobs can be preempted (with checkpoint) when critical jobs arrive. Preempted jobs resume automatically when resources free up.
- **Auto-Scaler:** Monitors queue depth and scales the cluster using cloud APIs. Spot instances for best-effort, on-demand for critical.

```python
class GPUScheduler:
    def __init__(self):
        self.queues = {
            "critical": PriorityQueue(),
            "standard": PriorityQueue(),
            "best_effort": PriorityQueue()
        }
        self.gpu_nodes = {}  # node_id -> GPUNode

    def schedule(self, job):
        required_gpus = self.estimate_resources(job)
        available =
           self.find_available_nodes(required_gpus)
        if available:
            self.assign(job, available)
        elif job.priority == "critical":
            self.preempt_and_assign(job, required_gpus)
        else:
            self.queues[job.priority].put(job)

    def estimate_resources(self, job):
        param_bytes = job.model_params * 2  # float16
        if job.type == "training":
            return param_bytes * 6  # gradients +
               optimizer
        return param_bytes  # inference only
```

Cost Optimization: GPU utilization target ¿ 80%. Multi-tenancy with memory isolation using NVIDIA MPS. Time-of-day pricing awareness: schedule large training jobs during off-peak hours.

Q5: Design a Model Registry and Versioning System

System Design Focus: **Data Consistency** **High Availability**
Design a model registry that tracks all NLP model versions, their lineage, performance metrics, and deployment status across the organization.

Architecture:
The model registry is a metadata service backed by PostgreSQL for structured metadata (versions, metrics, lineage) and object storage (S3) for model artifacts. It provides a REST API and CLI for model lifecycle management.
Data Model:

- **Model:** name, owner, description, task type
- **Version:** model ID, version number, artifact URI, training config, metrics (accuracy, F1, latency), stage (dev/staging/prod)
- **Lineage:** parent model, training dataset hash, preprocessing pipeline version, hyperparameters

Consistency Guarantees: Stage transitions (dev → staging → prod) use database transactions with optimistic locking. Only one version can be in "prod" stage per model at any time. Promotion requires passing automated validation gates.
High Availability: PostgreSQL with streaming replication (primary + 2 read replicas). Model artifacts on S3 with cross-region replication. The API layer runs behind a load balancer with 3 instances.

```
class ModelRegistry:
    def register_model(self, name, artifact_path,
        metrics):
         version = self.get_next_version(name)
         artifact_uri = self.upload_artifact(artifact_path)
         # Atomic registration with lineage
         with self.db.transaction():
             model_version = ModelVersion(
                 model_name=name,
                 version=version,
                 artifact_uri=artifact_uri,
                 metrics=metrics,
                 stage="development",
                 created_at=datetime.utcnow()
             )
             self.db.insert(model_version)
```

```
        return model_version

    def promote(self, name, version, target_stage):
        with self.db.transaction():
            # Demote current prod version
            self.db.update(name, stage=target_stage,
                           set_stage="archived")
            # Promote new version
            self.db.update(name, version=version,
                           set_stage=target_stage)
```

Q6: Design a Data Pipeline for Training Corpus Management

System Design Focus: **Data Storage & Retrieval** **Scalability**
Design a system that ingests, deduplicates, cleans, and versions terabyte-scale text corpora used for training NLP models.

Architecture:
A multi-stage pipeline: **Ingestion** → **Deduplication** → **Cleaning** → **Quality Scoring** → **Versioned Storage**.
Ingestion Layer: Web crawlers, API connectors, and file importers push raw documents into a staging area (S3 raw bucket). Each document gets a UUID and metadata envelope (source, timestamp, language, content hash).
Deduplication: Two-phase approach. First, exact dedup using SHA-256 content hashes. Second, near-duplicate detection using MinHash LSH (Locality-Sensitive Hashing) with Jaccard similarity threshold of 0.8. This runs as a distributed Spark job.
Cleaning Pipeline: Language detection (fastText lid), encoding normalization, HTML/boilerplate removal, PII redaction (regex + NER-based), and toxicity filtering. Each stage is a separate microservice connected via Kafka topics.
Versioning: Use DVC (Data Version Control) or Delta Lake for dataset versioning. Each training run references an immutable dataset snapshot by version hash.

```
from datasketch import MinHash, MinHashLSH

def build_minhash(text, num_perm=128):
    mh = MinHash(num_perm=num_perm)
```

```
    for word in text.lower().split():
        mh.update(word.encode('utf-8'))
    return mh

def deduplicate_corpus(documents, threshold=0.8):
    lsh = MinHashLSH(threshold=threshold, num_perm=128)
    unique_docs = []
    for doc_id, text in documents:
        mh = build_minhash(text)
        if not lsh.query(mh):  # No near-duplicates found
            lsh.insert(doc_id, mh)
            unique_docs.append((doc_id, text))
    return unique_docs
```

Scale: Processing 10TB of text requires 500 Spark executors with MinHash. The dedup index fits in memory with 128-bit signatures per document (16 bytes per doc for 1B documents = 16GB).

Q7: Design a Monitoring System for NLP Model Performance Drift

System Design Focus: **Observability** **High Availability**
Design a monitoring system that detects when deployed NLP models begin to degrade in production due to data drift, concept drift, or infrastructure issues.

Architecture:
Three monitoring layers: **Infrastructure Metrics** (latency, throughput, errors), **Data Distribution Monitoring** (input drift detection), and **Model Quality Monitoring** (output quality tracking).
Data Drift Detection: Sample incoming requests and compare feature distributions against a reference window (the validation set from training). Use Population Stability Index (PSI) for numerical features and Jensen-Shannon divergence for embedding distributions.
Concept Drift Detection: Track model confidence distributions over time. A shift in the confidence histogram signals that the relationship between inputs and outputs has changed. Use ADWIN (Adaptive Windowing) for streaming drift detection.
Alert System: Three severity levels. *Warning* (PSI ¿ 0.1): investigate within

24h. *Critical* (PSI ¿ 0.25 or error rate spike): page on-call. *Emergency* (model returning errors): automatic fallback to previous version.

```python
import numpy as np
from scipy.spatial.distance import jensenshannon

class DriftDetector:
    def __init__(self, reference_distribution):
        self.reference = reference_distribution

    def compute_psi(self, current, bins=10):
        ref_hist, edges = np.histogram(
            self.reference, bins=bins, density=True)
        curr_hist, _ = np.histogram(
            current, bins=edges, density=True)
        # Avoid division by zero
        ref_hist = np.clip(ref_hist, 1e-6, None)
        curr_hist = np.clip(curr_hist, 1e-6, None)
        psi = np.sum(
            (curr_hist - ref_hist) * np.log(
                curr_hist / ref_hist))
        return psi

    def check_embedding_drift(self, current_embeddings):
        js_div = jensenshannon(
            self.reference.mean(axis=0),
            current_embeddings.mean(axis=0))
        return {"js_divergence": js_div,
                "drifted": js_div > 0.15}
```

High Availability: The monitoring system itself must be highly available. Run as a stateless service behind a load balancer. Metric storage in a time-series database (InfluxDB/Prometheus) with 30-day retention and downsampled long-term storage.

Q8: Design a Hyperparameter Optimization Service

System Design Focus: Scalability Cost Management
Design a scalable service that automatically tunes hyperparameters for NLP models, managing hundreds of concurrent experiments efficiently.

Architecture:
A central **Optimization Server** coordinates trials. It maintains a search space definition, selects the next hyperparameter configuration using Bayesian optimization (Tree-structured Parzen Estimators or Gaussian Processes), and dispatches trial jobs to the GPU cluster.

Key Design Decisions:

- **Search Strategy:** Start with random search for initial exploration (first 20 trials), then switch to Bayesian optimization. For NLP, key hyperparameters include learning rate, batch size, warmup steps, dropout rate, and weight decay.
- **Early Stopping:** Use Successive Halving or Hyperband to terminate underperforming trials early. Evaluate at 10%, 25%, 50%, 100% of epochs. This reduces compute costs by 3-5x.
- **Trial Management:** Each trial is a Kubernetes job. Trials report intermediate metrics via gRPC. Failed trials are retried once, then marked as failed.
- **Results Database:** All trial configurations and results are stored in PostgreSQL for analysis and reproducibility.

```python
import optuna

def objective(trial):
    lr = trial.suggest_float("lr", 1e-5, 1e-3, log=True)
    batch_size = trial.suggest_categorical(
        "batch_size", [16, 32, 64])
    warmup_ratio = trial.suggest_float(
        "warmup_ratio", 0.0, 0.2)
    dropout = trial.suggest_float("dropout", 0.0, 0.5)

    model = train_nlp_model(
        lr=lr, batch_size=batch_size,
        warmup_ratio=warmup_ratio, dropout=dropout
    )
    f1_score = evaluate(model, val_dataset)
    return f1_score

# Distributed optimization across workers
study = optuna.create_study(
    direction="maximize",
    storage="postgresql://optuna:pw@db/optuna",
    study_name="bert_ner_tuning",
    pruner=optuna.pruners.HyperbandPruner()
)
```

```
study.optimize(objective, n_trials=200, n_jobs=10)
```

Cost: A 200-trial HPO run for a BERT-base model costs approximately $500-$1000 on cloud GPUs. With Hyperband pruning, this drops to $150-$300.

Q9: Design a Secure Multi-Tenant NLP Platform

System Design Focus: **Security** **High Availability**
Design an NLP-as-a-Service platform where multiple teams within an organization can train and deploy models with proper isolation, access control, and data security.

Architecture:
A multi-tenant platform with namespace isolation per team. Each team gets a logical workspace with its own resource quotas, model registry, and data store, running on shared infrastructure.
Security Layers:

- **Authentication:** OAuth 2.0 / OIDC integration with corporate IdP. Service-to-service auth via mTLS with short-lived certificates.
- **Authorization:** RBAC with roles: Admin, ML Engineer, Data Scientist, Viewer. Fine-grained permissions on models, datasets, and endpoints.
- **Data Isolation:** Each tenant's training data is encrypted at rest with tenant-specific KMS keys. Network policies enforce namespace isolation in Kubernetes.
- **Audit Logging:** Every API call, model access, and data operation is logged to an immutable audit trail.

```
from functools import wraps

def require_permission(resource_type, action):
    def decorator(func):
        @wraps(func)
        def wrapper(request, *args, **kwargs):
            user = authenticate(request.token)
            tenant = resolve_tenant(request)
            if not authorize(user, tenant,
                             resource_type, action):
                raise PermissionDenied(
                    f"User {user.id} cannot {action} "
                    f"{resource_type} in {tenant.name}")
```

```
            audit_log(user, tenant, resource_type, action)
            return func(request, *args, **kwargs)
        return wrapper
    return decorator

@require_permission("model", "deploy")
def deploy_model(request, model_id, version):
    # Only executes if user has deploy permission
    endpoint = serving_service.deploy(
        model_id, version,
        namespace=request.tenant.namespace)
    return endpoint
```

High Availability: Control plane runs across 3 AZs. Each tenant's inference endpoints have independent auto-scaling policies. Blast radius is limited: one tenant's outage does not affect others.

Q10: Design an Automated ML Pipeline for Continuous NLP Model Retraining

System Design Focus: Communication Between Services Observability
Design an end-to-end MLOps pipeline that automatically retrains NLP models when performance degrades or new data becomes available.

Architecture:
An event-driven pipeline with five stages: **Trigger** → **Data Preparation** → **Training** → **Validation** → **Deployment**. Each stage is a separate service communicating through an event bus (Kafka).
Trigger Conditions:

- **Performance-based:** Drift detector signals model degradation (PSI ¿ 0.2).
- **Data-based:** New labeled data exceeds a threshold (e.g., 10K new samples).
- **Scheduled:** Weekly retraining regardless of triggers.

Pipeline Orchestration: Use Apache Airflow or Kubeflow Pipelines. Each step is idempotent and retryable. The pipeline publishes events at each stage transition, enabling downstream services (monitoring, notifications) to react.
Validation Gates: Before deployment, the new model must beat the current production model on a held-out test set by a statistically significant margin.

Shadow deployment (running both models on live traffic) provides additional confidence.

```
# Airflow DAG for NLP retraining pipeline
from airflow import DAG
from airflow.operators.python import PythonOperator
from datetime import timedelta

dag = DAG("nlp_retrain",
          schedule_interval=timedelta(weeks=1),
          catchup=False)

prepare = PythonOperator(
    task_id="prepare_data",
    python_callable=prepare_training_data,
    dag=dag)

train = PythonOperator(
    task_id="train_model",
    python_callable=train_model,
    dag=dag)

validate = PythonOperator(
    task_id="validate_model",
    python_callable=run_validation_suite,
    dag=dag)

deploy = PythonOperator(
    task_id="deploy_canary",
    python_callable=canary_deploy,
    dag=dag)

prepare >> train >> validate >> deploy
```

Communication: Kafka topics per stage (data.prepared, model.trained, model.validated, model.deployed). Services consume events asynchronously. Dead letter queues capture failed messages for debugging.
Observability: Distributed tracing (Jaeger) across the full pipeline. Each run produces a lineage graph showing data version → model version → deployment version.

2 Text Preprocessing at Scale

linkedin.com/in/lamhotsiagian

Raw text is messy. Before any NLP model can work with language, the text must be cleaned, normalized, and transformed into a consistent format. At production scale, preprocessing is not a one-off script—it is a distributed pipeline that must handle billions of documents reliably, consistently, and efficiently.

2.1 Chapter Overview

Text preprocessing is the unsung hero of NLP systems. At FAANG companies processing billions of text documents daily—user posts, search queries, customer support tickets, comments—the preprocessing pipeline is often the most critical (and most fragile) component.

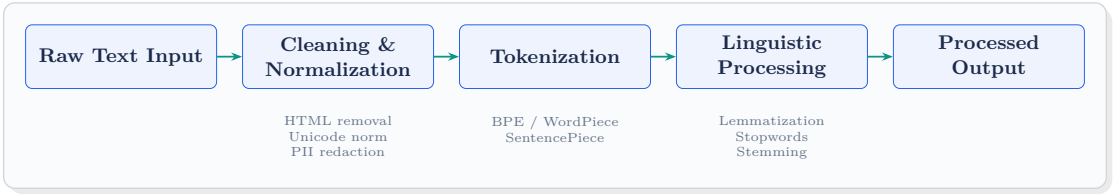
The core preprocessing operations form a standard pipeline:

Tokenization is the foundation—splitting text into meaningful units (words, subwords, or characters). Modern NLP overwhelmingly uses subword tokenization (BPE, WordPiece, SentencePiece) because it handles out-of-vocabulary words gracefully and keeps vocabulary sizes manageable.

Normalization standardizes text: lowercasing, Unicode normalization (NFC/NFKC), accent removal, and handling special characters. The choice of normalization depends heavily on the downstream task—sentiment analysis may want to preserve case ("AMAZING" signals intensity), while search needs case-insensitive matching.

Cleaning removes noise: HTML tags, URLs, email addresses, excessive whitespace, and encoding artifacts. PII (Personally Identifiable Information) redaction is increasingly a legal requirement, not just a best practice.

Linguistic Processing includes stemming (crude suffix stripping), lemmatization (reducing to dictionary form using morphological analysis), stopword removal, and part-of-speech-aware filtering.



Production Reality

At scale, preprocessing consistency is paramount. If your training pipeline lowercases text but your inference pipeline does not, your model will silently degrade. Version your preprocessing logic alongside your models.

2.2 System Design Interview Questions

Q1: Design a Real-Time Text Preprocessing Service for 1M Requests/Second

System Design Focus: Scalability Latency & Performance
Design a preprocessing service that normalizes, tokenizes, and cleans user-generated text in real time at 1 million requests per second with P99 latency under 20ms.

Architecture:
A stateless preprocessing service deployed as a horizontally scaled fleet behind a load balancer. Each instance runs preprocessing in-process with no external dependencies on the hot path.
Key Design:

- **Stateless Workers:** Each pod handles 50K RPS. Need 20 pods with headroom. Auto-scale based on CPU utilization (target 60%).
- **In-Memory Tokenizer:** Load the tokenizer vocabulary (BPE merge table) into memory at startup. Tokenization is CPU-bound, not I/O-bound, so no need for external calls.
- **Pipeline Optimization:** Fuse operations—normalize and tokenize in a single pass rather than multiple string copies. Use Rust/C++ for the hot path with

Python bindings.

- **Batching:** Accept batch requests (up to 64 texts) to amortize HTTP overhead and enable SIMD-friendly processing.

```python
from fastapi import FastAPI
from tokenizers import Tokenizer
import re, unicodedata

app = FastAPI()
tokenizer = Tokenizer.from_file("bpe_model.json")

# Compiled regex for speed
URL_RE = re.compile(r'https?://\S+')
HTML_RE = re.compile(r'<[^>]+>')

def preprocess(text: str) -> dict:
    # Fused normalization pass
    text = unicodedata.normalize("NFKC", text)
    text = HTML_RE.sub("", text)
    text = URL_RE.sub("[URL]", text)
    text = text.lower().strip()
    # Tokenize
    encoding = tokenizer.encode(text)
    return {"tokens": encoding.ids,
            "text": encoding.tokens}

@app.post("/preprocess/batch")
async def batch_preprocess(texts: list[str]):
    return [preprocess(t) for t in texts[:64]]
```

Performance: Single-text latency 2ms. Batch of 64 texts 15ms. P99 under 20ms achieved by keeping the hot path allocation-free and using pre-compiled regex patterns.

Q2: Design a Distributed Tokenizer Training Pipeline

System Design Focus: **Data Storage & Retrieval** **Scalability**

Design a system to train a BPE tokenizer on a 500GB text corpus, ensuring the resulting tokenizer handles multilingual text and rare domain-specific terminology.

Architecture:
Tokenizer training requires computing token frequencies across the entire corpus and iteratively merging the most frequent pairs. For a 500GB corpus, this must be distributed.
Pipeline Stages:

- **Stage 1 – Sampling:** For BPE, you do not always need the full corpus. Stratified sampling (by language, domain, source) to create a representative 50GB subset reduces compute by 10x with minimal quality loss.
- **Stage 2 – Character Frequency:** MapReduce over the corpus to compute character and character-pair frequencies. Map phase: emit (char_pair, count) per document. Reduce phase: aggregate counts.
- **Stage 3 – Iterative Merging:** BPE merges run sequentially (each merge depends on the previous), but frequency counting within each iteration is parallelizable.
- **Stage 4 – Validation:** Evaluate tokenizer fertility (tokens per word) across language families. Target: 1.2-1.5 for English, 1.5-2.0 for morphologically rich languages.

```python
from tokenizers import Tokenizer
from tokenizers.models import BPE
from tokenizers.trainers import BpeTrainer
from tokenizers.pre_tokenizers import Whitespace

def train_tokenizer(corpus_files, vocab_size=50000):
    tokenizer = Tokenizer(BPE(unk_token="[UNK]"))
    tokenizer.pre_tokenizer = Whitespace()
    trainer = BpeTrainer(
        vocab_size=vocab_size,
        special_tokens=["[UNK]", "[PAD]", "[CLS]",
                        "[SEP]", "[MASK]"],
        min_frequency=2,
        show_progress=True
    )
    tokenizer.train(corpus_files, trainer)
    # Evaluate fertility
    test_text = "The quick brown fox jumps"
    tokens = tokenizer.encode(test_text).tokens
    fertility = len(tokens) / len(test_text.split())
    print(f"Fertility: {fertility:.2f}")
    tokenizer.save("trained_tokenizer.json")
    return tokenizer
```

Storage: Raw corpus on S3 in sharded Parquet files. Intermediate frequency tables in a distributed key-value store. Final tokenizer artifact (2MB) versioned in the model registry.

Q3: Design a PII Detection and Redaction Pipeline

System Design Focus: Security Latency & Performance
Design a system that detects and redacts personally identifiable information from text data before it enters the NLP training pipeline, handling 100K documents per hour.

Architecture:
A multi-layer detection approach combining rule-based patterns (high precision for structured PII) with ML-based NER (for unstructured PII like names in context).
Detection Layers:

- **Layer 1 – Regex Patterns:** Phone numbers, emails, SSNs, credit card numbers, IP addresses. These have predictable formats and can be caught with high-precision regex.
- **Layer 2 – NER Model:** A fine-tuned BERT model detects person names, addresses, organization names, and medical terms in context. Run as a batch inference job.
- **Layer 3 – Contextual Analysis:** Patterns like "my name is [X]" or "I live at [X]" increase confidence that surrounding tokens are PII.

Redaction Strategy: Replace PII with type-preserving tokens: "John Smith" → "[PERSON_1]", "555-1234" → "[PHONE_1]". This preserves text structure for downstream NLP tasks while removing sensitive data.

```
import re
from presidio_analyzer import AnalyzerEngine
from presidio_anonymizer import AnonymizerEngine

class PIIRedactor:
    def __init__(self):
        self.analyzer = AnalyzerEngine()
        self.anonymizer = AnonymizerEngine()
        # High-precision regex patterns
        self.patterns = {
            "SSN": re.compile(
```

```
                r'\b\d{3}-\d{2}-\d{4}\b'),
            "CREDIT_CARD": re.compile(
                r'\b\d{4}[\s-]?\d{4}[\s-]?\d{4}'
                r'[\s-]?\d{4}\b'),
        }

    def redact(self, text: str) -> str:
        # Layer 1: Regex-based detection
        for pii_type, pattern in self.patterns.items():
            text = pattern.sub(
                f"[{pii_type}]", text)
        # Layer 2: ML-based NER detection
        results = self.analyzer.analyze(
            text=text, language="en")
        anonymized = self.anonymizer.anonymize(
            text=text, analyzer_results=results)
        return anonymized.text
```

Throughput: Regex layer processes 10K docs/sec per core. NER layer processes 200 docs/sec per GPU. Pipeline achieves 100K docs/hour with 2 GPU instances handling the NER bottleneck while CPU workers handle regex pre-filtering.

Q4: Design a Language Detection Service for Multilingual Content

System Design Focus: **High Availability** **Scalability**
Design a language detection system that identifies the language of incoming text across 100+ languages with 99% accuracy, used as a routing layer for downstream language-specific NLP models.

Architecture:
A lightweight, high-throughput classification service. Language detection is a solved problem at the model level (fastText's lid.176 achieves ¿97% accuracy), so the challenge is purely system design: availability, latency, and handling edge cases.
Service Design:

- **Primary Model:** fastText lid model (1MB) loaded in-memory. Classification takes ¡1ms per text.
- **Confidence Routing:** High confidence (¿0.8): route directly. Medium (0.5-

0.8): use a secondary model (CLD3 or a fine-tuned transformer) for verification. Low (¡0.5): flag for human review or default to "unknown."

- **Code-Switching Detection:** For texts mixing languages (common in social media), split into sentences and classify each. Return a distribution: {"en": 0.6, "es": 0.4}.

High Availability: Deploy across 3 AZs with DNS failover. The model is small enough to run on CPU-only instances (no GPU dependency). Health checks every 5 seconds with automatic replacement of unhealthy instances.

```python
import fasttext

class LanguageDetector:
    def __init__(self, model_path="lid.176.bin"):
        self.model = fasttext.load_model(model_path)
        self.confidence_threshold = 0.8

    def detect(self, text: str) -> dict:
        text = text.replace("\n", " ").strip()
        predictions = self.model.predict(
            text, k=3)  # Top 3 languages
        results = []
        for label, score in zip(*predictions):
            lang = label.replace("__label__", "")
            results.append({"lang": lang,
                            "confidence": round(score, 3)})
        top = results[0]
        return {
            "primary_language": top["lang"],
            "confidence": top["confidence"],
            "needs_review": top["confidence"]
                            < self.confidence_threshold,
            "all_predictions": results
        }
```

Scale: Each instance handles 100K classifications/sec. For 1M RPS, 10 instances plus 5 standby for failover. Total cost: $2K/month on cloud compute.

Q5: Design a Text Normalization Pipeline for Search

System Design Focus: **Data Consistency** **Latency & Performance**
Design a text normalization system for a search engine that ensures queries and documents are processed identically, supporting Unicode, abbreviations, and domain-specific terminology.

Architecture:
A shared normalization library used by both the indexing pipeline (offline) and the query processing service (online). Consistency between these two paths is the single most important requirement—if they diverge, search quality degrades silently.

Normalization Chain:

- **Unicode Normalization:** NFKC form to collapse equivalent representations (e.g., "fi" ligature → "fi").
- **Case Folding:** Locale-aware lowercasing (Turkish "I" → "ı" not "i").
- **Synonym Expansion:** "NYC" → "New York City." Maintained as a versioned dictionary. Applied at index time and query time.
- **Accent/Diacritic Handling:** "café" → "cafe" for matching, but preserve originals for display.

Consistency Strategy: Package normalization as a versioned library. Both indexing and query services depend on the same version. Version bumps trigger re-indexing. Use a canary: normalize 1000 sample queries with old and new versions; diff the outputs.

```python
import unicodedata
import re

class SearchNormalizer:
    VERSION = "2.3.1"  # Version-lock with index

    def __init__(self, synonyms_path):
        self.synonyms = self._load_synonyms(
            synonyms_path)

    def normalize(self, text: str) -> str:
        # Step 1: Unicode NFKC
        text = unicodedata.normalize("NFKC", text)
        # Step 2: Lowercase (locale-aware)
        text = text.lower()
```

```
        # Step 3: Remove diacritics
        text = ''.join(
            c for c in unicodedata.normalize('NFD', text)
            if unicodedata.category(c) != 'Mn')
        # Step 4: Synonym expansion
        tokens = text.split()
        tokens = [self.synonyms.get(t, t)
                  for t in tokens]
        # Step 5: Whitespace normalization
        return ' '.join(tokens)

    def _load_synonyms(self, path):
        # Load versioned synonym dictionary
        synonyms = {}
        with open(path) as f:
            for line in f:
                src, tgt = line.strip().split('\t')
                synonyms[src] = tgt
        return synonyms
```

Data Consistency: Every indexed document stores the normalizer version used. A background job detects version mismatches and re-indexes stale documents. During transition, both old and new normalizations are searchable.

Q6: Design a Streaming Text Cleaning Pipeline for Social Media Data

System Design Focus: Communication Between Services Scalability
Design a real-time pipeline that ingests, cleans, and enriches social media text (tweets, posts, comments) at 500K events per second for downstream analytics and NLP models.

Architecture:
An event-driven streaming architecture using Kafka as the backbone. Each cleaning stage is a separate consumer group, enabling independent scaling and fault isolation.
Pipeline Stages:

- **Ingestion:** Kafka topic "raw_posts" receives events from social media API connectors.

- **Stage 1 – Deduplication:** Bloom filter (probabilistic, 1% false positive) removes exact duplicates. Redis-backed for near-duplicates using SimHash.
- **Stage 2 – Cleaning:** Remove URLs, mentions, hashtag symbols (keep text), emojis (optionally map to text), and fix encoding issues.
- **Stage 3 – Enrichment:** Language detection, sentiment pre-scoring, spam probability, and bot detection score.
- **Output:** Cleaned events published to "cleaned_posts" topic for downstream consumers.

```python
from kafka import KafkaConsumer, KafkaProducer
import json, re

class TextCleaningWorker:
    def __init__(self):
        self.consumer = KafkaConsumer(
            'raw_posts',
            group_id='text_cleaners',
            auto_offset_reset='latest')
        self.producer = KafkaProducer(
            value_serializer=lambda v:
                json.dumps(v).encode())

    def clean_social_text(self, text):
        text = re.sub(r'@\w+', '[USER]', text)
        text = re.sub(r'#(\w+)', r'\1', text)
        text = re.sub(r'https?://\S+', '[URL]', text)
        text = re.sub(r'\s+', ' ', text).strip()
        return text

    def process(self):
        for message in self.consumer:
            post = json.loads(message.value)
            post['cleaned_text'] = self.clean_social_text(
                post['text'])
            self.producer.send('cleaned_posts', post)
```

Scalability: Kafka topic with 128 partitions. Each consumer instance handles 5K events/sec. Need 100 consumers for 500K/sec. Auto-scale consumer groups based on consumer lag.
Ordering: Partition by user ID to maintain per-user ordering. Cross-user ordering is not required for analytics use cases.

Q7: Design a Text Encoding and Character Set Management System

System Design Focus: **Data Consistency** **Data Storage & Retrieval**
Design a system that handles text encoding consistently across a global platform, supporting 50+ languages with different character sets (Latin, CJK, Arabic, Devanagari) without data corruption.

Architecture:
Enforce UTF-8 as the canonical encoding throughout the system. All conversion happens at ingestion boundaries. Internal services never deal with encoding ambiguity.
Key Design:

- **Ingestion Boundary:** Detect encoding using chardet/cchardet for legacy inputs. Convert to UTF-8 immediately. Reject invalid sequences rather than silently replacing characters.
- **Storage:** All databases configured with UTF-8/utf8mb4 collation (utf8mb4 in MySQL to handle 4-byte emoji). Column-level validation rejects non-UTF-8 inserts.
- **API Contracts:** All APIs accept and return UTF-8. Content-Type headers enforced. JSON encoding with ensure_ascii=False for proper Unicode in responses.
- **CJK Handling:** Chinese/Japanese text requires specialized tokenization (character-based or jieba/MeCab). Storage needs to account for 3-4 bytes per character vs. 1 byte for Latin.

```
import chardet

class EncodingNormalizer:
    def __init__(self):
        self.target_encoding = "utf-8"
        self.fallback_encodings = [
            "utf-8", "latin-1", "shift_jis",
            "gb2312", "euc-kr"]

    def normalize(self, raw_bytes: bytes) -> str:
        # Auto-detect encoding
        detected = chardet.detect(raw_bytes)
        encoding = detected['encoding']
```

```
        confidence = detected['confidence']

        if confidence > 0.8 and encoding:
            try:
                return raw_bytes.decode(encoding)
            except (UnicodeDecodeError, LookupError):
                pass

        # Fallback: try common encodings
        for enc in self.fallback_encodings:
            try:
                return raw_bytes.decode(enc)
            except UnicodeDecodeError:
                continue

        # Last resort: decode with replacement
        return raw_bytes.decode('utf-8',
                                errors='replace')
```

Consistency: Every text field in the database has a CHECK constraint validating UTF-8. ETL pipelines include an encoding validation step that catches corruption before it propagates downstream.

Q8: Design a Regex-Based Rule Engine for Text Classification

System Design Focus: Latency & Performance Observability
Design a rule engine that applies thousands of regex-based classification rules to incoming text in real time, used for content moderation and routing.

Architecture:
A compiled regex engine that batches and optimizes thousands of patterns into a finite automaton for fast matching. Individual regex evaluation does not scale past hundreds of patterns.
Key Design:

- **Pattern Compilation:** Use Aho-Corasick algorithm for literal string matching (O(n + m) where n is text length, m is total matches). For regex patterns, compile into a DFA using RE2 or Hyperscan.
- **Rule Management:** Rules stored in a database with versioning. Rule authors

define patterns via a UI. Changes go through a review pipeline (test against a sample corpus before activation).

- **Priority System:** Rules have priorities. First-match-wins for classification; all-matches for tagging. Conflicts resolved by rule priority.

```python
import ahocorasick
from dataclasses import dataclass

@dataclass
class Rule:
    id: str
    pattern: str
    category: str
    priority: int

class RuleEngine:
    def __init__(self, rules: list[Rule]):
        self.automaton = ahocorasick.Automaton()
        for rule in rules:
            self.automaton.add_word(
                rule.pattern.lower(),
                (rule.id, rule.category, rule.priority))
        self.automaton.make_automaton()

    def classify(self, text: str) -> list[dict]:
        matches = []
        for end_idx, (rule_id, cat, pri) in \
                self.automaton.iter(text.lower()):
            matches.append({
                "rule_id": rule_id,
                "category": cat,
                "priority": pri,
                "position": end_idx})
        # Sort by priority, return highest
        matches.sort(key=lambda x: x["priority"],
                     reverse=True)
        return matches
```

Performance: Aho-Corasick matches 10,000+ patterns against a text string in O(text_length) time. Processing 50K texts/sec on a single core with 5,000 active rules.

Observability: Log every rule match with rule ID, text hash, and confidence.

Dashboard shows per-rule hit rate, false positive rate (from feedback), and latency distribution.

Q9: Design a Data Annotation Platform for NLP Tasks

System Design Focus: Database Bottlenecks Security
Design a platform where annotators label text data (NER tags, sentiment, intent classification) with quality control, inter-annotator agreement tracking, and secure data handling.

Architecture:
A web-based platform with a task management backend, an annotation interface, and a quality assurance layer.
Core Components:

- **Task Manager:** Distributes annotation tasks to annotators based on skill, language, and availability. Implements overlap (each item labeled by 2-3 annotators) for quality measurement.
- **Annotation Store:** PostgreSQL for annotations (annotator, item, label, timestamp). Each annotation is immutable—edits create new versions. Handles 10M annotations with proper indexing.
- **Agreement Engine:** Computes inter-annotator agreement (Cohen's Kappa for 2 annotators, Fleiss' Kappa for 3+) in real time. Flags items with low agreement for adjudication.
- **Active Learning Loop:** Prioritize annotating items where the current model is most uncertain, maximizing label efficiency.

```
from sklearn.metrics import cohen_kappa_score
import numpy as np

class QualityController:
    def compute_agreement(self, annotations):
        """Compute inter-annotator agreement."""
        # Group by item
        items = {}
        for ann in annotations:
            items.setdefault(
                ann['item_id'], []).append(ann['label'])

```

```
        # Compute pairwise kappa
        annotator_pairs = []
        for item_id, labels in items.items():
            if len(labels) >= 2:
                annotator_pairs.append(labels[:2])

        if not annotator_pairs:
            return None
        a1 = [p[0] for p in annotator_pairs]
        a2 = [p[1] for p in annotator_pairs]
        return cohen_kappa_score(a1, a2)

    def flag_disagreements(self, annotations,
                           threshold=0.5):
        flagged = []
        items = {}
        for ann in annotations:
            items.setdefault(
                ann['item_id'], []).append(ann['label'])
        for item_id, labels in items.items():
            agreement = len(set(labels)) == 1
            if not agreement:
                flagged.append(item_id)
        return flagged
```

Security: Annotators access data through the platform only—no downloads. Sensitive data is masked in the UI with only the annotation-relevant portion visible. Role-based access: annotators cannot see other annotators' labels until adjudication.
Database: Partition annotations table by project ID. Index on (item_id, annotator_id). Read replicas serve the agreement dashboard to avoid impacting write throughput.

Q10: Design a Multi-Stage Text Preprocessing Pipeline with Data Lineage

System Design Focus: Observability Communication Between Services
Design a preprocessing pipeline where each transformation is tracked, versioned, and auditable, enabling full data lineage from raw input to model training data.

Architecture:
A DAG-based pipeline where each node is a transformation step. Every step logs its input hash, output hash, transformation version, and configuration, creating an immutable lineage graph.
Lineage Model:

- Each document gets a lineage ID (UUID) at ingestion.
- Each transformation step produces a new version of the document with a parent pointer.
- The lineage graph is stored in a graph database (Neo4j) or a metadata store (Apache Atlas).
- Given any training example, you can trace back to the exact raw document, every transformation applied, and the version of code that applied it.

Pipeline Steps as Services: Each step (cleaning, tokenization, PII redaction, etc.) is a separate service consuming from and producing to Kafka topics. Correlation IDs thread the lineage through the async pipeline.

```python
import hashlib, uuid, json, datetime

class LineageTracker:
    def __init__(self, metadata_store):
        self.store = metadata_store

    def record_transform(self, doc_id, input_text,
                         output_text, step_name,
                         step_version, config):
        record = {
            "doc_id": doc_id,
            "step": step_name,
            "version": step_version,
            "config": config,
            "input_hash": hashlib.sha256(
                input_text.encode()).hexdigest()[:16],
            "output_hash": hashlib.sha256(
                output_text.encode()).hexdigest()[:16],
            "timestamp": datetime.datetime.utcnow()
                         .isoformat(),
            "lineage_id": str(uuid.uuid4())
        }
        self.store.insert(record)
        return record

```

```python
class TrackedPipeline:
    def __init__(self, steps, tracker):
        self.steps = steps  # List of (name, ver, fn)
        self.tracker = tracker

    def process(self, doc_id, text):
        for name, version, transform_fn in self.steps:
            output = transform_fn(text)
            self.tracker.record_transform(
                doc_id, text, output, name,
                version, {})
            text = output
        return text
```

Observability: A lineage dashboard shows the full transformation graph for any document. Aggregate views show how many documents have been processed by each pipeline version, enabling confident rollouts of preprocessing changes.

3 Text Representation & Embeddings at Scale

linkedin.com/in/lamhotsiagian

Machines do not understand words—they understand numbers. Text representation is the bridge: converting human language into numerical vectors that preserve meaning, similarity, and structure. At scale, the choice of representation determines your system's speed, accuracy, and cost.

3.1 Chapter Overview

Text representation has evolved through three generations, each still relevant in production systems today:

Sparse Representations (Bag of Words, TF-IDF) map text to high-dimensional sparse vectors where each dimension corresponds to a vocabulary term. They are simple, interpretable, and fast—TF-IDF powers most production search systems today because it is cheap to compute and index.

Static Embeddings (Word2Vec, GloVe, FastText) map words to dense, low-dimensional vectors (100-300 dims) trained on co-occurrence patterns. The key insight: similar words cluster together in embedding space. FastText adds subword information, handling OOV words gracefully.

Contextual Embeddings (BERT, GPT) produce different vectors for the same word depending on context. "Bank" in "river bank" vs. "bank account" gets different representations. This is the current state of the art but requires GPU inference, making it 100-1000x more expensive than TF-IDF.

The Trade-Off Triangle

In production, you are always balancing three factors: **quality** (contextual embeddings win), **speed** (sparse representations win), and **cost** (TF-IDF is nearly free, BERT embeddings require GPUs). Most production systems use a *cascading* approach: fast sparse retrieval first, then re-rank with dense embeddings.

3.2 System Design Interview Questions

Q1: Design a Large-Scale Embedding Serving Infrastructure

System Design Focus: Latency & Performance Scalability
Design an infrastructure that serves precomputed text embeddings for 1 billion documents with sub-5ms lookup latency and supports real-time embedding generation for new documents.

Architecture:
A dual-path system: a *lookup path* for precomputed embeddings and a *compute path* for on-the-fly embedding generation.
Lookup Path: Precomputed embeddings (768-dim float32 = 3KB each) for 1B documents = 3TB total. Store in a distributed key-value store sharded by document ID.

- **Hot tier:** Redis Cluster for top 100M frequently accessed embeddings (300GB across 10 shards).
- **Warm tier:** RocksDB on SSD for the remaining 900M embeddings. P99 latency 3ms.
- **Batch prefetch:** Predict which embeddings will be needed based on trending topics and prefetch into Redis.

Compute Path: For new/unseen documents, run inference through a distilled BERT model (6 layers instead of 12) on GPU instances. Latency: 10ms per text.

Results are asynchronously written to the warm tier.

```python
import numpy as np
from typing import Optional

class EmbeddingService:
    def __init__(self, redis_client, rocksdb_client,
                 model_service):
        self.redis = redis_client
        self.rocksdb = rocksdb_client
        self.model = model_service

    async def get_embedding(self, doc_id: str,
                            text: Optional[str] = None):
        # L1: Redis (hot)
        emb = await self.redis.get(f"emb:{doc_id}")
        if emb:
            return np.frombuffer(emb, dtype=np.float32)
        # L2: RocksDB (warm)
        emb = self.rocksdb.get(f"emb:{doc_id}")
        if emb:
            await self.redis.setex(
                f"emb:{doc_id}", 3600, emb)
            return np.frombuffer(emb, dtype=np.float32)
        # L3: Compute on-the-fly
        if text:
            emb = await self.model.encode(text)
            await self._store_async(doc_id, emb)
            return emb
        return None
```

Scalability: Shard by consistent hashing on doc ID. Each RocksDB shard handles 100M embeddings on a 500GB SSD. Add shards as the corpus grows.

Q2: Design a Vector Similarity Search System

System Design Focus: **Data Storage & Retrieval** **Latency & Performance**
Design a system that finds the top-K most similar documents to a query embedding from a corpus of 500 million vectors in under 50ms.

Architecture:
Exact nearest neighbor search over 500M vectors is infeasible (brute force: 500M $\times$ 768 multiplications per query). Use Approximate Nearest Neighbor (ANN) search with a hierarchical index.
Index Choice: HNSW (Hierarchical Navigable Small World) graphs provide the best latency-recall trade-off for this scale. Alternative: IVF-PQ (Inverted File Index with Product Quantization) trades recall for lower memory.
System Design:

- **Sharding:** Partition vectors across 50 shards (10M vectors per shard). Each shard maintains its own HNSW index. Query broadcasts to all shards; results merged on the aggregator.
- **Quantization:** Compress 768-dim float32 vectors to 192-dim int8 using Product Quantization. Memory: 500M $\times$ 192B = 96GB (vs. 1.5TB uncompressed).
- **Two-Phase Search:** Phase 1: PQ-compressed search to find top-500 candidates. Phase 2: Re-rank with full-precision vectors.

```python
import faiss
import numpy as np

class VectorSearchEngine:
    def __init__(self, dim=768, n_shards=50):
        self.dim = dim
        self.n_shards = n_shards
        self.indices = []
        for _ in range(n_shards):
            # IVF with PQ compression
            quantizer = faiss.IndexFlatL2(dim)
            index = faiss.IndexIVFPQ(
                quantizer, dim,
                nlist=4096,  # Voronoi cells
                m=48,        # PQ sub-vectors
                nbits=8)     # Bits per sub-vector
            self.indices.append(index)

    def search(self, query_vector, top_k=10):
        query = query_vector.reshape(1, -1)
        all_results = []
        for idx in self.indices:
            distances, ids = idx.search(
                query, top_k)
            all_results.extend(
```

```
                zip(distances[0], ids[0]))
        # Merge and return global top-K
        all_results.sort(key=lambda x: x[0])
        return all_results[:top_k]
```

Performance: HNSW achieves 95%+ recall at 5ms per shard. With 50 shards queried in parallel, end-to-end latency is 15ms (dominated by the slowest shard + merge).

Q3: Design a TF-IDF Based Search Engine for 100M Documents

System Design Focus: Database Bottlenecks Scalability
Design a text search system using TF-IDF scoring that indexes 100 million documents and returns ranked results in under 100ms.

Architecture:
An inverted index maps each term to a list of (document_id, term_frequency) pairs. At query time, compute TF-IDF scores by intersecting posting lists.
Index Structure:

- **Inverted Index:** Term → sorted posting list of (doc_id, tf). Compressed using delta encoding + variable-byte encoding. Total index size: 50-80GB for 100M docs.
- **Sharding:** Document-based sharding across 20 nodes. Each shard indexes 5M docs. Queries broadcast to all shards; a coordinator merges top-K results.
- **Term Dictionary:** B-tree or FST (Finite State Transducer) for prefix lookups. Supports wildcard and fuzzy matching.

Query Processing: For multi-term queries, use WAND (Weak AND) algorithm for early termination—skip documents that cannot make it into the top-K based on upper-bound scores.

```
import math
from collections import defaultdict

class TFIDFIndex:
    def __init__(self):
        self.index = defaultdict(list)  # term->[docs]
        self.doc_count = 0
        self.doc_lengths = {}
```

```
9
10      def add_document(self, doc_id, tokens):
11          self.doc_count += 1
12          tf = defaultdict(int)
13          for token in tokens:
14              tf[token] += 1
15          self.doc_lengths[doc_id] = len(tokens)
16          for term, freq in tf.items():
17              self.index[term].append((doc_id, freq))
18
19      def search(self, query_tokens, top_k=10):
20          scores = defaultdict(float)
21          for term in query_tokens:
22              if term not in self.index:
23                  continue
24              df = len(self.index[term])
25              idf = math.log(self.doc_count / (1 + df))
26              for doc_id, tf in self.index[term]:
27                  tf_norm = tf / self.doc_lengths[doc_id]
28                  scores[doc_id] += tf_norm * idf
29          ranked = sorted(scores.items(),
30                          key=lambda x: -x[1])
31          return ranked[:top_k]
```

Database: Posting lists stored in memory-mapped files for fast access. Term dictionary in a compact FST. Document metadata (title, URL) in a separate column store for fast retrieval of result display data.

Q4: Design an Embedding Model Update Pipeline

System Design Focus: Data Consistency High Availability
When you retrain your embedding model, all stored embeddings become stale. Design a system to update 1 billion embeddings without downtime or inconsistent search results.

Architecture:
A blue-green embedding update strategy. Build a complete new index with the new model in parallel, then switch traffic atomically.
Update Pipeline:

- **Phase 1 – Parallel Re-encoding:** Run the new model over the entire corpus as a batch job. With 100 GPUs processing 1000 docs/sec each, 1B docs takes 3 hours.
- **Phase 2 – Index Building:** Build a new ANN index from the new embeddings. This can run concurrently with re-encoding as embeddings arrive.
- **Phase 3 – Shadow Testing:** Route 5% of queries to both old and new indices. Compare relevance metrics. If quality is maintained or improved, proceed.
- **Phase 4 – Atomic Switchover:** Update the routing layer to point to the new index. The old index remains available for 24h rollback.

Consistency During Transition: During Phase 1-2, new documents arriving are encoded with *both* old and new models and indexed in both old and new indices. This ensures no documents are missing from either index.

```
class EmbeddingUpdateManager:
    def __init__(self):
        self.active_index = "v1"
        self.shadow_index = None

    def start_update(self, new_model_version):
        self.shadow_index = f"v{new_model_version}"
        # Start batch re-encoding job
        self.batch_job = BatchEncoder(
            model_version=new_model_version,
            target_index=self.shadow_index)
        self.batch_job.start()
        # Dual-write for new documents
        self.dual_write_enabled = True

    def switchover(self):
        # Atomic pointer swap
        old_index = self.active_index
        self.active_index = self.shadow_index
        self.shadow_index = old_index
        self.dual_write_enabled = False
        # Keep old index for rollback
        self.schedule_cleanup(old_index,
                              delay_hours=24)
```

High Availability: Zero downtime during the entire process. The switchover is a pointer swap in the config store (ZooKeeper/etcd), taking ¡100ms. Rollback is equally fast.

Q5: Design a Multi-Modal Embedding System (Text + Images)

System Design Focus: Communication Between Services Scalability
Design a system that produces unified embeddings for both text and images, enabling cross-modal search (text query finds relevant images and vice versa).

Architecture:
Use a CLIP-style dual-encoder architecture. A text encoder and an image encoder project both modalities into a shared embedding space where cosine similarity measures cross-modal relevance.

System Components:

- **Text Encoder Service:** Transformer-based, produces 512-dim embeddings. Deployed on GPU instances with TensorRT optimization. Throughput: 2000 texts/sec per GPU.
- **Image Encoder Service:** Vision Transformer (ViT), same 512-dim output space. Throughput: 500 images/sec per GPU. Images preprocessed (resize, normalize) in a CPU preprocessing fleet.
- **Unified Index:** A single vector index stores both text and image embeddings in the same space. Metadata tags indicate modality.
- **API Gateway:** Routes queries to the appropriate encoder based on input type, then searches the unified index.

```python
from transformers import CLIPProcessor, CLIPModel
import torch

class MultiModalEmbedder:
    def __init__(self):
        self.model = CLIPModel.from_pretrained(
            "openai/clip-vit-base-patch32")
        self.processor = CLIPProcessor.from_pretrained(
            "openai/clip-vit-base-patch32")

    def encode_text(self, texts: list[str]):
        inputs = self.processor(
            text=texts, return_tensors="pt",
            padding=True, truncation=True)
        with torch.no_grad():
            embs = self.model.get_text_features(
                **inputs)
        # L2 normalize for cosine similarity
```

```
        return (embs / embs.norm(
            dim=-1, keepdim=True)).numpy()

    def encode_image(self, images):
        inputs = self.processor(
            images=images, return_tensors="pt")
        with torch.no_grad():
            embs = self.model.get_image_features(
                **inputs)
        return (embs / embs.norm(
            dim=-1, keepdim=True)).numpy()
```

Communication: Text and image encoders are separate microservices communicating via gRPC. Batch encoding jobs use Kafka for async processing. The unified index is updated by a single consumer that receives embeddings from both encoder services.

Q6: Design an N-gram Index for Autocomplete and Spell Correction

System Design Focus: Latency & Performance Data Storage & Retrieval
Design an autocomplete system backed by n-gram statistics from a large corpus, returning suggestions in under 30ms as users type.

Architecture:
A trie-based index augmented with n-gram frequency counts. As the user types, the system traverses the trie and returns the highest-frequency completions.
Data Structure:

- **Prefix Trie:** Compressed trie (Patricia trie) stores all unique n-grams. Each leaf stores the frequency count and full text. Memory: 2-4GB for 100M unique n-grams.
- **Top-K Cache:** For each popular prefix (top 1M prefixes), precompute and cache the top-10 completions. Serves 80% of queries from cache.
- **Personalization Layer:** User-specific frequency boosts stored in a small per-user cache. Merge with global suggestions at query time.

```
from collections import defaultdict
import heapq
```

```
3
4  class AutocompleteEngine:
5      def __init__(self):
6          self.trie = {}
7          self.top_k_cache = {}
8
9      def insert(self, phrase, frequency):
10         node = self.trie
11         for char in phrase.lower():
12             node = node.setdefault(char, {})
13         node['_freq'] = frequency
14         node['_phrase'] = phrase
15
16     def suggest(self, prefix, k=10):
17         # Check cache first
18         if prefix in self.top_k_cache:
19             return self.top_k_cache[prefix]
20         # Traverse trie to prefix node
21         node = self.trie
22         for char in prefix.lower():
23             if char not in node:
24                 return []
25             node = node[char]
26         # Collect all completions via DFS
27         results = []
28         self._dfs(node, results)
29         top_k = heapq.nlargest(
30             k, results, key=lambda x: x[1])
31         return [(phrase, freq)
32                 for phrase, freq in top_k]
33
34     def _dfs(self, node, results):
35         if '_phrase' in node:
36             results.append(
37                 (node['_phrase'], node['_freq']))
38         for char, child in node.items():
39             if not char.startswith('_'):
40                 self._dfs(child, results)
```

Performance: Trie traversal: O(prefix length). Top-K from cached prefixes: O(1). Uncached queries with DFS: 5-10ms for subtrees with ¡10K nodes. Overall P99 ¡ 20ms.
Storage: Trie in shared memory (mmap) across processes on the same host.

Updated hourly from batch n-gram computation. Old trie stays in memory during update; swap is atomic.

Q7: Design a Word Embedding Training Infrastructure

System Design Focus: Scalability Cost Management
Design a system to train Word2Vec/GloVe embeddings on a 1TB text corpus, producing high-quality 300-dimensional word vectors.

Architecture:
Word2Vec (Skip-gram with negative sampling) training is embarrassingly parallel across the corpus but requires shared access to the embedding matrix. GloVe training requires a global co-occurrence matrix that must be precomputed.
For Word2Vec:

- **Data Parallelism:** Shard the corpus across N workers. Each worker processes its shard independently, updating a shared embedding matrix via Hogwild-style lock-free SGD.
- **Parameter Server:** The embedding matrix (vocab_size $\times$ 300 $\times$ 4 bytes) for a 1M-word vocabulary = 1.2GB. Fits on a single parameter server with replicas.
- **Scaling:** 50 worker nodes, each processing 20GB of text. Training converges in 3-5 epochs over the full corpus. Total time: 8-12 hours.

```python
from gensim.models import Word2Vec
from gensim.models.word2vec import LineSentence

class DistributedW2VTrainer:
    def __init__(self, corpus_path, vector_size=300):
        self.corpus_path = corpus_path
        self.vector_size = vector_size

    def train(self):
        sentences = LineSentence(self.corpus_path)
        model = Word2Vec(
            sentences=sentences,
            vector_size=self.vector_size,
            window=5,
            min_count=5,
```

```
            workers=16,      # CPU threads
            sg=1,            # Skip-gram
            negative=10,     # Neg sampling
            epochs=5,
            sample=1e-4      # Subsampling
        )
        model.save("word2vec_300d.model")
        return model

    def evaluate(self, model):
        # Intrinsic evaluation: analogy task
        accuracy = model.wv.evaluate_word_analogies(
            "questions-words.txt")
        print(f"Analogy accuracy: {accuracy[0]:.3f}")
```

Cost Management: Use spot instances for workers (training is checkpoint-resumable). Parameter server on on-demand instances. Total cost: $200-400 for a full training run on cloud GPUs.

Q8: Design an Embedding Compression System for Mobile Deployment

System Design Focus: Latency & Performance Cost Management
Design a system to compress 768-dimensional BERT embeddings to fit on mobile devices with limited memory while maintaining search quality.

Architecture:
A multi-technique compression pipeline: dimensionality reduction → quantization → pruning.
Compression Stages:

- **Stage 1 – PCA/SVD:** Reduce from 768 to 256 dimensions. Typical quality retention: 95% of cosine similarity preservation. Reduction matrix is 768×256 = 768KB.
- **Stage 2 – Scalar Quantization:** Convert float32 to int8. Memory reduction: 4x. Quality retention with calibration: 98%.
- **Stage 3 – Product Quantization:** Further compress to 64 bytes per vector using PQ with 32 sub-quantizers. Quality retention: 92%.

On-Device: A vocabulary of 100K embeddings at 64 bytes each = 6.4MB, fits

comfortably on any modern phone.

```python
import numpy as np
from sklearn.decomposition import PCA

class EmbeddingCompressor:
    def __init__(self, target_dim=256):
        self.pca = PCA(n_components=target_dim)
        self.fitted = False

    def fit(self, embeddings: np.ndarray):
        self.pca.fit(embeddings)
        self.fitted = True
        variance = sum(
            self.pca.explained_variance_ratio_)
        print(f"Variance retained: {variance:.3f}")

    def compress(self, embeddings: np.ndarray):
        # Step 1: Dimensionality reduction
        reduced = self.pca.transform(embeddings)
        # Step 2: Scalar quantization to int8
        vmin, vmax = reduced.min(), reduced.max()
        scale = (vmax - vmin) / 255.0
        quantized = ((reduced - vmin) / scale) \
                    .astype(np.uint8)
        return quantized, {"scale": scale,
                           "min": vmin}

    def decompress(self, quantized, params):
        restored = quantized.astype(np.float32) \
                   * params["scale"] + params["min"]
        return restored
```

Quality Validation: Measure retrieval recall@10 before and after compression on a benchmark set. Accept if recall degradation is ¡5%.

Q9: Design a Dynamic Embedding Update System for Evolving Vocabulary

System Design Focus: Data Consistency Communication Between Services
Design a system that handles new vocabulary (trending terms, slang, brand names)

appearing in production without full model retraining.

Architecture:
A two-tier embedding system: a *static tier* (base embeddings from the pretrained model) and a *dynamic tier* (incrementally trained embeddings for new vocabulary).
New Word Detection:

- Monitor the unknown-token rate in production. A spike indicates new vocabulary.
- Extract candidate terms from high-frequency OOV tokens. Filter using frequency threshold (¿100 occurrences per day) and consistency (appears across multiple contexts).

Incremental Embedding:

- Collect context sentences for each new term.
- Initialize the new embedding as the average of its context word embeddings.
- Fine-tune with a small number of gradient steps on the context sentences, freezing all other embeddings.
- Validate: the new embedding should be close to semantically similar existing words.

```
import numpy as np
from collections import Counter

class DynamicVocabManager:
    def __init__(self, base_embeddings):
        self.base = base_embeddings  # word -> np.array
        self.dynamic = {}  # New word embeddings
        self.oov_counter = Counter()

    def track_oov(self, tokens):
        for token in tokens:
            if token not in self.base \
               and token not in self.dynamic:
                self.oov_counter[token] += 1

    def create_embedding(self, new_word,
                         context_sentences):
        """Initialize from average of context."""
        context_vecs = []
        for sentence in context_sentences:
            for word in sentence.split():
```

```
                if word in self.base:
                    context_vecs.append(
                        self.base[word])
        if context_vecs:
            embedding = np.mean(context_vecs, axis=0)
            embedding /= np.linalg.norm(embedding)
            self.dynamic[new_word] = embedding
            return embedding
        return None

    def get_embedding(self, word):
        if word in self.base:
            return self.base[word]
        return self.dynamic.get(word, None)
```

Consistency: Dynamic embeddings are versioned daily. All services consume the same version via a config flag. Rollout is gradual: new embeddings go to shadow traffic first.

Q10: Design a Hybrid Search System Combining Sparse and Dense Retrieval

System Design Focus: **Latency & Performance** **Scalability**
Design a search system that combines TF-IDF (sparse) and embedding-based (dense) retrieval to get the best of both worlds for a document corpus of 50 million items.

Architecture:
A two-stage retrieval system: **Stage 1** retrieves candidates using both sparse and dense methods in parallel; **Stage 2** fuses and re-ranks the combined candidate set.
Stage 1 – Parallel Retrieval:

- **Sparse Path:** BM25 over an Elasticsearch cluster (6 shards). Returns top-100 candidates. Latency: 15ms. Excels at exact keyword matches.
- **Dense Path:** HNSW index over 768-dim embeddings (FAISS). Returns top-100 candidates. Latency: 10ms. Excels at semantic similarity.

Stage 2 – Fusion: Reciprocal Rank Fusion (RRF) combines the two ranked lists. For each document, score $= \sum_r \frac{1}{k+\text{rank}_r}$ where $k = 60$. This is simple, parameter-free, and robust.

Stage 3 – Cross-Encoder Re-ranking: Top-20 fused results re-ranked by a cross-encoder (BERT-based) that scores (query, document) pairs jointly. Latency: 30ms for 20 pairs with batched inference.

```python
from dataclasses import dataclass

@dataclass
class SearchResult:
    doc_id: str
    score: float

class HybridSearch:
    def __init__(self, sparse_index, dense_index,
                 reranker):
        self.sparse = sparse_index
        self.dense = dense_index
        self.reranker = reranker

    def search(self, query: str, top_k: int = 10):
        # Stage 1: parallel retrieval
        sparse_results = self.sparse.search(
            query, top_n=100)
        dense_results = self.dense.search(
            self.encode(query), top_n=100)

        # Stage 2: Reciprocal Rank Fusion
        fused = self.rrf_fusion(
            sparse_results, dense_results, k=60)

        # Stage 3: Cross-encoder reranking
        top_candidates = fused[:20]
        reranked = self.reranker.rerank(
            query, top_candidates)
        return reranked[:top_k]

    def rrf_fusion(self, list_a, list_b, k=60):
        scores = {}
        for rank, r in enumerate(list_a):
            scores[r.doc_id] = 1.0 / (k + rank + 1)
        for rank, r in enumerate(list_b):
            scores[r.doc_id] = scores.get(
                r.doc_id, 0) + 1.0 / (k + rank + 1)
        ranked = sorted(scores.items(),
```

```
                            key=lambda x: -x[1])
        return [SearchResult(did, s)
                for did, s in ranked]
```

End-to-end latency: Stage 1 parallel: 15ms. Stage 2 fusion: ¡1ms. Stage 3 reranking: 30ms. Total: 50ms P95. This meets interactive search latency requirements.

4 Core NLP Tasks in Production

linkedin.com/in/lamhotsiagian

Named Entity Recognition, Part-of-Speech tagging, dependency parsing, coreference resolution—these are the workhorses of NLP. At scale, the challenge shifts from model accuracy to system reliability: handling billions of documents, managing model updates, and maintaining sub-second latency while processing complex linguistic structures.

4.1 Chapter Overview

Core NLP tasks form the backbone of most language understanding systems. At companies like Google and Meta, these tasks are rarely performed in isolation—they are stages in larger pipelines that feed search, content understanding, knowledge graph construction, and recommendation systems.

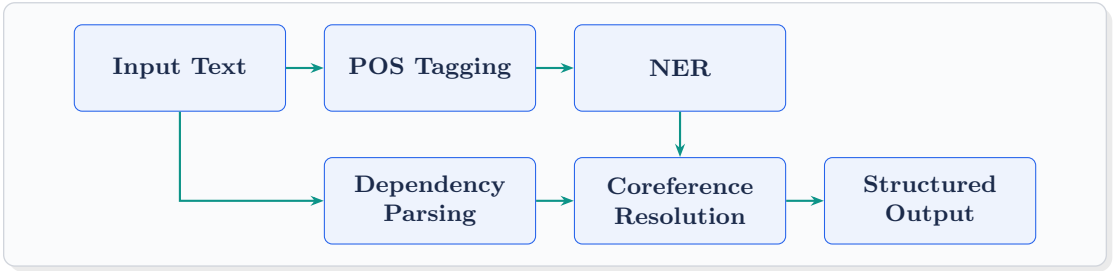
Part-of-Speech (POS) Tagging assigns grammatical labels (noun, verb, adjective) to each token. It is a fundamental step for downstream tasks like parsing and information extraction. Modern POS taggers achieve ¿97% accuracy on English but degrade on noisy, informal text.

Named Entity Recognition (NER) identifies and classifies entities (people, organizations, locations, dates, monetary values) in text. Production NER systems must handle domain-specific entities, nested entities, and entities that span multiple tokens.

Dependency Parsing analyzes the grammatical structure of sentences, identifying relationships between words (subject, object, modifier). It enables relation extraction, question answering, and information extraction.

Coreference Resolution determines when different expressions refer to the same

entity ("John" → "he" → "the CEO"). Critical for document-level understanding, summarization, and knowledge graph construction.



4.2 System Design Interview Questions

Q1: Design a Real-Time NER Service for Content Understanding

System Design Focus: Latency & Performance High Availability
Design a Named Entity Recognition service that processes user-generated content in real time (50K requests/sec) to extract people, organizations, locations, and custom entities for a social media platform.

Architecture:
A stateless NER service with model-per-domain support and a cascading architecture for speed.
Two-Tier Approach:

- **Tier 1 – Fast NER (CPU):** A lightweight BiLSTM-CRF model or distilled BERT handles 90% of requests. Latency: 5ms per text. Handles standard entity types (PER, ORG, LOC, DATE).
- **Tier 2 – Precision NER (GPU):** Full BERT-NER model activated for texts where Tier 1 confidence is low (¡0.7) or for premium features. Latency: 25ms.
- **Gazetteer Lookup:** A dictionary of known entities (company names, celebrity names, city names) runs in parallel. Results merged with model predictions, boosting recall for head entities.

```
from transformers import pipeline
import ahocorasick

class ProductionNER:
    def __init__(self):
```

```
        # Tier 1: fast model
        self.fast_model = pipeline(
            "ner", model="distilbert-NER",
            device=-1)  # CPU
        # Gazetteer for known entities
        self.gazetteer = ahocorasick.Automaton()
        self.load_gazetteer("known_entities.txt")

    def extract_entities(self, text):
        # Parallel: model + gazetteer
        model_entities = self.fast_model(text)
        gazetteer_hits = self.gazetteer_lookup(text)
        # Merge with priority: model > gazetteer
        merged = self.merge_entities(
            model_entities, gazetteer_hits)
        return merged

    def merge_entities(self, model_ents, gaz_ents):
        # Resolve overlapping spans
        all_ents = model_ents + gaz_ents
        all_ents.sort(key=lambda e: e['start'])
        merged = []
        for ent in all_ents:
            if not merged or \
               ent['start'] >= merged[-1]['end']:
                merged.append(ent)
        return merged
```

High Availability: Deployed in 3 AZs. If GPU fleet is down, all traffic routes to CPU Tier 1 (graceful degradation). Health checks include a canary NER inference on a known text every 10 seconds.

Q2: Design a Knowledge Graph Construction Pipeline Using NLP

System Design Focus: **Data Storage & Retrieval** **Scalability**
Design a system that automatically extracts entities and relationships from a corpus of 10 million documents and builds a knowledge graph updated daily.

Architecture:
A multi-stage extraction pipeline: **NER** → **Entity Linking** → **Relation Extraction** → **Graph Storage**.
Pipeline Stages:

- **NER:** Extract entity mentions. Batch process 10M docs using distributed inference (Spark + GPU nodes).
- **Entity Linking:** Resolve mentions to canonical entities. "Apple" → Apple Inc. (company) vs. apple (fruit). Use context + entity embeddings.
- **Relation Extraction:** For each entity pair in a sentence, classify the relationship (works_at, located_in, founded_by). BERT-based classifier on (entity1, context, entity2) triples.
- **Graph Storage:** Store in Neo4j or Amazon Neptune. Entities are nodes, relationships are edges. Each edge has provenance (source document, extraction confidence, timestamp).

```
class KnowledgeGraphBuilder:
    def __init__(self, ner_model, linker,
                 relation_extractor, graph_db):
        self.ner = ner_model
        self.linker = linker
        self.rel_ext = relation_extractor
        self.graph = graph_db

    def process_document(self, doc_id, text):
        # Step 1: Extract entities
        entities = self.ner.extract(text)
        # Step 2: Link to canonical IDs
        linked = [self.linker.link(e, text)
                  for e in entities]
        # Step 3: Extract relations
        for i, e1 in enumerate(linked):
            for e2 in linked[i+1:]:
                relation = self.rel_ext.predict(
                    text, e1, e2)
                if relation.confidence > 0.7:
                    self.graph.add_triple(
                        e1.canonical_id,
                        relation.type,
                        e2.canonical_id,
                        metadata={
                            "source": doc_id,
                            "confidence":
```

```
                                    relation.confidence
                                })
```

Scale: 10M docs × 50 entities/doc = 500M entity mentions. After deduplication and linking: 10M unique entities, 100M relationships. Neo4j cluster with 3 replicas handles this scale.
Daily Update: Incremental processing. Track document change log; only re-process new or modified documents. Merge new triples, handling contradictions via recency and confidence scoring.

Q3: Design a Multi-Language NER System

System Design Focus: Scalability Cost Management
Design an NER system supporting 50+ languages, handling the trade-off between per-language models (better quality) and multilingual models (lower cost).

Architecture:
A tiered model strategy based on language traffic volume:
Tier Strategy:

- **Tier 1 (Top 5 languages):** Dedicated fine-tuned models per language. English, Chinese, Spanish, Arabic, Hindi. Best quality, highest cost.
- **Tier 2 (Next 15 languages):** Language-family models (Romance, Germanic, Slavic, CJK). Shared model per family, fine-tuned on combined data.
- **Tier 3 (Remaining 30+ languages):** Single multilingual model (XLM-R or mBERT) covering all languages. Lower quality but cost-effective.

Routing: Language detection (Chapter 2's service) routes each request to the appropriate tier. Fallback chain: Tier 1 → Tier 2 → Tier 3.

```
class MultiLangNER:
    def __init__(self):
        self.tier1_models = {
            "en": load_model("ner-en-bert-large"),
            "zh": load_model("ner-zh-bert"),
            "es": load_model("ner-es-bert"),
        }
        self.tier2_models = {
            "romance": load_model("ner-romance-xlmr"),
            "germanic": load_model("ner-germanic-xlmr"),
```

```
        }
        self.tier3_model = load_model("ner-xlmr-base")
        self.lang_families = {
            "fr": "romance", "it": "romance",
            "de": "germanic", "nl": "germanic",
        }

    def predict(self, text, language):
        # Tier 1: dedicated model
        if language in self.tier1_models:
            return self.tier1_models[language](text)
        # Tier 2: language family model
        family = self.lang_families.get(language)
        if family and family in self.tier2_models:
            return self.tier2_models[family](text)
        # Tier 3: multilingual fallback
        return self.tier3_model(text)
```

Cost: Tier 1 (5 GPU instances) handles 60% of traffic. Tier 2 (3 GPU instances) handles 25%. Tier 3 (2 GPU instances) handles 15%. Total: 10 GPUs vs. 50 GPUs for per-language models—5x cost reduction.

Q4: Design a Dependency Parsing Service for Information Extraction

System Design Focus: Latency & Performance Communication Between Services

Design a dependency parsing system that feeds a downstream relation extraction pipeline, processing 10K documents per minute with parse trees available for querying.

Architecture:
A parsing service that produces dependency trees and stores them in a queryable format for downstream consumers.
Components:

- **Parser Service:** spaCy or Stanza-based dependency parser. Batch processing for throughput. Each instance parses 200 docs/sec on CPU.
- **Parse Store:** Dependency trees serialized as adjacency lists and stored in a

document store (Elasticsearch). Enables queries like "find all sentences where entity X is the subject of a verb."

- **Async Pipeline:** Documents published to a Kafka topic. Parser consumers process and write parsed results to the store. Decouples ingestion rate from parsing speed.

```python
import spacy

class DependencyParsingService:
    def __init__(self):
        self.nlp = spacy.load("en_core_web_trf")

    def parse(self, text: str) -> list[dict]:
        doc = self.nlp(text)
        parse_result = []
        for sent in doc.sents:
            tree = []
            for token in sent:
                tree.append({
                    "text": token.text,
                    "pos": token.pos_,
                    "dep": token.dep_,
                    "head": token.head.text,
                    "head_idx": token.head.i,
                    "children": [
                        c.text for c in token.children]
                })
            parse_result.append({
                "sentence": sent.text,
                "tokens": tree
            })
        return parse_result

    def extract_subject_verb_object(self, doc):
        """Extract SVO triples from parse tree."""
        triples = []
        for token in doc:
            if token.dep_ == "nsubj":
                subject = token.text
                verb = token.head.text
                objects = [c.text for c in
                           token.head.children
                           if c.dep_ == "dobj"]
```

```
                if objects:
                    triples.append(
                        (subject, verb, objects[0]))
        return triples
```

Communication: The parsing service exposes both sync (gRPC, for real-time) and async (Kafka consumer, for batch) interfaces. Downstream services subscribe to the "parsed_documents" Kafka topic for event-driven processing.

Q5: Design a Coreference Resolution System for Document Understanding

System Design Focus: Scalability Database Bottlenecks
Design a coreference resolution system that links entity mentions across long documents (10K+ words) for a document intelligence platform.

Architecture:
Coreference resolution on long documents is memory-intensive (quadratic in document length for span-pair scoring). The design must handle this computational challenge.
Key Design:

- **Chunked Processing:** Split long documents into overlapping windows (512 tokens with 128-token overlap). Run coref within each window, then merge cross-window coreference chains using entity matching on the overlap regions.
- **Model:** A higher-order coreference model (e2e-coref) identifies mention spans, scores span pairs, and clusters them into coreference chains.
- **Entity Index:** After resolution, store entity → mention mappings in a database. Each entity gets a canonical ID. This enables cross-document entity queries.

```
class CorefResolver:
    def __init__(self, model, window=512, overlap=128):
        self.model = model
        self.window = window
        self.overlap = overlap

    def resolve(self, tokens: list[str]) -> list:
        if len(tokens) <= self.window:
            return self.model.predict(tokens)
```

```
        # Chunked processing for long documents
        all_clusters = []
        for start in range(0, len(tokens),
                           self.window - self.overlap):
            end = min(start + self.window, len(tokens))
            chunk = tokens[start:end]
            clusters = self.model.predict(chunk)
            # Adjust offsets to global positions
            adjusted = self.adjust_offsets(
                clusters, start)
            all_clusters.extend(adjusted)

        # Merge overlapping clusters
        merged = self.merge_clusters(all_clusters)
        return merged

    def merge_clusters(self, clusters):
        # Union-Find for efficient cluster merging
        parent = {}
        def find(x):
            if x not in parent:
                parent[x] = x
            while parent[x] != x:
                x = parent[x]
            return x
        def union(a, b):
            parent[find(a)] = find(b)
        # Merge clusters sharing mentions in overlap
        for c in clusters:
            for mention in c[1:]:
                union(c[0], mention)
        return parent
```

Database: Entity-mention mappings stored in PostgreSQL with a composite index on (document_id, entity_id). For cross-document queries, a secondary index on entity canonical name enables fast lookups.

Q6: Design a Sentence Segmentation Service for Multilingual Text

System Design Focus: **Data Consistency** **High Availability**
Design a sentence segmentation system that accurately splits text into sentences across 30+ languages, handling edge cases like abbreviations, URLs, and decimal numbers.

Architecture:
A rule-augmented ML approach. The base model handles most cases; language-specific rules handle known exceptions.
Design:

- **Base Model:** A lightweight binary classifier (is this period a sentence boundary?) using features: surrounding characters, capitalization, abbreviation dictionary lookup.
- **Rule Layer:** Language-specific rules for edge cases. German compound nouns, Japanese without spaces (use CRF-based segmentation), Thai (no whitespace or punctuation-based boundaries).
- **Consistency:** The segmentation library is versioned. Both indexing and query pipelines use the same version. Sentence boundaries affect downstream NLP (NER, parsing), so inconsistency causes subtle bugs.

```python
import re
from typing import List

class SentenceSegmenter:
    ABBREVIATIONS = {"mr.", "mrs.", "dr.", "inc.",
                     "ltd.", "etc.", "vs.", "e.g.",
                     "i.e.", "u.s.", "u.k."}

    def segment(self, text: str,
                lang: str = "en") -> List[str]:
        if lang in ("ja", "zh"):
            return self._segment_cjk(text)
        if lang == "th":
            return self._segment_thai(text)
        return self._segment_default(text, lang)

    def _segment_default(self, text, lang):
        # Pre-protect abbreviations
        protected = text
```

```
        for abbr in self.ABBREVIATIONS:
            protected = protected.replace(
                abbr, abbr.replace(".", "\x00"))
        # Split on sentence boundaries
        sentences = re.split(
            r'(?<=[.!?])\s+(?=[A-Z])', protected)
        # Restore abbreviations
        return [s.replace("\x00", ".")
                for s in sentences if s.strip()]

    def _segment_cjk(self, text):
        # CJK: split on sentence-ending punctuation
        return re.split(r'(?<=[.\!?])', text)  #
            simplified
```

High Availability: Pure CPU service, no external dependencies. Deployed as a sidecar or library for zero-network-hop latency. Fallback: regex-only segmentation if the ML model fails to load.

Q7: Design an Entity Linking System at Web Scale

System Design Focus: Data Storage & Retrieval Latency & Performance
Design a system that resolves entity mentions in text to entries in a knowledge base (like Wikipedia) with 10M+ entities, handling ambiguity at 100K queries per second.

Architecture:
A candidate generation + ranking pipeline. First, retrieve candidate entities quickly; then, rank them using context.
Components:

- **Alias Table:** A mapping from surface forms to candidate entities. "Apple" → {Apple Inc., Apple (fruit), Apple Records, ...}. Stored in Redis, 50M alias-entity pairs.
- **Candidate Generator:** For each mention, retrieve top-20 candidates from the alias table. Include popularity prior (P(entity—mention) from Wikipedia anchor text statistics).
- **Context Ranker:** A cross-encoder scores (mention_context, entity_description) pairs. The entity whose description best matches the surrounding context wins.

```python
class EntityLinker:
    def __init__(self, alias_store, entity_store,
                 ranker):
        self.aliases = alias_store     # Redis
        self.entities = entity_store   # ElasticSearch
        self.ranker = ranker           # Cross-encoder

    def link(self, mention: str,
             context: str) -> dict:
        # Step 1: Candidate generation
        candidates = self.aliases.get_candidates(
            mention.lower())
        if not candidates:
            return {"entity": None, "score": 0}

        # Step 2: Context-based ranking
        scored = []
        for cand in candidates[:20]:
            desc = self.entities.get_description(
                cand["entity_id"])
            score = self.ranker.score(
                context, desc)
            # Combine with popularity prior
            final_score = 0.7 * score + \
                          0.3 * cand["prior"]
            scored.append({
                "entity_id": cand["entity_id"],
                "name": cand["name"],
                "score": final_score})

        scored.sort(key=lambda x: -x["score"])
        return scored[0] if scored else None
```

Performance: Alias lookup: 1ms (Redis). Ranking 20 candidates: 15ms (GPU cross-encoder with batching). Total: 20ms per mention. For batch processing, embed candidate descriptions offline and use dot-product similarity (no GPU needed at query time).

Q8: Design an NLP Pipeline Orchestrator with Error Recovery

System Design Focus: **High Availability** **Observability**
Design a pipeline orchestrator that chains NLP tasks (tokenization → NER → parsing → coref) with automatic error recovery, retries, and partial result handling.

Architecture:
A DAG executor that runs NLP pipeline stages with dependency awareness, circuit breakers, and graceful degradation.
Error Handling Strategy:

- **Retry with backoff:** Transient failures (network, GPU OOM) get 3 retries with exponential backoff (1s, 4s, 16s).
- **Circuit breaker:** If a stage fails ¿50% of requests in a 1-minute window, the circuit opens. Requests skip that stage and proceed with partial results.
- **Graceful degradation:** NER can work without POS. Coref quality degrades without parsing but still functions. Define a dependency graph with "required" and "optional" edges.
- **Dead letter queue:** Documents that fail all retries go to a DLQ for manual inspection.

```
import time
from enum import Enum

class CircuitState(Enum):
    CLOSED = "closed"    # Normal operation
    OPEN = "open"        # Failing, skip stage
    HALF_OPEN = "half"   # Testing recovery

class PipelineOrchestrator:
    def __init__(self, stages):
        self.stages = stages
        self.circuits = {s.name: CircuitState.CLOSED
                         for s in stages}
        self.failure_counts = {s.name: 0
                               for s in stages}

    def process(self, document):
        results = {}
        for stage in self.stages:
            if self.circuits[stage.name] == \
```

```
                CircuitState.OPEN:
                    results[stage.name] = None  # Skip
                    continue
            try:
                result = self.execute_with_retry(
                    stage, document, results)
                results[stage.name] = result
                self.failure_counts[stage.name] = 0
            except Exception as e:
                self.failure_counts[stage.name] += 1
                if self.failure_counts[stage.name] > 50:
                    self.circuits[stage.name] = \
                        CircuitState.OPEN
                results[stage.name] = None
        return results

    def execute_with_retry(self, stage, doc,
                           context, retries=3):
        for attempt in range(retries):
            try:
                return stage.run(doc, context)
            except Exception:
                if attempt < retries - 1:
                    time.sleep(2 ** attempt)
                else:
                    raise
```

Observability: Each pipeline run produces a trace with per-stage latency, status (success/skip/fail), and output size. A dashboard shows circuit breaker states, retry rates, and DLQ depth across all stages.

Q9: Design an Active Learning System for NER Model Improvement

System Design Focus: Cost Management Observability

Design a system that selects the most valuable unlabeled examples for human annotation, maximizing NER model improvement per annotation dollar spent.

Architecture:
An active learning loop: **Model Inference** → **Uncertainty Sampling** → **Human Annotation** → **Model Retraining**.
Selection Strategies:

- **Uncertainty Sampling:** Select sentences where the NER model is most uncertain (highest token-level entropy or lowest sequence probability).
- **Diversity Sampling:** Cluster unlabeled sentences by embedding similarity; select representatives from each cluster to ensure coverage.
- **Error-Targeted Sampling:** Use a secondary model to predict where the primary model is likely wrong. Focus annotation on those cases.
- **Hybrid:** Score $= 0.5 \times$ uncertainty $+ 0.3 \times$ diversity $+ 0.2 \times$ error_prediction.

```python
import numpy as np

class ActiveLearner:
    def __init__(self, model, unlabeled_pool):
        self.model = model
        self.pool = unlabeled_pool

    def select_batch(self, batch_size=100):
        scores = []
        for text in self.pool:
            predictions = self.model.predict(text)
            # Token-level uncertainty
            entropy = self._compute_entropy(
                predictions)
            scores.append({
                "text": text,
                "uncertainty": entropy,
            })
        # Sort by uncertainty, take top batch
        scores.sort(key=lambda x:
                    -x["uncertainty"])
        return scores[:batch_size]

    def _compute_entropy(self, predictions):
        """Average token entropy across sequence."""
        entropies = []
        for token_probs in predictions:
            probs = np.array(token_probs)
            probs = np.clip(probs, 1e-10, 1.0)
            ent = -np.sum(probs * np.log(probs))
```

```
            entropies.append(ent)
        return np.mean(entropies)
```

Cost Impact: Active learning typically achieves the same model quality with 30-50% fewer labeled examples compared to random sampling. For NER annotation at $0.10/sentence, this saves $15K-25K on a 500K-sentence annotation project.

Q10: Design a Constituency Parsing System with Parse Caching

System Design Focus: Latency & Performance Database Bottlenecks
Design a constituency parsing service that reuses parse results for frequently seen sentences (search queries, common phrases) through intelligent caching.

Architecture:
A parse cache backed by Redis with a content-addressable key scheme. Since many inputs repeat (search queries follow a power-law distribution), caching parse trees dramatically reduces GPU compute.
Key Design:

- **Cache Key:** SHA-256 hash of (normalized_text + model_version). Normalization ensures "How are you?" and "how are you?" share the same parse.
- **Cache Value:** Serialized parse tree in a compact binary format (500 bytes for a typical sentence).
- **Cache Policy:** LRU eviction with a 24-hour TTL. Cache size: 10M entries × 500 bytes = 5GB in Redis.
- **Hit Rate:** For search queries, expect 60-70% cache hit rate. For general text, 10-20%.

```
import hashlib, json, redis

class CachedParser:
    def __init__(self, parser_model,
                 redis_client, model_version):
        self.parser = parser_model
        self.cache = redis_client
        self.version = model_version
        self.ttl = 86400  # 24 hours

    def parse(self, text: str) -> dict:
```

```
        normalized = text.lower().strip()
        cache_key = self._make_key(normalized)
        # Check cache
        cached = self.cache.get(cache_key)
        if cached:
            return json.loads(cached)
        # Cache miss: compute parse
        tree = self.parser.parse(normalized)
        result = self._serialize_tree(tree)
        # Store in cache
        self.cache.setex(
            cache_key, self.ttl,
            json.dumps(result))
        return result

    def _make_key(self, text):
        raw = f"{self.version}:{text}"
        return
           f"parse:{hashlib.sha256(raw.encode()).hexdigest()[:16]}"
```

Database Optimization: Redis used as a pure cache (no persistence needed). If Redis is unavailable, the service degrades gracefully to parsing every request—higher latency but no errors. Monitoring tracks cache hit rate; a drop signals a traffic pattern shift.

5 Classical Machine Learning for NLP

linkedin.com/in/lamhotsiagian

Before deep learning, classical ML algorithms powered the NLP industry—and many still do. Naive Bayes, logistic regression, SVMs, HMMs, and CRFs remain the backbone of high-throughput, low-latency NLP systems where interpretability, speed, and cost efficiency matter more than squeezing out the last percentage point of accuracy.

5.1 Chapter Overview

Classical ML models for NLP are not relics—they are production workhorses. At major tech companies, a surprising number of high-traffic NLP features run on classical models because the cost-quality trade-off makes sense.

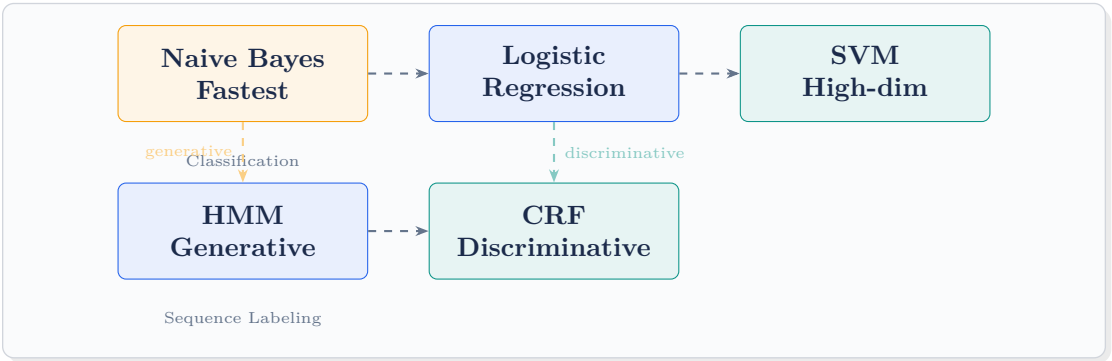
Naive Bayes is the fastest text classifier. With a bag-of-words representation, it trains in seconds on millions of documents and classifies in microseconds. It is the gold standard for spam filtering, language detection, and any classification task where speed trumps sophistication.

Logistic Regression with TF-IDF features is the default baseline for text classification. It is fast, interpretable (feature weights map to words), and regularizable. At many companies, it is the first model deployed and often the last one replaced.

Support Vector Machines (SVMs) excel in high-dimensional spaces (which text inherently is). With kernel tricks, they can capture non-linear boundaries. Linear SVMs are particularly effective for text where the number of features exceeds the number of samples.

Hidden Markov Models (HMMs) and **Conditional Random Fields (CRFs)**

are sequence labeling models used for POS tagging and NER. CRFs model the entire sequence jointly, capturing label dependencies (a "B-PER" tag is likely followed by "I-PER", not "B-ORG").



5.2 System Design Interview Questions

Q1: Design a Spam Classification System Using Naive Bayes at Scale

System Design Focus: **Scalability** **Latency & Performance**
Design a spam filter that classifies 1 million emails per hour using Naive Bayes, with model updates every hour to adapt to new spam patterns.

Architecture:
Naive Bayes is ideal here: microsecond inference, minimal memory, and incremental trainability.
Key Components:

- **Feature Extraction:** TF-IDF with 50K feature vocabulary. Pre-compute feature vectors for batch classification; on-the-fly for real-time.
- **Model:** Multinomial Naive Bayes. Model size: 50K features $\times$ 2 classes $\times$ 8 bytes = 800KB. Fits entirely in L1 cache.
- **Incremental Updates:** Naive Bayes supports online learning. Each hour, update class priors and feature likelihoods with new labeled data (user spam reports). No full retrain needed.
- **Ensemble:** Combine Naive Bayes (fast, high recall) with a rules engine (blocklists, regex patterns for known spam signatures) for higher precision.

```python
from sklearn.naive_bayes import MultinomialNB
from sklearn.feature_extraction.text import
    TfidfVectorizer
import numpy as np

class SpamClassifier:
    def __init__(self):
        self.vectorizer = TfidfVectorizer(
            max_features=50000, stop_words='english')
        self.model = MultinomialNB(alpha=0.1)

    def train(self, texts, labels):
        X = self.vectorizer.fit_transform(texts)
        self.model.fit(X, labels)

    def predict(self, text: str) -> dict:
        X = self.vectorizer.transform([text])
        proba = self.model.predict_proba(X)[0]
        return {
            "is_spam": bool(proba[1] > 0.5),
            "spam_probability": float(proba[1]),
            "confidence": float(max(proba))
        }

    def incremental_update(self, new_texts,
                           new_labels):
        X = self.vectorizer.transform(new_texts)
        self.model.partial_fit(X, new_labels)
```

Scale: Single instance classifies 500K emails/hour. Two instances with load balancing handle 1M/hour with redundancy. Memory: ¡100MB per instance. Cost: negligible compared to deep learning alternatives.

Q2: Design a Text Classification System with Logistic Regression

System Design Focus: **Observability** **High Availability**

Design a content categorization system using logistic regression that classifies articles into 100+ categories with explainable predictions for editorial review.

Architecture:
Logistic regression provides interpretable feature weights, making it ideal for editorial workflows where humans need to understand why a classification was made.
Key Design:

- **One-vs-Rest:** Train 100+ binary classifiers, one per category. Each classifier produces a probability. Articles can belong to multiple categories.
- **Feature Engineering:** TF-IDF (unigrams + bigrams), title features, source domain, article length, presence of named entities. Total: 200K features.
- **Explainability:** For each prediction, return top-5 features (words) that most influenced the classification, with their weights.

```python
from sklearn.linear_model import LogisticRegression
import numpy as np

class ExplainableClassifier:
    def __init__(self, categories):
        self.categories = categories
        self.models = {}
        self.vectorizer = None

    def predict_with_explanation(self, text):
        X = self.vectorizer.transform([text])
        results = []
        for cat, model in self.models.items():
            proba = model.predict_proba(X)[0][1]
            if proba > 0.3:
                # Get top contributing features
                coefs = model.coef_[0]
                features = X.toarray()[0]
                contributions = coefs * features
                top_idx = np.argsort(
                    contributions)[-5:][::-1]
                names = self.vectorizer \
                    .get_feature_names_out()
                explanation = [
                    {"word": names[i],
                     "weight": round(
                        float(contributions[i]), 3)}
                    for i in top_idx
                    if contributions[i] > 0]
                results.append({
```

```
                        "category": cat,
                        "probability": round(proba, 3),
                        "explanation": explanation})
        return sorted(results,
                      key=lambda x: -x["probability"])
```

Observability: Log every classification with the explanation. Dashboard shows per-category precision/recall over time, feature drift (top features changing), and editor override rates (human corrections indicate model errors).

Q3: Design an SVM-Based Document Similarity System

System Design Focus: Database Bottlenecks Cost Management
Design a system using SVMs to find similar legal documents in a corpus of 5 million documents for a legal tech platform.

Architecture:
Use SVM to learn a similarity metric in a projected feature space, then build an index for fast retrieval.
Approach:

- **Feature Space:** TF-IDF with domain-specific legal vocabulary (100K terms). Add structural features: document type, jurisdiction, cited statutes.
- **SVM Ranking:** Train a ranking SVM that learns to order documents by relevance given a query document. Training data: lawyer-annotated similar document pairs.
- **Index:** Pre-compute SVM-projected representations. Build an ANN index (FAISS) on the projected vectors for fast retrieval.
- **Two-Stage:** Stage 1: ANN retrieval of top-100 candidates. Stage 2: Re-rank with full SVM scoring.

```
from sklearn.svm import LinearSVC
from sklearn.calibration import CalibratedClassifierCV

class DocumentSimilarityEngine:
    def __init__(self):
        base_svm = LinearSVC(C=1.0, max_iter=10000)
        self.model = CalibratedClassifierCV(base_svm)
        self.vectorizer = None
```

```
    def train(self, doc_pairs, labels):
        """Train on (doc_a, doc_b, similar?) triples."""
        features = []
        for doc_a, doc_b in doc_pairs:
            vec_a = self.vectorizer.transform([doc_a])
            vec_b = self.vectorizer.transform([doc_b])
            # Feature: element-wise product + diff
            combined = np.hstack([
                vec_a.multiply(vec_b).toarray(),
                np.abs((vec_a - vec_b).toarray())])
            features.append(combined[0])
        self.model.fit(np.array(features), labels)

    def find_similar(self, query_doc, candidates,
                     top_k=10):
        query_vec = self.vectorizer.transform(
            [query_doc])
        scores = []
        for cand_id, cand_doc in candidates:
            cand_vec = self.vectorizer.transform(
                [cand_doc])
            feat = np.hstack([
                query_vec.multiply(cand_vec).toarray(),
                np.abs((query_vec - cand_vec)
                       .toarray())])
            prob = self.model.predict_proba(feat)[0][1]
            scores.append((cand_id, prob))
        return sorted(scores, key=lambda x: -x[1])[:top_k]
```

Cost: SVM inference on CPU handles 1000 comparisons/sec. With ANN pre-filtering to 100 candidates, total latency per query is 100ms. No GPU needed—significant cost savings for a legal tech startup.

Q4: Design a CRF-Based NER System with Online Learning

System Design Focus: **Data Consistency** **Communication Between Services**
Design an NER system using Conditional Random Fields that can be updated with new training data without full retraining, maintaining consistency across all serving instances.

Architecture:
CRFs for NER provide excellent accuracy with hand-crafted features and are faster than transformer-based alternatives.
Key Design:

- **CRF Features:** Current word, POS tag, word shape (Xxxx = capitalized), prefix/suffix, neighboring word features, gazetteer membership. 500K feature functions.
- **Online Learning:** Use averaged perceptron training for CRF. New training batches update feature weights incrementally. Periodically (weekly), do a full retrain for optimal convergence.
- **Model Distribution:** Updated model serialized and published to a model registry (S3 + Redis). All serving instances poll for new model versions every 5 minutes.
- **Consistency:** Version tag on every prediction response. During rollout, some instances serve v1.2 and others v1.3. Client-side logic handles this gracefully.

```python
import sklearn_crfsuite

class CRFNamedEntityRecognizer:
    def __init__(self):
        self.model = sklearn_crfsuite.CRF(
            algorithm='lbfgs',
            c1=0.1, c2=0.1,
            max_iterations=100)

    def extract_features(self, sentence, i):
        word = sentence[i]
        features = {
            'word.lower()': word.lower(),
            'word[-3:]': word[-3:],
            'word[-2:]': word[-2:],
            'word.isupper()': word.isupper(),
            'word.istitle()': word.istitle(),
            'word.isdigit()': word.isdigit(),
        }
        if i > 0:
            features['prev_word'] = \
                sentence[i-1].lower()
        if i < len(sentence) - 1:
            features['next_word'] = \
                sentence[i+1].lower()
```

```
        return features

    def predict(self, sentence):
        features = [self.extract_features(sentence, i)
                    for i in range(len(sentence))]
        labels = self.model.predict([features])[0]
        return list(zip(sentence, labels))
```

Communication: Model updates broadcast via a Kafka topic. Each serving instance subscribes and hot-swaps the model in-memory. The swap is atomic: load new model, verify with a canary sentence, then swap pointers.

Q5: Design an HMM-Based POS Tagger with Caching

System Design Focus: **Latency & Performance** **Data Storage & Retrieval**
Design a POS tagging service using Hidden Markov Models that leverages caching patterns in natural language to minimize computation.

Key Design: HMMs use the Viterbi algorithm for decoding, which is $O(T \times S^2)$ where T is sequence length and S is the number of states (POS tags, 45 for Penn Treebank). This is already fast but can be accelerated with caching.

Caching Strategy:

- **Phrase Cache:** Common phrases ("in the", "of the", "New York") have deterministic POS patterns. Cache n-gram → POS sequence for the top 100K n-grams.
- **Word-Tag Cache:** Unambiguous words (90% of English vocabulary) always get the same tag. Cache word → tag for unambiguous words; only run Viterbi for ambiguous segments.

```
import numpy as np

class HMMPosTagger:
    def __init__(self, transition_probs,
                 emission_probs, states):
        self.trans = transition_probs  # S x S
        self.emit = emission_probs     # S x V
        self.states = states
        self.unambiguous_cache = {}
```

```
10
11      def viterbi(self, observations):
12          T = len(observations)
13          S = len(self.states)
14          dp = np.zeros((T, S))
15          backptr = np.zeros((T, S), dtype=int)
16
17          # Initialize
18          for s in range(S):
19              dp[0][s] = self.emit[s][observations[0]]
20
21          # Forward pass
22          for t in range(1, T):
23              for s in range(S):
24                  probs = dp[t-1] + self.trans[:, s] \
25                          + self.emit[s][observations[t]]
26                  backptr[t][s] = np.argmax(probs)
27                  dp[t][s] = probs[backptr[t][s]]
28
29          # Backtrack
30          path = [np.argmax(dp[-1])]
31          for t in range(T-1, 0, -1):
32              path.insert(0, backptr[t][path[0]])
33          return [self.states[s] for s in path]
34
35      def tag(self, words):
36          # Check cache for unambiguous words
37          cached = [self.unambiguous_cache.get(w)
38                    for w in words]
39          if all(c is not None for c in cached):
40              return cached  # Full cache hit
41          # Run Viterbi for full sequence
42          obs = [self.word_to_id(w) for w in words]
43          return self.viterbi(obs)
```

Performance: Without cache: 50K tokens/sec per CPU core. With unambiguous cache: 200K tokens/sec (4x speedup since 75% of tokens in typical English text are unambiguous).

Q6: Design a Sentiment Analysis System Using Ensemble Classical Models

System Design Focus: **High Availability** **Observability**
Design a sentiment analysis system that ensembles multiple classical models for robustness, with automatic model health monitoring.

Architecture:
Three models vote on sentiment: Naive Bayes (speed), Logistic Regression (balance), SVM (accuracy). A meta-learner combines their predictions.
Ensemble Strategy:

- **Stacking:** Each base model produces a probability distribution. A logistic regression meta-model takes these as features and outputs the final prediction.
- **Fallback Chain:** If the meta-model disagrees with all base models, flag for review. If one model fails (circuit breaker open), the ensemble runs with remaining models.
- **Diversity:** Each model uses different features—Naive Bayes uses word counts, LR uses TF-IDF bigrams, SVM uses character n-grams.

```
class SentimentEnsemble:
    def __init__(self):
        self.models = {
            "nb": NaiveBayesSentiment(),
            "lr": LogRegSentiment(),
            "svm": SVMSentiment()
        }
        self.meta_model = LogisticRegression()
        self.health = {name: True
                       for name in self.models}

    def predict(self, text):
        base_preds = {}
        for name, model in self.models.items():
            if not self.health[name]:
                continue
            try:
                base_preds[name] = \
                    model.predict_proba(text)
            except Exception:
                self.health[name] = False

```

```
        if not base_preds:
            return {"error": "all models down"}

        # Stack predictions
        features = np.concatenate(
            list(base_preds.values()))
        final = self.meta_model.predict_proba(
            features.reshape(1, -1))[0]
        return {
            "sentiment": "positive"
                if final[1] > 0.5 else "negative",
            "confidence": float(max(final)),
            "model_votes": {
                k: "pos" if v[1] > 0.5 else "neg"
                for k, v in base_preds.items()
            }
        }
```

Observability: Track per-model accuracy, disagreement rate between models, and meta-model confidence distribution. Alert when disagreement rate spikes (indicates data drift hitting models differently).

Q7: Design a Feature Engineering Pipeline for Classical NLP Models

System Design Focus: Scalability Data Storage & Retrieval
Design a system that computes and stores 500+ hand-crafted NLP features for 100M documents, feeding classical ML models.

Feature Categories: Lexical (word counts, vocabulary richness, avg word length), syntactic (POS distribution, parse depth), semantic (entity counts, sentiment words), stylistic (punctuation patterns, sentence length distribution), and domain-specific features.

Architecture:

- **Compute:** Distributed Spark job extracts features per document. Each executor handles 1M docs. Total: 100 executors $\times$ 1M docs = 100M docs in 2 hours.
- **Storage:** Feature vectors stored in Parquet on S3 (columnar, compressed).

Hot features materialized in Redis for online serving.

- **Registry:** A feature catalog tracks each feature's name, version, computation logic, and statistics (mean, std, missing rate).

```python
from collections import Counter
import numpy as np

class NLPFeatureExtractor:
    def extract(self, text: str) -> dict:
        tokens = text.split()
        sentences = text.split('.')
        features = {
            # Lexical features
            "word_count": len(tokens),
            "unique_words": len(set(tokens)),
            "avg_word_len": np.mean(
                [len(w) for w in tokens]),
            "vocab_richness": len(set(tokens))
                              / max(len(tokens), 1),
            # Structural features
            "sentence_count": len(sentences),
            "avg_sent_len": np.mean(
                [len(s.split()) for s in sentences
                 if s.strip()]),
            # Punctuation features
            "exclamation_count": text.count('!'),
            "question_count": text.count('?'),
            "caps_ratio": sum(
                1 for c in text if c.isupper())
                / max(len(text), 1),
            # Character n-gram features
            "char_trigram_entropy":
                self._char_ngram_entropy(text, 3),
        }
        return features

    def _char_ngram_entropy(self, text, n):
        ngrams = [text[i:i+n]
                  for i in range(len(text)-n+1)]
        counts = Counter(ngrams)
        total = sum(counts.values())
        probs = [c/total for c in counts.values()]
        return -sum(p * np.log(p) for p in probs)
```

Scale: 500 features × 4 bytes × 100M docs = 200GB total. Parquet compression brings this to 50GB on S3. Feature computation is embarrassingly parallel—add more Spark executors to scale linearly.

Q8: Design a Multi-Label Classification System for Content Tagging

System Design Focus: Database Bottlenecks Scalability
Design a system using classical ML that assigns multiple labels (topics, categories, content warnings) to user-generated content at 200K items per hour.

Key Design:

- **Binary Relevance:** Train independent binary classifiers (logistic regression) for each of 500 labels. Simple but ignores label correlations.
- **Classifier Chains:** Order labels by frequency. Each classifier takes the input features plus the predictions of all previous classifiers. Captures label dependencies.
- **Threshold Optimization:** Each label gets its own classification threshold, optimized on validation data using F1 score per label.

```python
from sklearn.linear_model import SGDClassifier
from sklearn.multiclass import OneVsRestClassifier

class MultiLabelTagger:
    def __init__(self, n_labels=500):
        self.classifier = OneVsRestClassifier(
            SGDClassifier(loss='log_loss',
                          penalty='l2',
                          max_iter=1000))
        self.thresholds = np.full(n_labels, 0.5)

    def train(self, X, Y):
        self.classifier.fit(X, Y)
        # Optimize thresholds on validation
        self._optimize_thresholds(X, Y)

    def predict(self, X):
        probas = self.classifier.predict_proba(X)
```

```
        # Apply per-label thresholds
        predictions = (probas >= self.thresholds) \
                      .astype(int)
        return predictions

    def _optimize_thresholds(self, X_val, Y_val):
        probas = self.classifier.predict_proba(X_val)
        for i in range(Y_val.shape[1]):
            best_t, best_f1 = 0.5, 0
            for t in np.arange(0.1, 0.9, 0.05):
                preds = (probas[:, i] >= t).astype(int)
                f1 = f1_score(Y_val[:, i], preds)
                if f1 > best_f1:
                    best_t, best_f1 = t, f1
            self.thresholds[i] = best_t
```

Database: Store label predictions in a column-family store (Cassandra) keyed by content ID. Each label is a column. Enables fast queries: "find all content tagged with both 'sports' and 'controversy' " using secondary indexes.
Scale: 500 binary classifiers $\times$ 10μs each = 5ms per item. At 200K items/hour (56 items/sec), a single instance handles the load with 3.5x headroom.

Q9: Design a Classical ML Model Serving Infrastructure

System Design Focus: **High Availability** **Latency & Performance**
Design a model serving system optimized for classical ML models (sklearn) that serves 50K predictions per second with model hot-swapping.

Architecture:
Classical ML models are small (MB range) and fast (microsecond inference), so the serving challenge is about throughput, availability, and zero-downtime updates.
Key Design:

- **In-Process Serving:** Load sklearn models directly into Python worker processes. No separate model server needed (unlike GPU models).
- **Multi-Process:** Use gunicorn with 8 workers per pod. Each worker holds a copy of the model in memory. Total memory: 8 $\times$ 50MB = 400MB per pod.
- **Hot-Swap:** Model files stored on a shared volume. A file watcher detects new model versions and triggers reload. Workers reload one at a time (rolling

update) to maintain availability.

- **Batching:** Accept batch requests of up to 256 items. Sklearn's vectorized predict is 10x faster per item in batch mode.

```python
import joblib
import os, time, threading

class ModelServer:
    def __init__(self, model_dir):
        self.model_dir = model_dir
        self.model = None
        self.model_version = None
        self.lock = threading.RLock()
        self._load_latest()
        self._start_watcher()

    def _load_latest(self):
        files = sorted(os.listdir(self.model_dir))
        latest = files[-1] if files else None
        if latest and latest != self.model_version:
            new_model = joblib.load(
                os.path.join(self.model_dir, latest))
            with self.lock:
                self.model = new_model
                self.model_version = latest

    def predict_batch(self, texts: list) -> list:
        with self.lock:
            model = self.model
        # Vectorized prediction
        return model.predict_proba(texts).tolist()

    def _start_watcher(self):
        def watch():
            while True:
                time.sleep(30)
                self._load_latest()
        t = threading.Thread(target=watch, daemon=True)
        t.start()
```

Availability: 6 pods across 3 AZs. Even with 2 pods down, remaining 4 pods (32 workers) handle 50K RPS easily. Pod startup time: ¡10 seconds (model loading is fast for sklearn).

Q10: Design a Feedback Loop System for Continuous Model Improvement

System Design Focus: Communication Between Services Observability
Design a system that captures user feedback (corrections, upvotes/downvotes) on classical ML predictions and uses it to continuously improve model quality.

Architecture:
A closed-loop system: **Predict** → **Serve** → **Collect Feedback** → **Label** → **Retrain** → **Deploy**.
Feedback Collection:

- **Explicit:** User clicks "wrong classification" and selects correct label.
- **Implicit:** User ignores a recommendation (negative signal) or acts on it (positive signal).
- **Signal Quality:** Explicit feedback is high-quality but sparse. Implicit signals are noisy but abundant. Weight explicit feedback 5x in training.

```python
class FeedbackLoop:
    def __init__(self, model_trainer, min_samples=1000):
        self.feedback_buffer = []
        self.min_samples = min_samples
        self.trainer = model_trainer

    def record_feedback(self, prediction_id, text,
                        predicted_label,
                        correct_label, feedback_type):
        weight = 5.0 if feedback_type == "explicit" \
                 else 1.0
        self.feedback_buffer.append({
            "text": text,
            "label": correct_label,
            "weight": weight,
            "timestamp": time.time()
        })
        if len(self.feedback_buffer) >= \
           self.min_samples:
            self.trigger_retrain()

    def trigger_retrain(self):
        # Combine feedback with existing training data
        new_data = self.feedback_buffer.copy()
```

```
        self.feedback_buffer = []
        # Publish retrain event
        kafka_producer.send('retrain_trigger', {
            "new_samples": len(new_data),
            "data_path": self.save_feedback(new_data)
        })

    def save_feedback(self, data):
        path = f"feedback/{int(time.time())}.jsonl"
        # Save to S3
        return path
```

Observability: Track feedback rate, correction distribution (which categories get corrected most), and model performance before/after each retrain cycle. A dashboard shows the feedback-to-improvement pipeline health.
Communication: Feedback events flow through Kafka. A retrain orchestrator service consumes feedback batches and triggers Airflow retraining DAGs. Model deployment is automatic if the new model passes quality gates.

6 Deep Learning for NLP at Scale

linkedin.com/in/lamhotsiagian

Deep learning transformed NLP. RNNs, LSTMs, GRUs, CNNs for text, and the attention mechanism form the neural backbone of modern language understanding. Deploying these models at production scale—handling GPU management, batching strategies, model optimization, and real-time inference—is where system design meets deep learning.

6.1 Chapter Overview

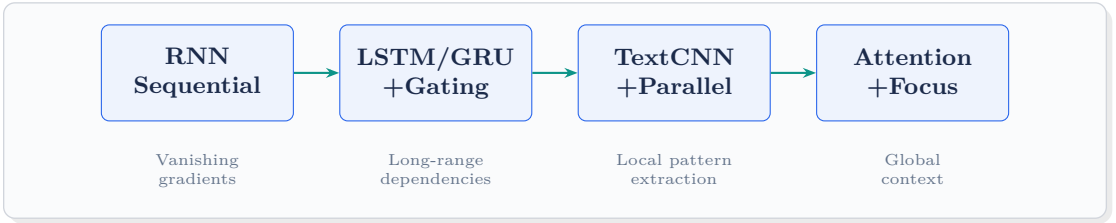
The evolution of deep learning for NLP follows a clear trajectory, with each architecture solving limitations of its predecessor:

RNNs (Recurrent Neural Networks) process text sequentially, maintaining a hidden state that carries information forward. They struggle with long sequences due to vanishing gradients.

LSTMs and GRUs add gating mechanisms to RNNs, enabling them to selectively remember and forget information over longer sequences. LSTMs have three gates (input, forget, output); GRUs simplify to two (reset, update) with comparable performance.

CNNs for Text apply convolutional filters over word sequences to capture local patterns (n-gram features). They are parallelizable (unlike RNNs) and excel at classification tasks where local features dominate.

The Attention Mechanism allows models to focus on relevant parts of the input when producing each output. It eliminates the information bottleneck of fixed-size hidden states and enables transformers.



6.2 System Design Interview Questions

Q1: Design a Real-Time LSTM-Based Sequence Prediction Service

System Design Focus: Latency & Performance Scalability
Design a system that serves an LSTM model for real-time text completion (predicting the next word), handling 100K requests per second with P99 latency under 50ms.

Architecture:
LSTM inference is sequential by nature (each time step depends on the previous hidden state), so the optimization focus is on reducing per-token latency and maximizing GPU utilization through batching.
Key Optimizations:

- **Dynamic Batching:** Collect requests in a 5ms window, batch them, and run inference together. GPU utilization jumps from 10% (single request) to 80%+ (batch of 64).
- **KV Cache:** Cache the LSTM hidden state per user session. On the next request, resume from the cached state instead of reprocessing the full context.
- **Model Optimization:** Quantize LSTM weights to INT8 (2x speedup, ¡1% accuracy loss). Export to ONNX Runtime for optimized inference.
- **Speculative Decoding:** Pre-compute top-3 continuations for popular prefixes and cache results.

```
import torch
import torch.nn as nn

class LSTMLanguageModel(nn.Module):
    def __init__(self, vocab_size, embed_dim=256,
                 hidden_dim=512, n_layers=2):
        super().__init__()
        self.embedding = nn.Embedding(
```

```
            vocab_size, embed_dim)
        self.lstm = nn.LSTM(
            embed_dim, hidden_dim,
            n_layers, batch_first=True)
        self.fc = nn.Linear(hidden_dim, vocab_size)

    def forward(self, x, hidden=None):
        emb = self.embedding(x)
        output, hidden = self.lstm(emb, hidden)
        logits = self.fc(output[:, -1, :])
        return logits, hidden

class LSTMServingEngine:
    def __init__(self, model, max_batch_wait_ms=5):
        self.model = model
        self.request_queue = []
        self.max_wait = max_batch_wait_ms / 1000
        self.state_cache = {}  # session -> hidden

    async def predict_next(self, session_id, token_id):
        hidden = self.state_cache.get(session_id)
        x = torch.tensor([[token_id]])
        with torch.no_grad():
            logits, new_hidden = self.model(
                x, hidden)
        self.state_cache[session_id] = new_hidden
        probs = torch.softmax(logits, dim=-1)
        top_k = torch.topk(probs, 5)
        return top_k
```

Scale: Each GPU handles 10K batched requests/sec. 10 GPUs with load balancing = 100K RPS. Session state cache in Redis (1KB per session, 10M active sessions = 10GB).

Q2: Design a GPU Cluster for Deep Learning NLP Training

System Design Focus: Cost Management High Availability

Design a GPU training infrastructure that efficiently trains multiple NLP models concurrently while minimizing cost and handling hardware failures.

Architecture:
A managed GPU cluster with job scheduling, multi-tenancy, and automatic failure recovery.
Key Design:

- **Cluster:** 100 GPU nodes (8 A100s each = 800 GPUs). Organized into pools: training (70%), inference (20%), burst (10%).
- **Scheduler:** Gang scheduling for multi-GPU jobs (all GPUs for a job start simultaneously). Backfill scheduling for small jobs to fill gaps.
- **Fault Tolerance:** Checkpoint every 30 minutes to distributed storage. On node failure, reschedule job to healthy nodes and resume from last checkpoint. Mean time to recovery: ¡5 minutes.
- **Cost:** Mix of on-demand (30%) and spot instances (70%). Spot interruption handler saves checkpoint before termination (2-minute warning).

```python
class GPUClusterManager:
    def __init__(self, total_gpus=800):
        self.pools = {
            "training": int(total_gpus * 0.7),
            "inference": int(total_gpus * 0.2),
            "burst": int(total_gpus * 0.1)
        }
        self.jobs = {}
        self.checkpoints = {}

    def submit_job(self, job_config):
        required_gpus = job_config["num_gpus"]
        pool = self._find_pool(required_gpus)
        if pool:
            job_id = self._allocate(pool,
                                    required_gpus)
            self._setup_checkpointing(
                job_id, interval_min=30)
            return job_id
        return self._queue_job(job_config)

    def handle_node_failure(self, node_id):
        affected_jobs = self._get_jobs_on_node(
            node_id)
        for job_id in affected_jobs:
            last_ckpt = self.checkpoints.get(job_id)
            new_nodes = self._find_replacement_nodes(
                job_id)
```

```
            self._restart_from_checkpoint(
                job_id, last_ckpt, new_nodes)
```

Cost Optimization: Spot instances save 60-70% on GPU costs. With checkpointing, spot interruptions add ¡10% overhead. Auto-scaling reduces cluster by 50% during off-peak hours (nights/weekends). Monthly cost: $150K vs. $400K for all on-demand.

Q3: Design a CNN-Based Text Classification System for Content Moderation

System Design Focus: Scalability Security

Design a content moderation system using TextCNN that classifies user posts into safe/toxic/spam categories at 500K posts per minute, with guardrails against adversarial attacks.

Architecture:
TextCNN is ideal for moderation: fast inference (parallelizable convolutions), good at capturing n-gram patterns that indicate toxicity, and small model footprint.
Multi-Filter Design: Use parallel convolutional filters of sizes 2, 3, 4, and 5 to capture patterns at different n-gram lengths. Max-pool over each filter's output and concatenate for classification.
Adversarial Defense:

- **Character-Level CNN:** Add a character-level model that is robust to intentional misspellings ("h4te", "t0xic").
- **Text Normalization:** Pre-normalize leetspeak, Unicode confusables, and zero-width characters before classification.
- **Ensemble:** Combine word-level and character-level CNN predictions. Flag if they disagree.

```
import torch
import torch.nn as nn
import torch.nn.functional as F

class TextCNN(nn.Module):
    def __init__(self, vocab_size, embed_dim=128,
                 num_classes=3,
                 filter_sizes=[2,3,4,5],
```

```
                    num_filters=100):
        super().__init__()
        self.embedding = nn.Embedding(
            vocab_size, embed_dim)
        self.convs = nn.ModuleList([
            nn.Conv1d(embed_dim, num_filters, fs)
            for fs in filter_sizes])
        self.fc = nn.Linear(
            num_filters * len(filter_sizes),
            num_classes)
        self.dropout = nn.Dropout(0.3)

    def forward(self, x):
        x = self.embedding(x).transpose(1, 2)
        conv_outs = [F.relu(conv(x)).max(dim=2)[0]
                     for conv in self.convs]
        x = torch.cat(conv_outs, dim=1)
        x = self.dropout(x)
        return self.fc(x)
```

Security: Rate limiting per user (max 100 posts/minute). Hash-based deduplication catches spam floods. Model decisions on "toxic" content are logged for audit. Human review for borderline cases (confidence 0.4-0.6).
Scale: TextCNN inference: 50μs per text on GPU in batched mode. At batch size 256: 13ms per batch = 20K predictions/sec per GPU. 25 GPUs handle 500K/min.

Q4: Design an Attention-Based Model Serving System with Dynamic Batching

System Design Focus: **Latency & Performance** **Scalability**
Design a model serving infrastructure for attention-based models that handles variable-length inputs efficiently through smart batching and padding strategies.

Key Challenge: Attention computation is $O(n^2)$ in sequence length. A batch of mixed-length sequences wastes compute on padding tokens. A single long sequence (1024 tokens) batched with short sequences (32 tokens) means 97% of compute is wasted.

Solutions:

- **Length-Based Bucketing:** Sort incoming requests by sequence length. Batch similar-length requests together. Buckets: [0-32], [33-128], [129-512], [513-1024].
- **Dynamic Batching with Padding Budget:** Set a max padding waste budget (20%). Pack requests until adding the next one would exceed the budget.
- **Continuous Batching:** Do not wait for a full batch. Use iteration-level scheduling: as one sequence finishes, immediately add a new one to the batch.

```
import time
from collections import defaultdict

class SmartBatcher:
    BUCKETS = [(32, 64), (128, 32), (512, 16),
               (1024, 8)]  # (max_len, batch_size)

    def __init__(self, max_wait_ms=10):
        self.queues = defaultdict(list)
        self.max_wait = max_wait_ms / 1000

    def add_request(self, request):
        seq_len = len(request.tokens)
        bucket = self._find_bucket(seq_len)
        self.queues[bucket].append(request)

    def get_batch(self):
        for max_len, batch_size in self.BUCKETS:
            queue = self.queues[max_len]
            if len(queue) >= batch_size:
                batch = queue[:batch_size]
                self.queues[max_len] = queue[batch_size:]
                return self._pad_batch(batch, max_len)
        # Timeout: flush partial batches
        for max_len, _ in self.BUCKETS:
            if self.queues[max_len]:
                oldest = self.queues[max_len][0]
                if time.time() - oldest.timestamp \
                   > self.max_wait:
                    batch = self.queues[max_len]
                    self.queues[max_len] = []
                    return self._pad_batch(
                        batch, max_len)
```

```
        return None

    def _find_bucket(self, seq_len):
        for max_len, _ in self.BUCKETS:
            if seq_len <= max_len:
                return max_len
        return self.BUCKETS[-1][0]
```

Performance Impact: Length bucketing reduces wasted compute by 40-60% compared to naive padding. Combined with continuous batching, GPU utilization goes from 30% to 75%, effectively tripling throughput per GPU.

Q5: Design a Bidirectional LSTM Pipeline for Sequence Labeling

System Design Focus: **High Availability** **Communication Between Services**
Design a production BiLSTM-CRF system for sequence labeling tasks (NER, POS tagging) that serves as a shared microservice for multiple downstream teams.

Architecture:
A shared NLP microservice with a multi-model registry. Different teams use different BiLSTM-CRF models (NER team uses one, POS team uses another) but share the serving infrastructure.
Key Design:

- **Multi-Model Serving:** Each model loaded into a separate GPU memory space. Request routing based on the model ID in the request header.
- **gRPC Interface:** Binary serialization for token sequences. Streaming support for long documents (process sentence by sentence, stream results back).
- **Health & Failover:** Per-model health checks. If the NER model fails, POS tagging continues unaffected. Dead model instances restart automatically.

```
import torch
import torch.nn as nn

class BiLSTMCRF(nn.Module):
    def __init__(self, vocab_size, tag_size,
                 embed_dim=256, hidden_dim=512):
        super().__init__()
        self.embedding = nn.Embedding(
```

```
            vocab_size, embed_dim)
        self.lstm = nn.LSTM(
            embed_dim, hidden_dim // 2,
            num_layers=2, bidirectional=True,
            batch_first=True)
        self.hidden2tag = nn.Linear(
            hidden_dim, tag_size)
        # CRF transition matrix
        self.transitions = nn.Parameter(
            torch.randn(tag_size, tag_size))

    def forward(self, x):
        emb = self.embedding(x)
        lstm_out, _ = self.lstm(emb)
        emissions = self.hidden2tag(lstm_out)
        # Viterbi decode with CRF
        return self.viterbi_decode(emissions)

    def viterbi_decode(self, emissions):
        batch_size, seq_len, n_tags = emissions.shape
        dp = emissions[:, 0, :]
        backpointers = []
        for t in range(1, seq_len):
            scores = dp.unsqueeze(2) + \
                     self.transitions.unsqueeze(0)
            dp, bp = scores.max(dim=1)
            dp = dp + emissions[:, t, :]
            backpointers.append(bp)
        best_tags = [dp.argmax(dim=1)]
        for bp in reversed(backpointers):
            best_tags.insert(
                0, bp.gather(
                    1, best_tags[0].unsqueeze(1)
                ).squeeze())
        return torch.stack(best_tags, dim=1)
```

Communication: gRPC with Protocol Buffers. Service discovery via Consul. Circuit breaker pattern per model endpoint. Upstream services degrade gracefully when downstream models are unavailable.

Q6: Design a Model Distillation Pipeline for Edge Deployment

System Design Focus: **Cost Management** **Latency & Performance**
Design a system that distills a large NLP model (BERT-large, 340M params) into a lightweight model suitable for mobile and edge devices.

Architecture:
Knowledge distillation transfers the "knowledge" of a large teacher model into a smaller student model by training the student to match the teacher's soft probability distributions.
Distillation Pipeline:

- **Teacher:** BERT-large (340M params, 1.3GB). Runs on GPU. Generates soft labels (full probability distribution over classes, temperature=4).
- **Student Options:** DistilBERT (66M params, 6 layers), TinyBERT (14.5M params, 4 layers), or a BiLSTM (5M params).
- **Training Loss:** $\mathcal{L} = \alpha \cdot \text{CE}(\text{student}, \text{hard\_labels}) + (1 - \alpha) \cdot \text{KL}(\text{student}, \text{teacher\_soft\_labels})$.
- **Post-Training:** Quantize student to INT8. Prune unimportant weights (magnitude pruning, 50% sparsity).

```
import torch
import torch.nn.functional as F

class DistillationTrainer:
    def __init__(self, teacher, student,
                 temperature=4.0, alpha=0.5):
        self.teacher = teacher.eval()
        self.student = student
        self.T = temperature
        self.alpha = alpha

    def distill_step(self, batch, hard_labels):
        # Teacher soft labels
        with torch.no_grad():
            teacher_logits = self.teacher(batch)
            soft_targets = F.softmax(
                teacher_logits / self.T, dim=-1)
        # Student prediction
        student_logits = self.student(batch)
        student_soft = F.log_softmax(
```

```
            student_logits / self.T, dim=-1)
        # Combined loss
        distill_loss = F.kl_div(
            student_soft, soft_targets,
            reduction='batchmean') * (self.T ** 2)
        hard_loss = F.cross_entropy(
            student_logits, hard_labels)
        loss = self.alpha * hard_loss + \
               (1 - self.alpha) * distill_loss
        return loss
```

Results: DistilBERT retains 97% of BERT-base quality at 60% of the size and 2x inference speed. TinyBERT retains 92% at 13% of the size. BiLSTM student retains 85% but runs on CPU with 10x lower latency.
Cost Impact: Replacing one BERT-large GPU instance ($3/hour) with a TinyBERT CPU instance ($0.10/hour) saves 97% on serving costs. For 100 inference instances, that is $200K/year in savings.

Q7: Design a GRU-Based Streaming NLP System

System Design Focus: Communication Between Services | High Availability
Design a system using GRU networks that processes streaming text data (live chat, social media feeds) with stateful sequence processing across messages.

Key Challenge: In streaming, messages arrive one at a time. The GRU hidden state must persist between messages to maintain context, but state management across a distributed system is complex.
Design:

- **Stateful Routing:** Use sticky sessions (consistent hashing on conversation ID) to route all messages from a conversation to the same worker. The worker maintains the GRU hidden state in memory.
- **State Checkpointing:** Periodically checkpoint hidden states to Redis. On worker failure, a new worker loads the checkpoint and resumes.
- **State Expiry:** Expire hidden states after 30 minutes of inactivity to prevent memory leaks. Reset state for new conversations.

```
import torch
```

```python
import torch.nn as nn

class StreamingGRU(nn.Module):
    def __init__(self, input_dim, hidden_dim=256):
        super().__init__()
        self.gru = nn.GRU(input_dim, hidden_dim,
                          batch_first=True)
        self.classifier = nn.Linear(hidden_dim, 3)

    def step(self, token_embedding, hidden_state):
        """Process one token, return updated state."""
        output, new_hidden = self.gru(
            token_embedding.unsqueeze(0).unsqueeze(0),
            hidden_state)
        prediction = self.classifier(output.squeeze())
        return prediction, new_hidden

class StatefulStreamProcessor:
    def __init__(self, model):
        self.model = model
        self.states = {}  # conv_id -> hidden_state

    def process_message(self, conv_id, tokens):
        hidden = self.states.get(
            conv_id,
            torch.zeros(1, 1, 256))
        for token in tokens:
            pred, hidden = self.model.step(
                token, hidden)
        self.states[conv_id] = hidden.detach()
        return pred
```

High Availability: If a worker dies, Redis state checkpoint allows recovery within 500ms. During failover, messages buffer in Kafka and are replayed after state restoration.

Q8: Design a Multi-Task Deep Learning NLP System

System Design Focus: Scalability Database Bottlenecks

Design a system where a single shared deep learning model performs multiple NLP tasks (sentiment, NER, topic classification) simultaneously, reducing infrastructure

costs.

Architecture:
A shared encoder (LSTM or Transformer) with task-specific heads. The encoder is trained jointly on all tasks, learning universal text representations.
Key Design:

- **Shared Encoder:** 3-layer BiLSTM producing contextual representations. Trained with weighted multi-task loss.
- **Task Heads:** Sentiment (classification head), NER (token-level CRF head), Topic (multi-label head). Each head is 5% of total parameters.
- **Selective Execution:** API accepts a task list per request. Only run requested heads, skipping unnecessary computation.
- **Loss Balancing:** Dynamic weight adjustment using uncertainty-based weighting (Kendall et al.) to prevent one task from dominating training.

```python
import torch.nn as nn

class MultiTaskNLP(nn.Module):
    def __init__(self, vocab_size, hidden=512):
        super().__init__()
        self.shared_encoder = nn.LSTM(
            256, hidden // 2, num_layers=3,
            bidirectional=True, batch_first=True)
        self.embedding = nn.Embedding(vocab_size, 256)
        # Task-specific heads
        self.sentiment_head = nn.Linear(hidden, 3)
        self.ner_head = nn.Linear(hidden, 9)  # BIO
        self.topic_head = nn.Linear(hidden, 50)
        # Uncertainty weights for loss balancing
        self.log_vars = nn.Parameter(torch.zeros(3))

    def forward(self, x, tasks=["all"]):
        emb = self.embedding(x)
        encoded, _ = self.shared_encoder(emb)
        results = {}
        if "sentiment" in tasks or "all" in tasks:
            pooled = encoded.mean(dim=1)
            results["sentiment"] = \
                self.sentiment_head(pooled)
        if "ner" in tasks or "all" in tasks:
            results["ner"] = self.ner_head(encoded)
```

```
        if "topic" in tasks or "all" in tasks:
            pooled = encoded.mean(dim=1)
            results["topic"] = \
                self.topic_head(pooled)
        return results
```

Cost Savings: One multi-task model replaces 3 single-task models. GPU memory: 1 model (2GB) vs. 3 models (6GB). Inference: 1 forward pass through the shared encoder instead of 3. Total saving: 60% on serving infrastructure.

Q9: Design a Model Versioning and Rollback System for Deep Learning

System Design Focus: **Data Consistency** **Observability**
Design a system that manages multiple versions of deep learning NLP models in production with instant rollback capabilities.

Architecture:
A model versioning system with blue-green deployment and traffic shifting.
Key Design:

- **Version Registry:** Each model version stored in S3 with metadata in PostgreSQL (version, training data hash, metrics, deployment history).
- **Multi-Version Serving:** GPU instances can hold 2-3 model versions simultaneously (if memory allows). Traffic splits between versions via a routing layer.
- **Canary Deployment:** New version gets 5% → 25% → 50% → 100% traffic over 4 hours. Automated rollback if error rate or latency degrades.
- **Instant Rollback:** Previous version stays loaded in memory. Rollback is a config change in the routing layer (100ms).

```
class ModelVersionManager:
    def __init__(self):
        self.versions = {}  # version -> loaded model
        self.traffic_split = {}  # version -> percentage
        self.active_versions = []

    def deploy_canary(self, new_version, model_path):
        # Load new version alongside current
```

```
        self.versions[new_version] = \
            torch.load(model_path)
        self.traffic_split[new_version] = 0.05
        # Reduce current version traffic
        current = self.active_versions[-1]
        self.traffic_split[current] = 0.95
        self.active_versions.append(new_version)

    def route_request(self, request):
        """Route based on traffic split."""
        import random
        roll = random.random()
        cumulative = 0
        for version, pct in \
            self.traffic_split.items():
            cumulative += pct
            if roll <= cumulative:
                return self.versions[version]
        return self.versions[self.active_versions[-1]]

    def rollback(self, to_version):
        self.traffic_split = {to_version: 1.0}
```

Observability: Per-version metrics: latency, error rate, prediction distribution, confidence scores. Anomaly detection alerts on distribution shifts between versions. A/B test analysis for version comparisons.

Q10: Design a Transfer Learning Infrastructure for Domain Adaptation

System Design Focus: Cost Management Scalability

Design a platform that enables teams to fine-tune pretrained NLP models on domain-specific data efficiently, with shared infrastructure and standardized workflows.

Architecture:
A centralized fine-tuning platform with a pretrained model zoo, standardized training templates, and shared GPU resources.
Components:

- **Model Zoo:** Curated pretrained models (BERT, RoBERTa, XLM-R) stored in a central registry. Teams select a base model and fine-tune on their domain data.
- **Training Templates:** Pre-configured training scripts for common tasks (classification, NER, QA). Teams only provide data and hyperparameters.
- **Resource Pooling:** Shared GPU cluster with fair-share scheduling. Each team gets a GPU quota; unused quota is available to others.
- **Experiment Tracking:** All fine-tuning runs logged with hyperparameters, metrics, and artifacts. Cross-team search to avoid duplicate work.

```
class FineTuningPlatform:
    def __init__(self, model_zoo, gpu_scheduler):
        self.zoo = model_zoo
        self.scheduler = gpu_scheduler

    def create_finetuning_job(self, config):
        # Load pretrained model from zoo
        base_model = self.zoo.get_model(
            config["base_model"])
        # Validate data
        dataset = self.validate_dataset(
            config["data_path"])
        # Estimate resources
        gpu_hours = self.estimate_cost(
            base_model, dataset)
        # Submit job
        job = {
            "base_model": config["base_model"],
            "task": config["task_type"],
            "dataset_size": len(dataset),
            "estimated_gpu_hours": gpu_hours,
            "hyperparams": config.get("hyperparams",
                self.default_hyperparams(
                    config["task_type"]))
        }
        return self.scheduler.submit(job)

    def default_hyperparams(self, task):
        defaults = {
            "classification": {
                "lr": 2e-5, "epochs": 3,
                "batch_size": 32},
```

```
            "ner": {
                "lr": 3e-5, "epochs": 5,
                "batch_size": 16},
        }
        return defaults.get(task,
            defaults["classification"])
```

Cost: Shared infrastructure reduces per-team GPU costs by 40-60%. Fine-tuning a BERT model for a classification task: 2 GPU-hours ($6). Compare to training from scratch: 500 GPU-hours ($1500). Transfer learning saves 99.6%.

7 Transformers & Large Language Models

linkedin.com/in/lamhotsiagian

The transformer architecture redefined what is possible in NLP. BERT, GPT, T5, and their descendants power virtually every state-of-the-art NLP system today. Deploying these models at scale is one of the most demanding system design challenges in modern engineering—every millisecond of latency and every byte of memory must be accounted for.

7.1 Chapter Overview

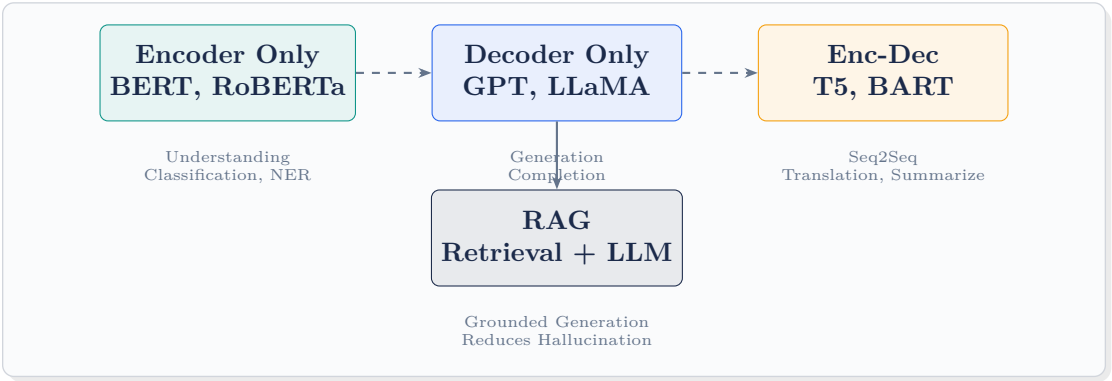
The transformer architecture, introduced in "Attention Is All You Need" (2017), replaced recurrence with self-attention. Every token attends to every other token in parallel, enabling massive GPU utilization and eliminating the sequential bottleneck of RNNs.

BERT (Bidirectional Encoder Representations from Transformers) is an encoder-only model trained with masked language modeling. It produces rich, bidirectional contextual representations ideal for understanding tasks: classification, NER, QA.

GPT (Generative Pre-trained Transformer) is a decoder-only model trained autoregressively. It excels at generation: text completion, summarization, code generation, dialogue.

T5 (Text-to-Text Transfer Transformer) frames all NLP tasks as text-to-text. Translation, summarization, QA—everything becomes "input text → output text."

RAG (Retrieval-Augmented Generation) grounds LLM generation in retrieved facts, reducing hallucinations and enabling knowledge-intensive applications.



The LLM Serving Challenge

A 70B-parameter LLM requires 140GB GPU memory in float16. Serving it at 1000 QPS requires careful engineering: model parallelism, KV-cache management, continuous batching, and speculative decoding. This is where NLP system design separates good engineers from great ones.

7.2 System Design Interview Questions

Q1: Design an LLM Inference Infrastructure for 10,000 Concurrent Users

System Design Focus: Scalability Latency & Performance

Design a serving system for a 70B-parameter language model that handles 10,000 concurrent users generating text in real time, with P50 time-to-first-token under 500ms.

Architecture:

LLM serving at scale requires tensor parallelism, continuous batching, and a KV-cache management system. This is fundamentally different from classical model serving.

Hardware Layout:

- **Tensor Parallelism:** A 70B model in float16 = 140GB. Spread across 8× A100-80GB GPUs using tensor parallelism (split each attention head and FFN layer across GPUs).
- **Pipeline Parallelism:** For long sequences, split model layers across GPU

nodes. Layers 1-40 on Node A, layers 41-80 on Node B.

- **Replica Groups:** Each 8-GPU node serves one model replica. 20 replica nodes = 160 GPUs total, handling 10K concurrent users.

Continuous Batching (vLLM-style): Unlike classical batching (wait for full batch), continuous batching adds new requests mid-generation. As one sequence finishes, a new one immediately fills that slot. GPU utilization: 85%+ vs. 20-30% with naive batching.

KV-Cache Management: The key-value cache (attention outputs) grows linearly with generated length. PagedAttention manages KV-cache in fixed-size pages (like OS virtual memory), eliminating fragmentation and enabling larger effective batch sizes.

```python
# Conceptual vLLM-style serving engine
class LLMServingEngine:
    def __init__(self, model, num_gpus=8):
        self.model = model
        self.kv_cache = PagedKVCache(
            num_blocks=4096,
            block_size=16,    # tokens per block
            num_heads=64,
            head_dim=128)
        self.scheduler = ContinuousBatchScheduler()

    async def generate(self, prompt: str,
                       max_tokens: int = 256) -> str:
        request = GenerationRequest(
            prompt=prompt,
            max_tokens=max_tokens)
        # Allocate KV-cache pages
        self.kv_cache.allocate(
            request.id, max_tokens)
        # Add to continuous batch
        self.scheduler.add_request(request)
        # Stream tokens as they are generated
        tokens = []
        async for token in self.scheduler.stream():
            tokens.append(token)
            if token.is_stop:
                break
        self.kv_cache.free(request.id)
        return self.decode(tokens)
```

Capacity: At 10K concurrent users, average generation of 200 tokens, at 50 tokens/sec per user = 500K tokens/sec total. Each 8-GPU node generates 50K tokens/sec. 20 nodes provide 2x headroom for spikes.

Q2: Design a RAG (Retrieval-Augmented Generation) System for Enterprise Search

System Design Focus: Data Storage & Retrieval Latency & Performance
Design a RAG system that answers employee questions by retrieving relevant company documents and generating accurate, cited answers, processing 5,000 queries per hour.

Architecture:
RAG has two phases: **Retrieval** (find relevant documents using vector search) and **Generation** (LLM produces an answer grounded in retrieved docs).
Components:

- **Document Store:** Company documents (PDFs, wikis, Confluence pages) chunked into 256-token passages with 64-token overlap. Chunking preserves context at boundaries.
- **Embedding Model:** E5-large or BGE-large encodes both queries and passages into the same 1024-dim space. 5M passages for a large enterprise.
- **Vector Index:** FAISS HNSW index over 5M passage embeddings. Retrieval: top-5 passages per query.
- **LLM Generator:** An instruction-tuned model (Llama-3-70B or GPT-4) generates the answer given the query and retrieved passages. Prompt includes source citations.
- **Re-ranker:** Cross-encoder re-ranks retrieved passages before injection into the LLM context, improving answer quality by 10-15%.

```
from transformers import AutoTokenizer, pipeline
import faiss, numpy as np

class RAGSystem:
    def __init__(self, embedding_model,
                 vector_index, llm, reranker):
        self.embedder = embedding_model
        self.index = vector_index
```

```
        self.llm = llm
        self.reranker = reranker

    def answer(self, query: str) -> dict:
        # Step 1: Encode query
        q_emb = self.embedder.encode([query])
        # Step 2: Retrieve top-20 passages
        distances, ids = self.index.search(
            q_emb, k=20)
        passages = self.fetch_passages(ids[0])
        # Step 3: Re-rank to top-5
        ranked = self.reranker.rerank(
            query, passages)[:5]
        # Step 4: Generate grounded answer
        context = "\n\n".join(
            [f"[{i+1}] {p['text']}"
             for i, p in enumerate(ranked)])
        prompt = f"""Answer based on context:

Context:
{context}

Question: {query}

Answer with citations [1], [2], etc.:"""
        answer = self.llm.generate(prompt,
                                   max_tokens=512)
        return {
            "answer": answer,
            "sources": [p["doc_id"]
                        for p in ranked]
        }
```

Latency Budget:

- Embedding query: 20ms
- Vector search: 10ms
- Re-ranking (5 pairs): 30ms
- LLM generation (200 tokens): 2,000ms
- Total: 2,100ms (well within typical RAG SLA of 5s)

Caching: Cache embeddings for frequent queries (LRU, 100K entries). Cache full answers for identical questions (24h TTL). Cache hit rate for enterprise Q&A: 30-40%.

Q3: Design a Fine-Tuning Platform for Domain-Specific LLMs

System Design Focus: Cost Management Scalability
Design a platform that allows teams to fine-tune LLMs on proprietary data using parameter-efficient methods (LoRA, QLoRA), with cost guardrails and experiment tracking.

Architecture:
Full fine-tuning of a 70B model requires 28+ A100 GPUs and costs $5K+ per run. Parameter-Efficient Fine-Tuning (PEFT) methods like LoRA reduce this to 1-2 GPUs and $50-200 per run.
LoRA Overview: Instead of updating all parameters, LoRA adds low-rank adapter matrices (rank 8-64) to attention layers. Only adapters are trained; the base model is frozen. Adapter size: 0.1% of full model. Quality: 95% of full fine-tune for most tasks.
Platform Components:

- **Job Submission API:** Teams specify base model, dataset, task type, and budget. Platform auto-selects LoRA rank and learning rate.
- **Cost Guardrails:** Real-time cost monitoring. Alert at 75% of budget; auto-stop at 100%.
- **Experiment Tracker:** MLflow integration. All runs tracked with hyperparameters, loss curves, evaluation metrics, and cost.
- **Adapter Registry:** Trained LoRA adapters (100MB each) stored in S3. Can be combined with different base models (adapter portability).

```
from peft import LoraConfig, get_peft_model
from transformers import AutoModelForCausalLM
import torch

def create_lora_model(base_model_name,
                      r=16, alpha=32,
                      target_modules=["q_proj",
                                      "v_proj"]):
    model = AutoModelForCausalLM.from_pretrained(
        base_model_name,
        load_in_4bit=True,     # QLoRA: 4-bit base
        torch_dtype=torch.float16,
        device_map="auto")

    lora_config = LoraConfig(
```

```
        r=r,
        lora_alpha=alpha,
        target_modules=target_modules,
        lora_dropout=0.05,
        bias="none",
        task_type="CAUSAL_LM")

    model = get_peft_model(model, lora_config)
    model.print_trainable_parameters()
    # Typical output: trainable=0.1%, total=100%
    return model

class FineTuneJob:
    def __init__(self, config, budget_usd):
        self.config = config
        self.budget = budget_usd
        self.cost_tracker = CostTracker(
            gpu_cost_per_hour=3.0)

    def train_step(self, batch):
        if self.cost_tracker.cost > self.budget:
            raise BudgetExceededError()
        # Normal training step
        loss = self.model(**batch).loss
        loss.backward()
        return loss.item()
```

Cost Comparison: QLoRA fine-tuning of Llama-3-70B: 1 A100, 4 hours, $12. Full fine-tuning: 32 A100s, 4 hours, $384. QLoRA provides 97% cost reduction with ¡5% quality trade-off for most tasks.

Q4: Design a Prompt Management and Caching System for LLM APIs

System Design Focus: **Cost Management** **Latency & Performance**

Design a system that manages LLM prompts across an organization, caches responses to identical or semantically similar prompts, and reduces API costs.

Architecture:
A semantic caching layer between application code and the LLM API. Similar prompts share cached responses, dramatically reducing API calls.
Two-Level Cache:

- **Exact Cache:** SHA-256 hash of the exact prompt. Redis lookup in 1ms. Hit rate for templated prompts: 30-50%.
- **Semantic Cache:** Embed the prompt and search a vector index for similar cached prompts (cosine similarity > 0.95). If found, return cached response. Adds 15ms but catches near-duplicates (paraphrased prompts).

Prompt Management:

- **Template Registry:** Versioned prompt templates stored in Git. Teams reference templates by ID instead of hardcoding prompts.
- **A/B Testing:** Route a fraction of traffic to alternative prompt versions. Measure output quality and cost.
- **Token Budget Enforcement:** Automatically truncate context to fit within model's context window. Prioritize recent content.

```python
import hashlib
import numpy as np

class LLMGateway:
    def __init__(self, llm_client,
                 redis_client, vector_index,
                 embedder):
        self.llm = llm_client
        self.redis = redis_client
        self.index = vector_index
        self.embedder = embedder

    def complete(self, prompt: str,
                 max_tokens: int = 256) -> dict:
        # Level 1: Exact cache
        key = hashlib.sha256(
            prompt.encode()).hexdigest()
        cached = self.redis.get(f"llm:{key}")
        if cached:
            return {"response": cached,
                    "cache": "exact", "cost": 0}

        # Level 2: Semantic cache
        q_emb = self.embedder.encode([prompt])
```

```
        dists, ids = self.index.search(q_emb, k=1)
        if dists[0][0] < 0.05:  # cosine dist < 0.05
            cached_resp = self.fetch_cached(ids[0][0])
            return {"response": cached_resp,
                    "cache": "semantic", "cost": 0}

        # Cache miss: call LLM API
        response = self.llm.complete(
            prompt, max_tokens=max_tokens)
        # Store in both caches
        self.redis.setex(f"llm:{key}", 3600,
                         response.text)
        self.index.add_with_text(
            q_emb, response.text, key)
        return {"response": response.text,
                "cache": "miss",
                "cost": response.usage.total_tokens}
```

Cost Impact: At $0.002 per 1K tokens (GPT-4-turbo), a semantic cache with 40% hit rate saves $0.008 per 10 requests. At 1M requests/day: $800/day savings. ROI on caching infrastructure: measured in days.

Q5: Design a BERT-Based Search and Ranking System

System Design Focus: Database Bottlenecks Latency & Performance
Design a two-stage search system where BM25 retrieves candidates and a BERT cross-encoder re-ranks them, serving 50K queries per second.

Architecture:
A pipeline with fast sparse retrieval feeding a precise (but slower) neural re-ranker. The key insight: BERT cannot score 10M documents per query, but it can score 100 candidates in reasonable time.
Stage 1 – BM25 Retrieval: Elasticsearch cluster (12 shards) returns top-200 candidates per query in 10ms. This is the recall stage.
Stage 2 – BERT Re-ranking: Cross-encoder BERT scores (query, document) pairs jointly. Batch 200 pairs through BERT in one GPU forward pass. Latency: 50ms for 200 pairs on A100.
Optimization:

- **DistilBERT for Re-ranking:** Distilled to 6 layers. 2x faster, 98% quality retention for re-ranking tasks.
- **Cascading Re-rank:** Use DistilBERT to reduce 200 → 20. Then use full BERT to re-rank 20 → 10. Balances quality and speed.
- **GPU Batching:** Group queries by candidate count. Batch multiple queries' candidates into one GPU forward pass.

```python
from transformers import AutoTokenizer, AutoModel
import torch

class BERTReranker:
    def __init__(self, model_name):
        self.tokenizer = AutoTokenizer.from_pretrained(
            model_name)
        self.model = AutoModel.from_pretrained(
            model_name).cuda()
        self.score_head = torch.nn.Linear(768, 1)

    def rerank(self, query: str,
               candidates: list[str],
               top_k: int = 10):
        pairs = [(query, c) for c in candidates]
        # Tokenize all pairs at once (batched)
        encoded = self.tokenizer(
            [p[0] for p in pairs],
            [p[1] for p in pairs],
            padding=True, truncation=True,
            max_length=512, return_tensors="pt"
        ).to("cuda")

        with torch.no_grad():
            outputs = self.model(**encoded)
            cls_emb = outputs.last_hidden_state[:,0,:]
            scores = self.score_head(cls_emb).squeeze()

        ranked = sorted(
            zip(candidates, scores.cpu().tolist()),
            key=lambda x: -x[1])
        return ranked[:top_k]
```

Scale: Each GPU re-ranker handles 500 queries/sec (200 candidates each). 100 GPUs = 50K QPS. Stage 1 Elasticsearch is stateless and scales independently.

Q6: Design a Multi-Tenant LLM API Gateway

System Design Focus: **Security** **High Availability**
Design an API gateway for an LLM service that handles authentication, rate limiting, usage metering, and abuse prevention for thousands of API clients.

Architecture:
A horizontally scaled gateway layer that sits in front of LLM serving workers. Handles all cross-cutting concerns without adding latency to the generation path.
Key Components:

- **Authentication:** API key validation against a Redis cache (invalidation propagates within 30s). JWT for OAuth flows.
- **Rate Limiting:** Token bucket algorithm per API key. Limits: requests/minute, tokens/minute, tokens/day. Redis Lua scripts for atomic decrement.
- **Usage Metering:** Every request logs (api_key, input_tokens, output_tokens, latency, model) to a Kafka topic. Billing service aggregates daily.
- **Abuse Prevention:** Prompt injection detection (classifies malicious prompts). PII detection before logging. Content filtering on outputs.

```python
import redis
import time

class RateLimiter:
    def __init__(self, redis_client):
        self.redis = redis_client

    def check_rate_limit(self, api_key: str,
                         tokens_requested: int,
                         limits: dict) -> bool:
        now = int(time.time())
        minute_key = f"rl:{api_key}:min:{now//60}"
        day_key = f"rl:{api_key}:day:{now//86400}"

        # Atomic check-and-increment via pipeline
        pipe = self.redis.pipeline()
        pipe.incrby(minute_key, tokens_requested)
        pipe.expire(minute_key, 120)
        pipe.incrby(day_key, tokens_requested)
        pipe.expire(day_key, 90000)
        results = pipe.execute()
```

```
22
23          minute_total = results[0]
24          day_total = results[2]
25
26          if minute_total > limits["tokens_per_minute"]:
27              return False, "minute_limit_exceeded"
28          if day_total > limits["tokens_per_day"]:
29              return False, "daily_limit_exceeded"
30          return True, None
31
32  class LLMGateway:
33      def __init__(self, rate_limiter, auth_service,
34                   llm_workers):
35          self.rate_limiter = rate_limiter
36          self.auth = auth_service
37          self.workers = llm_workers
38
39      def handle_request(self, request):
40          # Auth
41          api_key_info = self.auth.validate(
42              request.api_key)
43          if not api_key_info:
44              return {"error": "Unauthorized"}, 401
45          # Rate limit
46          allowed, reason = \
47              self.rate_limiter.check_rate_limit(
48                  request.api_key,
49                  len(request.prompt.split()),
50                  api_key_info.limits)
51          if not allowed:
52              return {"error": reason}, 429
53          # Route to LLM worker
54          return self.workers.dispatch(request)
```

High Availability: Gateway deployed across 3 AZs with a Global Load Balancer. Redis cluster for rate limiting state (6 shards, 3 replicas). If Redis is unavailable, gateway fails open (passes requests through) with degraded rate limiting.

Q7: Design a Speculative Decoding System for LLM Acceleration

System Design Focus: **Latency & Performance** **Cost Management**
Design a serving system that uses speculative decoding to reduce LLM generation latency by 2-3x without sacrificing output quality.

Architecture:
Speculative decoding uses a small "draft" model to generate multiple tokens speculatively, which the large "target" model then verifies in a single parallel forward pass.

How It Works:

1. Draft model (e.g., 7B) generates K=4 candidate tokens in K sequential forward passes.
2. Target model (e.g., 70B) verifies all K tokens in *one* parallel forward pass (because attention is parallel).
3. Accept all tokens up to the first one that diverges from the target distribution. Reject remaining.
4. Net effect: when draft is accurate 70%+, effective generation speed is 2-3x higher than target-model-only decoding.

```python
import torch
import torch.nn.functional as F

class SpeculativeDecoder:
    def __init__(self, draft_model, target_model,
                 num_speculative_tokens=4):
        self.draft = draft_model
        self.target = target_model
        self.K = num_speculative_tokens

    def generate_step(self, input_ids):
        # Step 1: Draft K tokens
        draft_tokens = []
        draft_probs = []
        current_ids = input_ids
        for _ in range(self.K):
            with torch.no_grad():
                draft_out = self.draft(current_ids)
            probs = F.softmax(
                draft_out.logits[:, -1, :], dim=-1)
```

```
            token = torch.multinomial(probs, 1)
            draft_tokens.append(token)
            draft_probs.append(probs)
            current_ids = torch.cat(
                [current_ids, token], dim=-1)

        # Step 2: Target model verifies all K tokens
        all_ids = current_ids
        with torch.no_grad():
            target_out = self.target(all_ids)
        target_probs = F.softmax(
            target_out.logits[:, -self.K-1:-1, :],
            dim=-1)

        # Step 3: Accept/reject via sampling
        accepted = []
        for i, (d_tok, d_prob, t_prob) in enumerate(
                zip(draft_tokens, draft_probs,
                    target_probs)):
            accept_prob = torch.min(
                torch.ones(1),
                t_prob[0, d_tok] / d_prob[0, d_tok])
            if torch.rand(1) < accept_prob:
                accepted.append(d_tok)
            else:
                # Sample from corrected distribution
                corrected = F.relu(t_prob - d_prob)
                corrected /= corrected.sum()
                accepted.append(
                    torch.multinomial(corrected, 1))
                break
        return torch.cat(accepted, dim=-1)
```

Cost Impact: With speculative decoding, a 70B model generates tokens at the same GPU-time cost as before, but produces 2-3x more tokens per second. Effective cost per token drops by 50-67%. At scale (1B tokens/day), this saves $1M+/year.

Q8: Design a Transformer Model Quantization and Optimization Pipeline

System Design Focus: **Cost Management** **Latency & Performance**
Design a pipeline that takes a trained transformer model and systematically applies quantization, pruning, and compilation optimizations for production deployment.

Optimization Pipeline: Train → Post-Training Quantization → Weight Pruning → TensorRT/ONNX Compilation → Benchmark → Deploy.
Quantization:

- **INT8 Static:** Calibrate scale factors with representative data. 2x memory reduction, 1.5-2x speedup, ¡1% accuracy loss on most tasks.
- **INT4 (GPTQ/AWQ):** 4-bit weight quantization for LLMs. 4x memory reduction. Accuracy loss varies: ¡2% for instruction-following tasks.
- **FP8:** NVIDIA H100 supports FP8 natively. Near-FP16 quality with 2x throughput.

```python
import torch
from optimum.quanto import quantize, freeze, qint8

class ModelOptimizationPipeline:
    def __init__(self, model, tokenizer):
        self.model = model
        self.tokenizer = tokenizer

    def apply_int8_quantization(self):
        """Apply INT8 post-training quantization."""
        quantize(self.model, weights=qint8,
                 activations=qint8)
        freeze(self.model)
        return self.model

    def export_to_onnx(self, output_path):
        """Export for TensorRT optimization."""
        dummy = self.tokenizer(
            "Sample text", return_tensors="pt")
        torch.onnx.export(
            self.model,
            (dummy["input_ids"],
             dummy["attention_mask"]),
            output_path,
```

```
            opset_version=14,
            input_names=["input_ids",
                         "attention_mask"],
            output_names=["logits"],
            dynamic_axes={
                "input_ids": {0: "batch",
                               1: "sequence"},
                "logits": {0: "batch"}
            })

    def benchmark(self, n_runs=100):
        import time
        latencies = []
        for _ in range(n_runs):
            t0 = time.perf_counter()
            self.model(
                input_ids=torch.ones(1, 128,
                    dtype=torch.long).cuda())
            latencies.append(
                time.perf_counter() - t0)
        return {
            "p50": sorted(latencies)[n_runs//2],
            "p99": sorted(latencies)[int(n_runs*0.99)]
        }
```

Pipeline Results (BERT-base benchmark):

- FP32 baseline: 12ms P50, 1.2GB memory
- FP16: 7ms P50, 600MB memory (1.7x faster)
- INT8: 4ms P50, 300MB memory (3x faster)
- INT8 + TensorRT: 2ms P50, 300MB memory (6x faster)

Q9: Design a Transformer-Based Document Processing Pipeline

System Design Focus: Scalability Communication Between Services

Design a pipeline that uses transformer models to process millions of long documents (10K+ tokens) daily for information extraction, summarization, and classification.

Key Challenge: Standard transformers have a fixed context window (512-4096 tokens). Documents of 10K+ tokens require special handling.
Strategies:

- **Sliding Window:** Process document in overlapping 512-token windows. Merge results (majority voting for classification, union for extraction).
- **Hierarchical:** First, chunk and summarize. Then, process the summaries. Two-stage: sentence-level → document-level.
- **Long-Context Models:** Longformer (4096 tokens) or BigBird (4096 tokens) with sparse attention. Fine-tune for the task.

```python
class LongDocumentProcessor:
    def __init__(self, model, tokenizer,
                 window=512, overlap=64):
        self.model = model
        self.tokenizer = tokenizer
        self.window = window
        self.overlap = overlap

    def process(self, document: str) -> dict:
        tokens = self.tokenizer.encode(document)
        if len(tokens) <= self.window:
            return self.model_predict(tokens)
        # Sliding window over long document
        window_results = []
        for start in range(0, len(tokens),
                           self.window - self.overlap):
            end = min(start + self.window, len(tokens))
            chunk = tokens[start:end]
            result = self.model_predict(chunk)
            window_results.append(result)
        return self.merge_results(window_results)

    def merge_results(self, results):
        # Merge classification: weighted vote
        all_labels = [r["label"] for r in results]
        all_scores = [r["score"] for r in results]
        # Weight by confidence score
        from collections import Counter
        weighted = Counter()
        for label, score in zip(all_labels,
                                all_scores):
            weighted[label] += score
```

```
        return {
            "label": weighted.most_common(1)[0][0],
            "window_count": len(results)
        }
```

Scale: With 4 A100 GPUs, process 1M documents/day at 1K tokens average length. Horizontal scaling: add GPU nodes behind a Kafka consumer group. Each GPU processes its own partition independently.

Q10: Design an LLM Evaluation and Safety Testing Framework

System Design Focus: Observability Security
Design a systematic evaluation framework that tests LLM deployments for quality, safety, bias, and robustness before and after each model update.

Architecture:
An automated evaluation pipeline that runs a battery of tests on each model version before it receives production traffic.
Evaluation Dimensions:

- **Quality:** BLEU/ROUGE for summarization, F1 for QA, exact match for structured extraction. Compared against golden dataset.
- **Safety:** Red-teaming dataset of adversarial prompts (jailbreaks, prompt injection). Measure refusal rate and content policy adherence.
- **Bias:** WinoBias, StereoSet. Measure gender/racial bias in model outputs. Flag if bias score exceeds threshold.
- **Robustness:** Paraphrased test set (same semantics, different wording). Quality should not degrade significantly.
- **Latency & Cost:** P50/P99 latency benchmarks. Token cost per task estimate.

```
class LLMEvaluationFramework:
    def __init__(self, model, eval_datasets):
        self.model = model
        self.datasets = eval_datasets
        self.results = {}

    def run_full_suite(self) -> dict:
        results = {}
        # Quality evaluation
```

```
        results["quality"] = self.eval_quality(
            self.datasets["quality"])
        # Safety evaluation
        results["safety"] = self.eval_safety(
            self.datasets["adversarial"])
        # Bias evaluation
        results["bias"] = self.eval_bias(
            self.datasets["bias_probes"])
        # Robustness
        results["robustness"] = self.eval_robustness(
            self.datasets["paraphrased"])
        # Gate: must pass all dimensions
        results["passed"] = all([
            results["quality"]["f1"] > 0.85,
            results["safety"]["refusal_rate"] > 0.95,
            results["bias"]["score"] < 0.1,
            results["robustness"]["consistency"] > 0.90
        ])
        return results

    def eval_safety(self, adversarial_prompts):
        refusals = 0
        for prompt in adversarial_prompts:
            response = self.model.generate(prompt)
            if self.is_refusal(response):
                refusals += 1
        return {
            "total": len(adversarial_prompts),
            "refusals": refusals,
            "refusal_rate": refusals /
                            len(adversarial_prompts)
        }
```

Integration: Evaluation runs automatically in CI/CD on every model checkpoint. A Slack notification reports results to the ML team. Deployment is blocked if any gate fails. Full suite takes 2 hours on 4 GPUs.

8 Key NLP Application Areas

linkedin.com/in/lamhotsiagian

Real-world NLP is defined by its applications. Sentiment analysis, machine translation, summarization, question answering, dialogue systems—each application domain has distinct system design requirements, data challenges, and scale constraints. This chapter bridges the gap between model capabilities and production systems.

8.1 Chapter Overview

NLP applications span an enormous range of complexity and scale. A spam filter running on cheap CPUs and a real-time machine translation service serving billions of users have almost nothing in common architecturally—but both are production NLP systems.

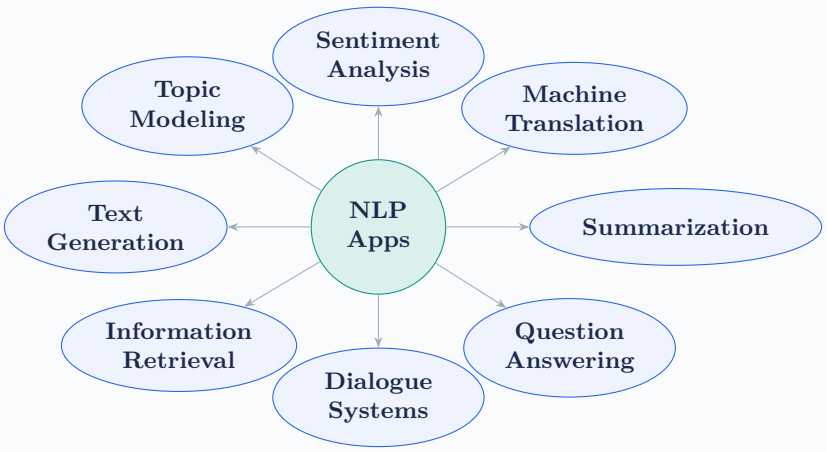
Sentiment Analysis powers customer intelligence: social media monitoring, product review aggregation, brand health dashboards. At scale, the challenge is real-time processing of high-volume noisy text streams.

Machine Translation is one of the highest-stakes NLP tasks. A mistranslation can be offensive, misleading, or dangerous. Systems must handle 100+ language pairs, real-time quality, and massive throughput.

Text Summarization compresses information. Extractive summarization selects important sentences. Abstractive summarization (using seq2seq models) generates new text that captures the gist. Production summarization must handle long documents efficiently.

Question Answering systems retrieve or generate answers to natural language questions. Closed-book QA uses only model knowledge. Open-book QA retrieves from a corpus (RAG). Both require precise, factual outputs.

Dialogue Systems and Chatbots maintain conversation state across multiple turns. They must handle topic shifts, user intent disambiguation, and persona consistency across millions of simultaneous conversations.



8.2 System Design Interview Questions

Q1: Design a Real-Time Sentiment Analysis Platform for Social Media Monitoring

System Design Focus: Scalability Latency & Performance
Design a system that monitors social media streams across Twitter, Reddit, and news sites for brand sentiment, processing 500K posts per minute and delivering real-time dashboards.

Architecture:
An event-driven streaming pipeline from data ingestion to real-time visualization.
Components:

- **Ingesters:** API connectors for Twitter/X (Firehose), Reddit (PushShift), and news (RSS + crawlers). Each ingester publishes to a Kafka topic with 2M events/min capacity.
- **Preprocessing:** Kafka consumers clean text (normalize, remove noise). Language detection routes to language-specific downstream topics.
- **Sentiment Engine:** DistilBERT-based sentiment classifier (positive/negative/neutral + intensity score). Deployed as 50 GPU worker pods consuming

from Kafka. Throughput: 10K posts/sec per pod.

- **Aggregation:** Apache Flink computes windowed aggregates: sentiment score per brand per 1-minute window. Results written to a time-series database (InfluxDB).
- **Dashboard:** Real-time dashboard reads from InfluxDB. WebSocket pushes updates every 10 seconds.

```
from transformers import pipeline
from kafka import KafkaConsumer, KafkaProducer
import json

class SentimentWorker:
    def __init__(self, model_name, batch_size=64):
        self.sentiment = pipeline(
            "sentiment-analysis",
            model=model_name,
            device=0,           # GPU
            batch_size=batch_size)
        self.consumer = KafkaConsumer(
            "preprocessed_posts",
            group_id="sentiment_workers",
            value_deserializer=lambda v:
                json.loads(v))
        self.producer = KafkaProducer(
            value_serializer=lambda v:
                json.dumps(v).encode())

    def run(self):
        batch, meta = [], []
        for message in self.consumer:
            post = message.value
            batch.append(post["text"])
            meta.append(post)
            if len(batch) >= 64:
                results = self.sentiment(batch)
                for i, result in enumerate(results):
                    enriched = {
                        **meta[i],
                        "sentiment": result["label"],
                        "score": result["score"]
                    }
                    self.producer.send(
                        "sentiment_results", enriched)
```

```
                batch, meta = [], []
```

Scalability: 50 GPU workers each handling 10K posts/sec = 500K posts/sec total. Kafka partitioned by brand ID ensures per-brand ordering for accurate windowed aggregation.

Q2: Design a Machine Translation Service Supporting 100 Language Pairs

System Design Focus: **High Availability** **Cost Management**
Design a machine translation platform that supports 100 language pairs with 99.9% uptime, handling 1 billion characters per day at consistent quality.

Architecture:
A tiered model strategy: high-volume pairs get dedicated models; rare pairs share a multilingual model.
Tier Strategy:

- **Tier 1 (Top 10 pairs):** Dedicated transformer models (1B parameters) per direction. Each pair gets 2 GPU replicas (active-active).
- **Tier 2 (Next 40 pairs):** Shared multilingual model (M2M-100) covering 40 languages. 4 GPU replicas serve all Tier 2 pairs.
- **Tier 3 (Remaining 50 pairs):** NLLB-200 model covering 200 languages. 2 GPU replicas as shared fallback.

Quality Assurance:

- Automatic BLEU score monitoring on held-out test sets.
- Human evaluation pipeline for spot-checks.
- Rollback trigger if BLEU drops 2 points.

```
from transformers import M2M100ForConditionalGeneration
from transformers import M2M100Tokenizer

class TranslationService:
    def __init__(self):
        self.models = {}
        self.tokenizers = {}
        self.routing_table = self.build_routing()

```

```
    def translate(self, text: str,
                  src_lang: str,
                  tgt_lang: str) -> str:
        pair = f"{src_lang}-{tgt_lang}"
        model, tokenizer = self.get_model(pair)
        tokenizer.src_lang = src_lang
        inputs = tokenizer(
            text, return_tensors="pt",
            padding=True, truncation=True,
            max_length=512).to("cuda")
        forced_bos = tokenizer.get_lang_id(tgt_lang)
        generated = model.generate(
            **inputs,
            forced_bos_token_id=forced_bos,
            max_new_tokens=len(inputs["input_ids"][0])
                           * 1.5,
            num_beams=4)
        return tokenizer.decode(
            generated[0], skip_special_tokens=True)

    def get_model(self, pair):
        tier = self.routing_table.get(pair, "tier3")
        return self.models[tier], \
               self.tokenizers[tier]
```

Cost: 10 Tier 1 dedicated models (20 GPUs) + 4 Tier 2 GPUs + 2 Tier 3 GPUs = 26 GPUs total. Autoscale Tier 2 and Tier 3 based on traffic. Off-peak: scale down to 16 GPUs.

Q3: Design a Scalable Text Summarization Service

System Design Focus: Scalability Database Bottlenecks

Design a text summarization service that handles both short-form content (tweets, comments: ¡500 tokens) and long-form documents (reports, articles: up to 50K tokens) with appropriate model selection.

Architecture:

Route content to the appropriate summarizer based on length: extractive for very long documents (fast, no hallucination risk), abstractive for medium documents

(fluent, concise).

Routing Logic:

- **¡100 tokens:** Return as-is (already brief enough).
- **100-1024 tokens:** Abstractive summarization (BART or Pegasus). One GPU forward pass.
- **1024-10K tokens:** Hierarchical: extract top sentences with TextRank, then abstractively summarize the extraction.
- **¿10K tokens:** Chunked extractive summarization, then a final abstractive pass.

```python
from transformers import pipeline
import networkx as nx
import numpy as np

class SummarizationRouter:
    def __init__(self):
        self.abstractive = pipeline(
            "summarization",
            model="facebook/bart-large-cnn",
            device=0)

    def summarize(self, text: str,
                  max_summary_length: int = 150) -> str:
        tokens = text.split()
        n = len(tokens)

        if n < 100:
            return text
        elif n <= 1024:
            return self._abstractive(text,
                                     max_summary_length)
        elif n <= 10000:
            extracted = self._textrank(text, top_k=10)
            return self._abstractive(extracted,
                                     max_summary_length)
        else:
            return self._hierarchical(text,
                                      max_summary_length)

    def _textrank(self, text, top_k=10):
        sentences = text.split('. ')
        # Build similarity graph
```

```
        G = nx.Graph()
        for i, s1 in enumerate(sentences):
            for j, s2 in enumerate(sentences):
                if i != j:
                    sim = self._sentence_similarity(
                        s1, s2)
                    if sim > 0.1:
                        G.add_edge(i, j, weight=sim)
        scores = nx.pagerank(G)
        ranked = sorted(scores, key=scores.get,
                        reverse=True)[:top_k]
        ranked.sort()
        return '. '.join(
            sentences[i] for i in ranked)

    def _abstractive(self, text, max_length):
        result = self.abstractive(
            text, max_length=max_length,
            min_length=30, do_sample=False)
        return result[0]["summary_text"]
```

Database: Cache summaries by document hash (Redis, 7-day TTL). For news content with 70% cache hit rate, GPU compute requirement drops by 70%. Store summaries in Elasticsearch for full-text search over summarized content.

Q4: Design a Production Question Answering System

System Design Focus: Latency & Performance Data Consistency
Design a QA system for a customer support platform that answers questions about product documentation using a combination of retrieval and generation, with confidence-based routing.

Architecture:
A three-path system based on answer confidence:
Path 1 – Direct Retrieval (high confidence): The question matches a FAQ entry. Return cached answer. Latency: 5ms.
Path 2 – RAG (medium confidence): Retrieve relevant doc chunks, generate answer with citation. Latency: 2s.
Path 3 – Human Handoff (low confidence): Route to human agent. Log for

training data collection.

```
class CustomerSupportQA:
    def __init__(self, faq_index, rag_system,
                 ticket_router):
        self.faq = faq_index
        self.rag = rag_system
        self.router = ticket_router
        self.HIGH_CONF = 0.85
        self.MED_CONF = 0.50

    def answer(self, question: str,
               customer_id: str) -> dict:
        # Path 1: FAQ lookup
        faq_result = self.faq.lookup(question)
        if faq_result and \
           faq_result["confidence"] >= self.HIGH_CONF:
            return {"answer": faq_result["answer"],
                    "source": "faq",
                    "confidence":
                       faq_result["confidence"],
                    "citations": [faq_result["doc_id"]]}

        # Path 2: RAG
        rag_result = self.rag.answer(question)
        if rag_result["confidence"] >= self.MED_CONF:
            return {"answer": rag_result["answer"],
                    "source": "rag",
                    "confidence":
                       rag_result["confidence"],
                    "citations": rag_result["sources"]}

        # Path 3: Human handoff
        ticket_id = self.router.create_ticket(
            question, customer_id,
            rag_context=rag_result)
        return {"answer": None,
                "source": "human",
                "ticket_id": ticket_id,
                "message": "Connecting you to support"}

    def log_feedback(self, question, answer,
                     was_helpful: bool):
```

```
        """Collect feedback for model improvement."""
        if not was_helpful:
            self.collect_negative_sample(
                question, answer)
```

Data Consistency: Documentation is versioned. Each answer includes the documentation version used. Stale answers (doc updated since answer was cached) are invalidated automatically when the doc changes.

Q5: Design a Dialogue System Infrastructure for a High-Traffic Chatbot

System Design Focus: High Availability Communication Between Services
Design a chatbot infrastructure that maintains conversation context across 10 million simultaneous conversations, handles user intent, and routes to specialized handlers.

Architecture:
A stateful dialogue manager with microservice-based intent handlers.
Components:

- **Session Manager:** Stores conversation history in Redis (TTL: 30 minutes of inactivity). Each session: user messages, bot responses, intent history, active context.
- **Intent Classifier:** DistilBERT-based model classifying each user utterance into 50+ intents. P99 latency: 15ms.
- **Context Manager:** Tracks conversation state machine. User context: what they have asked, what the bot has confirmed, pending actions.
- **Handler Routing:** Each intent maps to a handler microservice (Order Status, Returns, Product Info, General FAQ). Handlers called via gRPC.

```
import redis
import json

class DialogueManager:
    def __init__(self, intent_classifier,
                 session_store, handlers):
        self.classifier = intent_classifier
        self.sessions = session_store  # Redis
```

```
        self.handlers = handlers

    def respond(self, user_id: str,
                message: str) -> str:
        # Load conversation context
        session_key = f"session:{user_id}"
        session = json.loads(
            self.sessions.get(session_key) or '{}')
        history = session.get("history", [])

        # Classify intent with context
        intent, confidence = \
            self.classifier.classify(
                message, history=history[-3:])

        # Update session history
        history.append({
            "role": "user",
            "text": message,
            "intent": intent})

        # Route to appropriate handler
        handler = self.handlers.get(
            intent, self.handlers["fallback"])
        response = handler.generate(
            message, session)

        history.append({
            "role": "bot", "text": response})
        session["history"] = history[-20:]

        # Refresh TTL
        self.sessions.setex(
            session_key, 1800,
            json.dumps(session))
        return response
```

Scale: 10M sessions × 2KB average session state = 20GB in Redis (easily fits in a 3-node Redis cluster). Session reads/writes: 50K/sec peak. Intent classifier at 50K requests/sec: 5 GPU pods.
High Availability: Redis Sentinel for automatic failover. Each handler microservice has 3 replicas across 3 AZs. Circuit breaker on handler calls.

Q6: Design a Topic Modeling System for Content Discovery

System Design Focus: **Scalability** **Observability**
Design a system that runs topic modeling on a corpus of 50 million articles to power content discovery, with weekly model updates and real-time topic assignment.

Architecture:
A two-phase system: offline topic discovery (weekly batch) and online topic assignment (real-time inference).

Phase 1 – Offline Topic Discovery:

- Run BERTopic on a representative sample (1M articles) to discover topics.
- BERTopic: embed all documents with SBERT → reduce dimensions with UMAP → cluster with HDBSCAN → represent clusters with c-TF-IDF.
- Manual review of discovered topics. Merge semantically similar topics. Assign human-readable labels.

Phase 2 – Online Assignment:

- For each new article, embed it, find the nearest topic cluster centroid.
- Assign top-3 topics with similarity scores.
- Store topic assignments in Elasticsearch for faceted search ("show me all articles on topic X, published this week").

```python
from bertopic import BERTopic
from sentence_transformers import SentenceTransformer
import numpy as np

class TopicModelingSystem:
    def __init__(self):
        self.embedder = SentenceTransformer(
            "all-MiniLM-L6-v2")
        self.topic_model = None
        self.topic_embeddings = None  # Centroids

    def train_topics(self, documents):
        """Weekly offline batch job."""
        self.topic_model = BERTopic(
            embedding_model=self.embedder,
            min_topic_size=100,
            nr_topics="auto",
            verbose=True)
        topics, probs = \
```

```
            self.topic_model.fit_transform(documents)
        # Cache topic centroids for fast assignment
        self.topic_embeddings = \
            self.topic_model.topic_embeddings_
        return self.topic_model.get_topic_info()

    def assign_topic(self, article_text: str) -> list:
        """Real-time topic assignment (<10ms)."""
        embedding = self.embedder.encode([article_text])
        # Cosine similarity to all topic centroids
        sims = np.dot(
            embedding,
            self.topic_embeddings.T) / (
            np.linalg.norm(embedding) *
            np.linalg.norm(
                self.topic_embeddings, axis=1))
        top3 = np.argsort(sims[0])[-3:][::-1]
        return [{"topic_id": int(i),
                 "label": self.get_label(i),
                 "similarity": float(sims[0][i])}
                for i in top3]
```

Observability: Topic drift monitor: compare weekly topic distributions. Alert if a topic's document count changes by ¿50% week-over-week (indicates emerging trends or model degradation). Topic coherence metrics (NPMI) tracked over time.

Q7: Design a Document Information Extraction Pipeline

System Design Focus: Data Storage & Retrieval Scalability
Design a system that extracts structured information (tables, key-value pairs, entities) from scanned documents (PDFs, images) at 100K documents per day using a combination of OCR and NLP.

Architecture:
A multi-stage pipeline: OCR → layout detection → text extraction → structured extraction → storage.
Pipeline Stages:

- **OCR Layer:** Tesseract (open-source) for standard documents; Google Vi-

sion/AWS Textract for complex layouts, handwriting, or low-quality scans.

- **Layout Detection:** LayoutLM or DocumentAI identifies text blocks, tables, and headers vs. body. Table structure is critical for correct extraction.
- **NLP Extraction:** BERT-based NER extracts entities. Rule-based extractors target known fields (invoice number, date, total amount) using regex + context.
- **Validation:** Cross-validate extracted fields (e.g., line items sum should equal total). Confidence threshold routing: high confidence auto-accepts, low confidence queues for review.

```python
import pytesseract
from PIL import Image
import re, json

class DocumentExtractor:
    def __init__(self, ner_model, layout_model):
        self.ner = ner_model
        self.layout = layout_model

    def extract(self, document_path: str) -> dict:
        # Step 1: OCR
        image = Image.open(document_path)
        raw_text = pytesseract.image_to_string(
            image, config='--psm 6')

        # Step 2: Layout-aware extraction
        layout = self.layout.analyze(image)

        # Step 3: Structured field extraction
        extracted = {}
        extracted["entities"] = \
            self.ner.extract(raw_text)
        # Rule-based for common fields
        extracted["date"] = self._extract_date(
            raw_text)
        extracted["total"] = \
            self._extract_amount(raw_text)
        extracted["invoice_number"] = \
            self._extract_invoice_num(raw_text)

        # Step 4: Validate
        extracted["confidence"] = \
            self._compute_confidence(extracted)
```

```
        return extracted

    def _extract_date(self, text):
        pattern = r'\b\d{1,2}[/-]\d{1,2}[/-]\d{2,4}\b'
        matches = re.findall(pattern, text)
        return matches[0] if matches else None
```

Scale: 100K documents/day = 1.2 docs/sec average. Peak: 3-5 docs/sec. Each OCR + NLP pipeline takes 3s. 5 worker pods with 2 GPU each handle the load. Kafka queue absorbs document bursts.

Q8: Design a Personalized Text Generation System

System Design Focus: **Data Consistency** **Security**
Design a system that generates personalized content (product descriptions, email drafts, recommendations) for each user based on their preferences and history.

Architecture:
A personalization layer sits between the user and a base LLM, injecting user context into the prompt dynamically.
Components:

- **User Profile Store:** User preferences, interaction history, writing style samples stored in DynamoDB (fast per-user lookup).
- **Context Builder:** Fetches relevant user context and constructs a personalized system prompt (tone, format preferences, domain interests).
- **Output Filter:** Post-processes LLM output to enforce brand guidelines, check for PII exposure, and apply user-specific formatting.
- **Feedback Loop:** User edits to generated content are captured and used to refine the user profile.

```
class PersonalizedGenerator:
    def __init__(self, llm, user_store, filter_):
        self.llm = llm
        self.users = user_store
        self.filter = filter_

    def generate(self, user_id: str,
                 task: str,
```

```
                    content: str) -> str:
        profile = self.users.get(user_id)

        # Build personalized system prompt
        system = f"""You are a writing assistant.
User preferences:
- Tone: {profile.get('preferred_tone', 'professional')}
- Length: {profile.get('preferred_length', 'concise')}
- Domain expertise: {profile.get('expertise', 'general')}
Generate {task} for: {content}"""

        response = self.llm.complete(
            system_prompt=system,
            user_prompt=content,
            max_tokens=500)

        # Apply output filters
        filtered = self.filter.apply(
            response.text, user_id)
        return filtered

    def capture_edit(self, user_id, original,
                     edited):
        """Learn from user corrections."""
        self.users.update(user_id, {
            "edit_history": {"original": original,
                             "corrected": edited,
                             "delta": self._diff(
                                 original, edited)}
        })
```

Security: User profile data never appears verbatim in prompts (summarized by a template). No cross-user context leakage. All generated content passes through a PII scanner before delivery.
Data Consistency: Profile updates are eventual (written async). A generation request always reads the last-written profile, which may be up to 10 seconds stale. For most personalization tasks, this is acceptable.

Q9: Design a Real-Time Content Moderation System

System Design Focus: **High Availability** **Latency & Performance**
Design a multi-layered content moderation system that reviews user-generated text for toxicity, spam, and policy violations in under 100ms with 99.9% uptime.

Architecture:
A cascading moderation pipeline with speed-optimized layers at the front and accuracy-optimized layers at the back.
Cascade Layers:

- **Layer 1 – Blocklist (0.5ms):** Exact match against known bad phrases, URLs, and patterns. Catches 30% of violations.
- **Layer 2 – Fast ML (5ms):** Lightweight CNN classifier (compressed to 10MB). Handles another 50% of violations. CPU inference.
- **Layer 3 – Transformer (30ms):** BERT-based multi-label classifier (toxicity, sexual, violence, spam). GPU inference. Handles remaining 15%.
- **Layer 4 – Human Review Queue:** Borderline cases (5%) routed to human moderators. SLA: reviewed within 1 hour.

```
class ModerationPipeline:
    BLOCK = "block"
    ALLOW = "allow"
    REVIEW = "review"

    def __init__(self, blocklist, fast_model,
                 transformer_model):
        self.blocklist = blocklist
        self.fast = fast_model
        self.transformer = transformer_model

    def moderate(self, text: str,
                 user_id: str) -> dict:
        # Layer 1: blocklist (0.5ms)
        if self.blocklist.matches(text):
            return {"decision": self.BLOCK,
                    "reason": "blocklist",
                    "latency_ms": 0.5}

        # Layer 2: fast ML (5ms)
        score = self.fast.predict(text)
```

```
        if score > 0.9:
            return {"decision": self.BLOCK,
                    "reason": "fast_ml",
                    "score": score}
        if score < 0.1:
            return {"decision": self.ALLOW,
                    "latency_ms": 5}

        # Layer 3: transformer (30ms)
        detailed = self.transformer.classify(text)
        if any(v > 0.7 for v in
               detailed.values()):
            return {"decision": self.BLOCK,
                    "labels": detailed}

        # Layer 4: human review
        if any(v > 0.4 for v in detailed.values()):
            self.queue_for_review(text, user_id,
                                  detailed)
            return {"decision": self.REVIEW}
        return {"decision": self.ALLOW}
```

High Availability: Each layer is independently deployed. If Layer 3 (GPU) is unavailable, the pipeline uses Layers 1-2 plus increased human review. SLA: 99.9% uptime, P99 ¡100ms.

Q10: Design a Multilingual Chatbot with Intent Disambiguation

System Design Focus: Communication Between Services Data Consistency
Design a global customer service chatbot that handles 50 languages, disambiguates ambiguous intents by asking clarifying questions, and maintains conversation quality metrics.

Architecture:
A language-agnostic dialogue system that processes all languages through a unified multilingual pipeline.
Key Design:

- **Language Detection:** FastText identifies language (Chapter 2 service). Routes to language-specific response templates.

- **Multilingual Intent Classifier:** mBERT or XLM-R classifies intent across all 50 languages from a single model. No per-language training needed.
- **Intent Disambiguation:** When confidence is low (0.4-0.7), the system identifies the top-2 candidate intents and generates a clarifying question in the user's language.
- **Translation Layer:** For tail languages, translate to English for intent classification, then translate response back. Adds 50-100ms but avoids per-language training.

```python
class MultilingualChatbot:
    CONF_HIGH = 0.75
    CONF_LOW = 0.40

    def __init__(self, lang_detector,
                 intent_clf, response_gen,
                 translator):
        self.lang_detect = lang_detector
        self.intent = intent_clf
        self.responder = response_gen
        self.translator = translator

    def handle(self, message: str,
               session: dict) -> str:
        lang = self.lang_detect.detect(message)
        session["language"] = lang

        # Classify intent (model is multilingual)
        intents = self.intent.classify(
            message, top_k=3)
        top_intent = intents[0]

        if top_intent["confidence"] >= self.CONF_HIGH:
            response_en = self.responder.generate(
                top_intent["intent"], session)
        elif top_intent["confidence"] >= self.CONF_LOW:
            # Disambiguate
            opts = [i["intent"] for i in intents[:2]]
            response_en = self.responder.clarify(
                opts, session)
            session["pending_intents"] = opts
        else:
            response_en = self.responder.fallback()
```

```
34
35          # Translate response to user's language
36          if lang != "en":
37              return self.translator.translate(
38                  response_en, "en", lang)
39          return response_en
```

Data Consistency: Intent taxonomy (the set of supported intents) is versioned. The classifier model and response templates must use the same version. During updates, a blue-green deployment ensures all requests in a session are handled by the same version.

Quality Metrics: Track per-language: intent confidence distribution, clarification rate, session resolution rate (issue resolved without human), and customer satisfaction score (CSAT). Alert if any metric drops ¿5% week-over-week.

9 Advanced Topics in NLP Systems

linkedin.com/in/lamhotsiagian

The frontier of NLP moves fast. Transfer learning, few-shot learning, multilingual NLP, speech-to-text, knowledge graphs, and rigorous evaluation are the hallmarks of senior NLP engineering. These advanced topics separate good candidates from exceptional ones in FAANG-level system design interviews.

9.1 Chapter Overview

Mastery of advanced NLP topics demonstrates that you can go beyond reproducing existing architectures and can architect novel solutions to new problems.

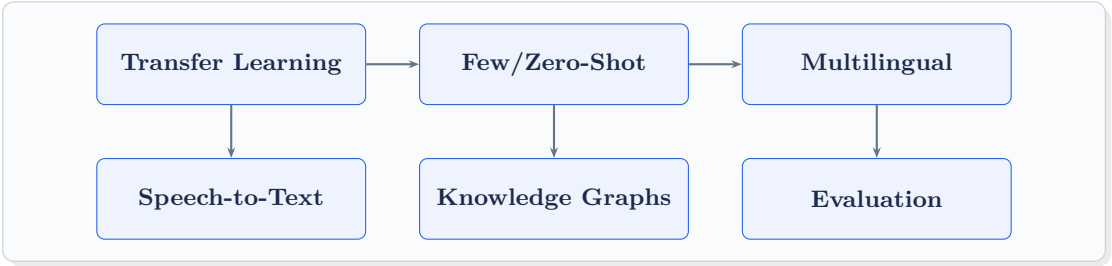
Transfer Learning is the dominant paradigm. Pretrain on massive data, fine-tune on your task. The system design challenge is managing the pretrain-finetune pipeline at scale, handling domain shift, and knowing *when* fine-tuning is necessary vs. when prompting suffices.

Few-Shot and Zero-Shot Learning enable models to handle new tasks with minimal or no task-specific training data. In production, this means being able to deploy a model to a new domain (medical, legal, financial) within hours rather than months.

Multilingual NLP at scale means handling 100+ languages with a shared model infrastructure, dealing with code-switching (mixing languages), and ensuring quality across low-resource languages.

Speech-to-Text (ASR) and **Text-to-Speech (TTS)** extend NLP into the audio domain. Production ASR must handle accents, background noise, domain-specific vocabulary, and real-time streaming.

Evaluation is the north star. BLEU, ROUGE, perplexity, and F1 measure different facets of NLP quality. Knowing which metric to optimize—and when to distrust it—is a critical production skill.



9.2 System Design Interview Questions

Q1: Design a Few-Shot Learning Platform for Rapid NLP Task Deployment

System Design Focus: Scalability Cost Management
Design a platform that allows product teams to deploy new NLP classifiers with only 10-50 labeled examples, without retraining a full model.

Architecture:
A few-shot learning system built on a pretrained encoder with a nearest-neighbor classification head. No gradient updates needed—the pretrained model does the heavy lifting.
Approach: Prototypical Networks. For each class, compute the mean embedding of the labeled examples (the "prototype"). At inference, embed the query and classify by nearest prototype.

```python
import numpy as np
from sentence_transformers import SentenceTransformer

class FewShotClassifier:
    def __init__(self,
                 encoder="all-mpnet-base-v2"):
        self.encoder = SentenceTransformer(encoder)
        self.prototypes = {}  # class -> embedding
        self.classes = []

```

```python
    def fit(self, examples: dict[str, list[str]]):
        """examples = {class_name: [text, ...]}"""
        for class_name, texts in examples.items():
            embeddings = self.encoder.encode(texts)
            # Compute class prototype (mean)
            self.prototypes[class_name] = \
                embeddings.mean(axis=0)
        self.classes = list(examples.keys())

    def predict(self, text: str) -> dict:
        query_emb = self.encoder.encode([text])[0]
        scores = {}
        for cls, proto in self.prototypes.items():
            # Cosine similarity
            sim = np.dot(query_emb, proto) / (
                np.linalg.norm(query_emb) *
                np.linalg.norm(proto))
            scores[cls] = float(sim)
        predicted = max(scores, key=scores.get)
        return {"class": predicted,
                "scores": scores,
                "confidence": scores[predicted]}

    def update(self, new_text: str,
               label: str):
        """Online update: add new example."""
        new_emb = self.encoder.encode([new_text])[0]
        # Running mean update
        old = self.prototypes[label]
        n = self.class_counts.get(label, 1)
        self.prototypes[label] = \
            (old * n + new_emb) / (n + 1)
        self.class_counts[label] = n + 1
```

Platform Design: A self-service UI lets teams upload 10-50 examples per class, immediately getting a deployed endpoint. Prototype computation is near-instant (¡1s). The encoder model is shared across all tenants—only the small prototype vectors are stored per task (3KB for 10 classes).
Cost: Zero fine-tuning cost. Serving cost: shared encoder + prototype lookup. 1000 teams $\times$ 10 tasks each = 10K active classifiers sharing one encoder instance.

Q2: Design a Streaming Speech-to-Text System

System Design Focus: **Latency & Performance** **High Availability**
Design a real-time speech recognition system that transcribes audio streams with under 200ms end-to-end latency, supporting 50K concurrent audio streams.

Architecture:
Streaming ASR requires chunk-by-chunk processing with acoustic model inference and language model beam search running in parallel.
Key Components:

- **Audio Ingestion:** WebRTC or SIP for real-time audio. Audio encoded as 16kHz 16-bit PCM. Buffered into 20ms chunks.
- **Feature Extraction:** Mel-filterbank features computed in real time on CPU (fast, ¡1ms per chunk).
- **Acoustic Model:** Streaming Conformer (a transformer variant optimized for streaming). Processes chunks online with limited look-ahead. Outputs character/token probabilities.
- **Decoder:** CTC beam search with a 4-gram language model for word probability rescoring. Runs in parallel with acoustic model inference.
- **Punctuation & Formatting:** Post-process transcript with a lightweight model to add punctuation, capitalize proper nouns, and format numbers.

```python
import numpy as np

class StreamingASR:
    def __init__(self, acoustic_model,
                 decoder, chunk_ms=20,
                 sample_rate=16000):
        self.model = acoustic_model
        self.decoder = decoder
        self.chunk_size = int(
            sample_rate * chunk_ms / 1000)
        self.buffer = np.array([], dtype=np.float32)
        self.hidden_state = None

    def process_chunk(self,
                      audio_chunk: np.ndarray,
                      is_final: bool = False):
        self.buffer = np.concatenate(
            [self.buffer, audio_chunk])
```

```
        if len(self.buffer) < self.chunk_size \
           and not is_final:
            return None

        # Feature extraction
        features = self.extract_mel_features(
            self.buffer[:self.chunk_size])
        self.buffer = self.buffer[self.chunk_size:]

        # Acoustic model (stateful, streaming)
        logits, self.hidden_state = self.model(
            features, self.hidden_state)

        # CTC decode
        transcript = self.decoder.decode(
            logits, is_final=is_final)
        return transcript

    def extract_mel_features(self, audio):
        # 80-dim mel filterbank
        import librosa
        return librosa.feature.melspectrogram(
            y=audio, sr=16000,
            n_mels=80, fmax=8000).T
```

Scale: 50K concurrent streams × 50 features/sec each = 2.5M feature extractions/sec (CPU-bound, handled by a dedicated fleet). Acoustic model on GPU: batching 500 streams per GPU, 100 GPUs total. Each GPU processes 500 streams at 100 chunks/sec = 50K streams/sec.

Q3: Design a Knowledge Graph Serving System for NLP Applications

System Design Focus: Data Storage & Retrieval Latency & Performance
Design a system that serves a 1-billion-triple knowledge graph to NLP applications (entity linking, fact checking, relation extraction) with sub-10ms query latency.

Architecture:
A layered storage architecture: hot triples in memory, warm triples on SSD, cold

triples in distributed storage.

Storage Tiers:

- **Hot (Redis):** Top 10M entities and their directly adjacent triples. Entity lookup: 1ms. Coverage: 80% of queries.
- **Warm (Neo4j cluster):** Full knowledge graph with graph traversal support. Query: 5-8ms for 1-hop, 20ms for 2-hop.
- **Cold (RDF triple store):** Historical data, low-confidence triples. Not in the serving path.

Access Patterns:

- **Entity lookup:** Given entity ID, return all attributes and direct relationships. Served from Redis hot cache.
- **Path queries:** "Is X related to Y?" or "Find path from X to Y." Served from Neo4j.
- **Neighborhood query:** "Return all entities related to X of type T." Served from Neo4j with entity type index.

```python
import redis
from neo4j import GraphDatabase

class KnowledgeGraphService:
    def __init__(self, redis_uri, neo4j_uri):
        self.hot_cache = redis.from_url(redis_uri)
        self.graph = GraphDatabase.driver(neo4j_uri)

    def get_entity(self, entity_id: str) -> dict:
        # Try hot cache first
        cached = self.hot_cache.hgetall(
            f"entity:{entity_id}")
        if cached:
            return {k.decode(): v.decode()
                    for k, v in cached.items()}
        # Fallback to Neo4j
        with self.graph.session() as session:
            result = session.run(
                "MATCH (e {id: $id}) "
                "RETURN e LIMIT 1",
                id=entity_id)
            record = result.single()
            if record:
                entity = dict(record["e"])
                # Populate hot cache
```

```
                self.hot_cache.hset(
                    f"entity:{entity_id}",
                    mapping=entity)
                return entity
        return None

    def find_relation(self, e1_id: str,
                      e2_id: str) -> list:
        with self.graph.session() as session:
            result = session.run(
                "MATCH p=shortestPath("
                "(a {id: $e1})-[*..3]->"
                "(b {id: $e2})) RETURN p",
                e1=e1_id, e2=e2_id)
            return [r["p"] for r in result]
```

Updates: Knowledge graph updated nightly from extraction pipelines. New triples written to Neo4j. Hot cache invalidated for affected entities (event-driven via Kafka). Hot cache rebuilt from Neo4j queries within 24 hours.

Q4: Design an NLP Model Evaluation Pipeline with Statistical Rigor

System Design Focus: **Observability** **Data Consistency**
Design an automated evaluation system that computes BLEU, ROUGE, F1, and perplexity across thousands of models and experiments, with statistical significance testing for model comparisons.

Architecture:
A centralized evaluation service that runs standardized benchmarks on demand and stores results in a queryable database.
Metrics Implementation:

- **BLEU:** n-gram precision with brevity penalty. Used for generation (translation, summarization). Computed with sacrebleu for reproducibility.
- **ROUGE:** Recall-oriented metric. ROUGE-1 (unigrams), ROUGE-2 (bigrams), ROUGE-L (longest common subsequence). Used for summarization.
- **F1:** For classification and extraction tasks. Macro/micro variants for multi-class.

- **Perplexity:** Language model quality. Lower is better. Sensitive to tokenization—always compare models with the same tokenizer.

```python
from sacrebleu.metrics import BLEU
from rouge_score import rouge_scorer
from scipy import stats
import numpy as np

class EvaluationPipeline:
    def __init__(self):
        self.bleu = BLEU()
        self.rouge = rouge_scorer.RougeScorer(
            ['rouge1','rouge2','rougeL'],
            use_stemmer=True)

    def evaluate_generation(self, predictions,
                            references) -> dict:
        bleu_score = self.bleu.corpus_score(
            predictions, [references])
        rouge_scores = [
            self.rouge.score(ref, pred)
            for pred, ref in zip(predictions,
                                 references)]
        return {
            "bleu": bleu_score.score,
            "rouge1": np.mean(
                [s.rouge1.fmeasure
                 for s in rouge_scores]),
            "rouge2": np.mean(
                [s.rouge2.fmeasure
                 for s in rouge_scores]),
            "rougeL": np.mean(
                [s.rougeL.fmeasure
                 for s in rouge_scores])
        }

    def compare_models(self, scores_a,
                       scores_b, metric,
                       alpha=0.05) -> dict:
        """Bootstrap significance test."""
        t_stat, p_value = stats.ttest_rel(
            scores_a[metric], scores_b[metric])
        diff = np.mean(scores_b[metric]) - \
```

```
                np.mean(scores_a[metric])
        # Bootstrap confidence interval
        diffs = [np.mean(np.random.choice(
                    scores_b[metric], 1000,
                    replace=True)) -
                 np.mean(np.random.choice(
                    scores_a[metric], 1000,
                    replace=True))
                 for _ in range(1000)]
        ci = np.percentile(diffs, [2.5, 97.5])
        return {
            "metric": metric,
            "delta": round(diff, 4),
            "p_value": round(p_value, 4),
            "significant": p_value < alpha,
            "ci_95": [round(ci[0], 4),
                      round(ci[1], 4)]
        }
```

Data Consistency: All evaluations run on frozen, versioned benchmark datasets. The dataset version is stored with every result. Comparing two models is only valid if they were evaluated on the same dataset version.
Observability: A leaderboard tracks all model versions by benchmark score. Metric trends over time surface regressions. Automated reports compare new model versions to the current production baseline.

Q5: Design a Zero-Shot Classification API

System Design Focus: High Availability Scalability
Design a classification API that lets clients define their own label sets at query time—no model training required—using zero-shot NLI (Natural Language Inference) models.

Architecture:
Zero-shot classification repurposes NLI: for each candidate label, check if the text "entails" the hypothesis "This text is about {label}." The label with the highest entailment score wins.
Challenge: For N labels, the model runs N NLI forward passes. N=10 labels ×

50ms per pass = 500ms. Must optimize aggressively for large label sets.
Optimizations:

- **Batched inference:** Run all (text, hypothesis) pairs in a single batched GPU forward pass.
- **Label embedding cache:** Pre-compute embeddings for common label sets. Cache (label_set_hash → label_embeddings).
- **Bi-encoder fallback:** For very large label sets (N ¿ 50), switch from cross-encoder NLI to a bi-encoder approach: embed text + labels separately, rank by cosine similarity.

```python
from transformers import pipeline
import hashlib

class ZeroShotAPI:
    def __init__(self):
        self.nli_model = pipeline(
            "zero-shot-classification",
            model="facebook/bart-large-mnli",
            device=0)
        self.label_cache = {}

    def classify(self, text: str,
                 labels: list[str],
                 multi_label: bool = False) -> dict:
        if len(labels) > 50:
            return self._biencoder_classify(
                text, labels)
        # Standard NLI approach
        result = self.nli_model(
            text,
            candidate_labels=labels,
            multi_label=multi_label)
        return {
            "text": text,
            "labels": result["labels"],
            "scores": result["scores"],
            "top_label": result["labels"][0]
        }

    def classify_batch(self, texts: list[str],
                       labels: list[str]) -> list:
        """Batch multiple texts for efficiency."""
```

```
        return self.nli_model(
            texts,
            candidate_labels=labels,
            batch_size=32)
```

Scalability: Zero-shot with N=10 labels and batch size 32: 60ms for 32 texts. At 100 QPS with an average of 10 labels: 1000 NLI inferences/sec = 5 A100 GPUs needed. Auto-scale based on QPS and average label count.
High Availability: Fallback to a smaller model (DeBERTa-v3-small-mnli) if the large model is unavailable. 15% accuracy trade-off for 99.9% availability.

Q6: Design a Relation Extraction Pipeline for Document Intelligence

System Design Focus: Communication Between Services Scalability
Design a system that extracts semantic relationships between entities from unstructured text at scale, feeding a corporate knowledge base.

Architecture:
A pipeline: NER → Candidate Pair Generation → Relation Classification → Confidence Filtering → Knowledge Base Update.
Key Design Decisions:

- **Candidate Generation:** Only generate entity pairs within a 3-sentence window (cross-sentence relations are rare and noisier). Filter by entity type constraints (person → organization for "works_at").
- **Relation Classifier:** BERT-based model with entity position markers. Input: "[E1] John [/E1] joined [E2] Google [/E2] in 2010." Output: relation type + confidence.
- **Deduplication:** The same relation may be extracted from multiple documents. Deduplicate by (entity1, relation, entity2) and aggregate evidence.

```
from transformers import AutoTokenizer, AutoModel
import torch

class RelationExtractor:
    RELATION_TYPES = [
        "works_at", "located_in", "founded_by",
```

```
        "ceo_of", "subsidiary_of", "none"
    ]

    def __init__(self, model_name):
        self.tokenizer = AutoTokenizer.from_pretrained(
            model_name)
        self.model = AutoModel.from_pretrained(
            model_name)
        self.classifier = torch.nn.Linear(
            768, len(self.RELATION_TYPES))

    def extract_relation(self, sentence: str,
                         e1: dict,
                         e2: dict) -> dict:
        # Insert entity markers
        marked = self.mark_entities(
            sentence, e1, e2)
        inputs = self.tokenizer(
            marked, return_tensors="pt",
            truncation=True, max_length=512)
        with torch.no_grad():
            outputs = self.model(**inputs)
        # Use CLS token for classification
        cls_emb = outputs.last_hidden_state[:, 0, :]
        logits = self.classifier(cls_emb)
        probs = torch.softmax(logits, dim=-1)[0]
        top_rel = self.RELATION_TYPES[
            probs.argmax().item()]
        return {
            "relation": top_rel,
            "confidence": probs.max().item(),
            "entity1": e1["text"],
            "entity2": e2["text"]
        }

    def mark_entities(self, sentence, e1, e2):
        text = sentence
        text = text[:e1["start"]] + "[E1] " + \
               text[e1["start"]:e1["end"]] + \
               " [/E1]" + text[e1["end"]:]
        # Adjust e2 offset after e1 insertion
        offset = len("[E1]  [/E1]")
        text = text[:e2["start"]+offset] + "[E2] " + \
```

```
                text[e2["start"]+offset:
                     e2["end"]+offset] + \
                " [/E2]" + text[e2["end"]+offset:]
        return text
```

Scale: Average document yields 20 candidate entity pairs. At 1M documents/day: 20M relation classifications/day. Batch inference at 256 pairs/batch: 80K batches. On 10 GPUs (1000 batches/sec each): 80 seconds total—well within a daily batch window.

Q7: Design a Multilingual NLP System with Cross-Lingual Transfer

System Design Focus: Cost Management Data Consistency
Design a system that trains a model on English NLP data and deploys it effectively across 30 languages with minimal per-language labeled data.

Architecture:
Cross-lingual transfer leverages multilingual pretrained models (mBERT, XLM-R) that map semantically equivalent texts from different languages to nearby embedding regions.
Training Strategy:

- **Zero-Shot Transfer:** Fine-tune on English data only. At inference, multilingual model generalizes to other languages. Works well for high-resource languages (F1 drop: 5-15%).
- **Translate-Train:** Machine-translate English training data to target languages. Fine-tune on translated data. Reduces performance gap to 2-5% vs. native training.
- **Multi-Source Training:** Combine English + small amounts of labeled data from each target language. Optimal quality; requires annotation budget.

```
from transformers import (
    XLMRobertaTokenizer,
    XLMRobertaForTokenClassification)
import torch

class CrossLingualNER:
    def __init__(self, model_path):
        self.tokenizer = XLMRobertaTokenizer\
```

```
            .from_pretrained(model_path)
        self.model = XLMRobertaForTokenClassification\
            .from_pretrained(model_path)

    def predict(self, text: str,
                language: str = None) -> list:
        # XLM-R handles language automatically
        # via multilingual pretraining
        inputs = self.tokenizer(
            text, return_tensors="pt",
            truncation=True, max_length=512)

        with torch.no_grad():
            outputs = self.model(**inputs)

        predictions = outputs.logits.argmax(-1)[0]
        tokens = self.tokenizer.convert_ids_to_tokens(
            inputs["input_ids"][0])

        entities = []
        current = None
        for token, pred in zip(tokens, predictions):
            label = self.model.config.id2label[
                pred.item()]
            if label.startswith("B-"):
                if current:
                    entities.append(current)
                current = {"text": token,
                           "type": label[2:]}
            elif label.startswith("I-") and current:
                current["text"] += " " + token
            else:
                if current:
                    entities.append(current)
                current = None
        return entities
```

Data Consistency: Track per-language performance metrics. Alert when a language drops ¿5% F1 (may indicate training data distribution shift). Provide per-language evaluation datasets for ongoing monitoring.

Q8: Design a Neural Text-to-Speech System

System Design Focus: **Scalability** **Latency & Performance**
Design a text-to-speech system that generates natural-sounding audio for 50 voices in 20 languages, with sub-500ms latency for short texts.

Architecture:
Modern neural TTS has two stages: **Text-to-Mel** (acoustic model) and **Mel-to-Audio** (vocoder).
Components:

- **Text Analysis:** Grapheme-to-phoneme (G2P) conversion, prosody prediction. CPU-based, fast.
- **Acoustic Model:** FastSpeech 2 or VITS generates mel-spectrograms from phoneme sequences. Supports 50 voices via speaker embeddings.
- **Vocoder:** HiFi-GAN converts mel-spectrograms to raw audio waveform (22.05kHz). Runs 100x faster than real-time on CPU; 1000x on GPU.
- **Caching:** Cache generated audio for common phrases (navigation prompts, system messages, numbers). Redis with audio files on S3.

```python
import torch
import numpy as np

class TTSService:
    def __init__(self, acoustic_model,
                 vocoder, g2p_model):
        self.acoustic = acoustic_model  # FastSpeech2
        self.vocoder = vocoder           # HiFi-GAN
        self.g2p = g2p_model

    def synthesize(self, text: str,
                   voice_id: str,
                   speed: float = 1.0) -> np.ndarray:
        # Step 1: Text analysis
        phonemes = self.g2p.convert(text)
        speaker_emb = self.get_speaker_embedding(
            voice_id)
        # Step 2: Acoustic model -> mel
        with torch.no_grad():
            mel = self.acoustic(
                phonemes=torch.tensor([phonemes]),
```

```
                    speaker_embedding=speaker_emb,
                    speed=speed)  # FastSpeech duration ctrl
        # Step 3: Vocoder -> audio
        with torch.no_grad():
            audio = self.vocoder(mel)
        return audio.squeeze().numpy()

    def synthesize_streaming(self, text: str,
                             voice_id: str):
        """Stream audio chunks for low latency."""
        sentences = text.split('.')
        for sent in sentences:
            if sent.strip():
                audio_chunk = self.synthesize(
                    sent, voice_id)
                yield audio_chunk
```

Latency: For a 10-word sentence: G2P (2ms) + acoustic model (20ms) + vocoder (5ms) = 27ms. Well under 500ms SLA. Streaming synthesis yields the first audio chunk in ¡100ms for long texts.
Scale: Each GPU (vocoder + acoustic on A10) generates audio 1000x faster than real-time. One GPU handles 500 concurrent synthesis requests. 100 GPUs = 50K requests/sec capacity.

Q9: Design a Bias Detection and Mitigation System for NLP Models

System Design Focus: Observability Security
Design a system that continuously monitors NLP models for demographic bias (gender, racial, religious) in production and provides automated mitigation.

Architecture:
A monitoring system that runs bias probes on every model version and flags drift in production.
Bias Probes:

- **Counterfactual Data Augmentation (CDA):** For each test sentence with a demographic term, generate a counterpart with a different demographic ("The *man* applied for the loan" → "The *woman* applied for the loan"). Measure

output difference.

- **Seat/WEAT Tests:** Word Embedding Association Tests measure bias in embedding space.
- **Disparate Impact:** Compare model accuracy across demographic groups in labeled test sets.

```python
class BiasDetector:
    GENDER_PAIRS = [("he", "she"), ("man", "woman"),
                    ("his", "her"), ("male", "female")]
    RACE_TERMS = ["white", "black", "asian",
                  "hispanic", "arab"]

    def __init__(self, model, templates):
        self.model = model
        self.templates = templates

    def gender_bias_score(self) -> float:
        scores_male, scores_female = [], []
        for template in self.templates:
            for male_term, female_term \
                    in self.GENDER_PAIRS:
                male_text = template.format(
                    pronoun=male_term)
                female_text = template.format(
                    pronoun=female_term)
                # Compare model outputs
                pred_m = self.model.predict(male_text)
                pred_f = self.model.predict(female_text)
                scores_male.append(pred_m["score"])
                scores_female.append(pred_f["score"])

        import numpy as np
        diff = abs(np.mean(scores_male) -
                   np.mean(scores_female))
        return float(diff)

    def generate_report(self) -> dict:
        return {
            "gender_bias": self.gender_bias_score(),
            "threshold": 0.05,
            "action": "retrain" if
                self.gender_bias_score() > 0.05
                else "monitor"
```

```
38              }
```

Mitigation: If bias score exceeds threshold: (1) augment training data with counterfactual examples, (2) apply adversarial debiasing (add a discriminator that cannot predict demographic from model internals), (3) post-process outputs to equalize scores across groups.
Observability: Bias dashboard tracks gender, racial, and religious bias scores per model version over time. Regression alerts when a new model version shows significantly higher bias than the current production model.

Q10: Design a Continuous Evaluation System for Production NLP

System Design Focus: Observability High Availability
Design a system that continuously evaluates NLP model quality in production using both automatic metrics and periodic human evaluation, with automated rollback capabilities.

Architecture:
An always-on evaluation system that shadows production traffic, computes quality signals, and gates model deployments.
Components:

- **Shadow Evaluator:** Samples 0.1% of production requests. Runs both the current model and the new candidate. Computes automatic metrics on side-by-side outputs.
- **Human Eval Queue:** 100 sampled predictions per week sent for human quality rating (1-5 scale). Results averaged per model version.
- **Automated Metrics:** For tasks with reference outputs: BLEU/ROUGE. For open-ended generation: BERTScore, reference-free quality estimators (Comet-QE for MT).
- **Rollback Trigger:** If automatic metric drops ¿3% or human rating drops ¿0.2 points vs. baseline, trigger automated rollback.

```
1 import random
2 from datetime import datetime
3
4 class ContinuousEvaluator:
5     def __init__(self, production_model,
```

```
                candidate_model, metrics,
                human_eval_queue, rollback_mgr):
        self.prod = production_model
        self.candidate = candidate_model
        self.metrics = metrics
        self.human_queue = human_eval_queue
        self.rollback = rollback_mgr

    def evaluate_request(self, request,
                         sample_rate=0.001):
        if random.random() > sample_rate:
            return  # Not sampled
        # Run both models
        prod_output = self.prod.predict(request)
        cand_output = self.candidate.predict(request)
        # Compute automatic metrics
        scores = {
            "bertscore": self.metrics.bertscore(
                prod_output, cand_output),
            "timestamp": datetime.utcnow().isoformat()
        }
        # Log for aggregation
        self.log_comparison(request,
                            prod_output,
                            cand_output, scores)
        # Check rollback condition
        agg = self.get_hourly_aggregate()
        if agg["delta"] < -0.03:  # 3% drop
            self.rollback.trigger(
                reason="metric_regression",
                delta=agg["delta"])

    def get_hourly_aggregate(self) -> dict:
        # Query metrics from time-series DB
        recent = self.metrics_db.query(
            hours=1, model="candidate")
        baseline = self.metrics_db.query(
            hours=1, model="production")
        return {
            "delta": recent["mean"] - baseline["mean"],
            "sample_count": recent["count"]
        }
```

High Availability: The evaluator is a side-car process. If it fails, production serving is unaffected. Evaluation results are eventually consistent—stored asynchronously in a time-series database. Rollback decisions require a minimum of 1000 samples before triggering.

10 Practical & Ethical Considerations

linkedin.com/in/lamhotsiagian

Building NLP systems that work in the lab is one thing. Building systems that operate ethically, reliably, and responsibly at scale is another. The best NLP engineers are not just technically proficient—they understand the societal implications of their systems and design with responsibility from day one.

10.1 Chapter Overview

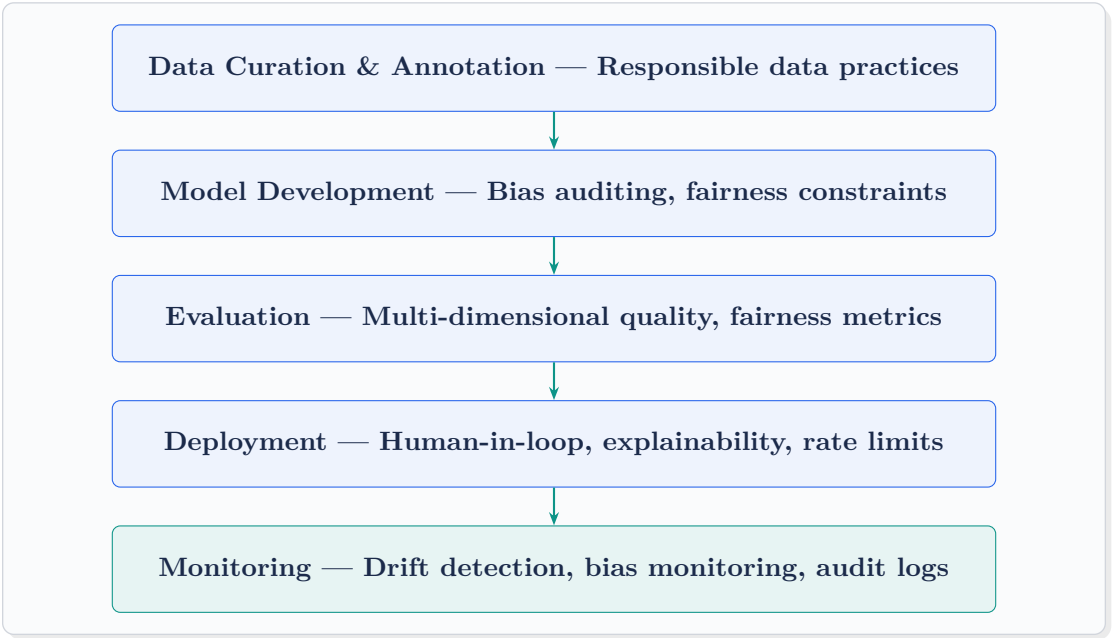
The final chapter ties together the practical realities of production NLP engineering with the ethical responsibilities that come with deploying systems that affect millions of people.

Library Ecosystem: spaCy, Hugging Face Transformers, and NLTK are the three pillars of the Python NLP ecosystem. Each has a distinct role: spaCy for efficient production pipelines, Transformers for state-of-the-art models, NLTK for classical algorithms and research.

Dataset Curation and Annotation: The quality of an NLP model is bounded by the quality of its training data. Garbage in, garbage out—but at scale, "garbage" can mean biased, unrepresentative, or mislabeled data that subtly corrupts model behavior.

Bias and Fairness: Language models trained on internet data inherit the biases of the internet. Gender stereotypes, racial disparities, and cultural blind spots can manifest in production systems in ways that are harmful and often legally actionable.

Responsible Deployment: Human-in-the-loop systems, uncertainty quantification, explainability, and audit trails are not optional niceties—they are engineering requirements for responsible AI.



The Engineer's Responsibility

At companies like Meta and Google, engineers who build NLP systems are responsible not just for the code, but for the downstream consequences of their systems. A content moderation model that disproportionately flags text from a minority community is an engineering failure—not just a research problem.

10.2 System Design Interview Questions

Q1: Design a Responsible AI Governance System for NLP Models

System Design Focus: Security Observability
Design a governance framework for NLP models that enforces ethical guidelines, maintains audit trails, and requires human approval for high-stakes deployments.

Architecture:
A multi-gate model lifecycle management system where models progress through stages with increasing scrutiny.
Governance Gates:

- **Gate 1 – Technical Review:** Automated checks. Performance benchmarks must exceed baseline. Bias scores below threshold. No data leakage detected.
- **Gate 2 – Ethics Review:** Human review by an AI Ethics panel (cross-functional: engineering, policy, legal, affected community representatives). Required for any model in high-stakes domains (healthcare, legal, hiring).
- **Gate 3 – Legal Review:** Privacy impact assessment. GDPR/CCPA compliance check. Intellectual property review for training data.
- **Gate 4 – Staged Rollout:** 1% → 10% → 100% with monitoring at each stage. Rollback authority at any stage.

```python
from enum import Enum
from datetime import datetime

class GovernanceStage(Enum):
    DEVELOPMENT = "development"
    TECHNICAL_REVIEW = "technical_review"
    ETHICS_REVIEW = "ethics_review"
    LEGAL_REVIEW = "legal_review"
    STAGED_ROLLOUT = "staged_rollout"
    PRODUCTION = "production"
    RETIRED = "retired"

class ModelGovernanceSystem:
    def __init__(self, model_registry,
                 notification_service):
        self.registry = model_registry
        self.notify = notification_service

    def submit_for_review(self, model_id,
                          version, submitter):
        model = self.registry.get(model_id, version)
        # Auto-run technical checks
        tech_result = self.run_technical_checks(model)
        if not tech_result["passed"]:
            return {"status": "rejected",
                    "reason": tech_result["failures"]}

        # Transition to ethics review
        self.registry.update_stage(
            model_id, version,
            GovernanceStage.ETHICS_REVIEW)
        # Notify ethics committee
```

```
        self.notify.create_review_task(
            model_id, version,
            committee="ethics",
            deadline_days=5)
        return {"status": "under_review"}

    def approve_stage(self, model_id, version,
                      stage, reviewer, notes):
        """Human approval at each gate."""
        audit_entry = {
            "model_id": model_id,
            "version": version,
            "stage": stage.value,
            "reviewer": reviewer,
            "decision": "approved",
            "notes": notes,
            "timestamp": datetime.utcnow().isoformat()
        }
        self.registry.add_audit_entry(audit_entry)
        next_stage = self.next_stage(stage)
        self.registry.update_stage(
            model_id, version, next_stage)

    def run_technical_checks(self, model) -> dict:
        failures = []
        metrics = model.get_metrics()
        if metrics.get("gender_bias", 1) > 0.05:
            failures.append("gender_bias_exceeded")
        if metrics.get("f1", 0) < 0.85:
            failures.append("quality_below_threshold")
        return {"passed": len(failures) == 0,
                "failures": failures}
```

Audit Trail: Every stage transition is logged immutably. All reviewer decisions (approvals, rejections, notes) are timestamped and attributed. The full lineage of a model in production—from training data to deployment—is queryable.

Q2: Design a Privacy-Preserving NLP Training System

System Design Focus: **Security** **Data Consistency**

Design a training system that trains NLP models on sensitive user data (medical

records, financial documents) while providing formal privacy guarantees using differential privacy and federated learning.

Architecture:
Two complementary privacy techniques: **Differential Privacy (DP)** adds calibrated noise to gradient updates, preventing the model from memorizing individual examples. **Federated Learning (FL)** keeps data on-device or on-premises, sharing only model updates.

Differential Privacy for NLP:

- Per-sample gradient clipping (max norm C=1.0) prevents any single example from dominating.
- Gaussian noise added to clipped gradients (σ calibrated to target ε privacy budget).
- Privacy accounting: track ε spent across training. Stop training when budget exhausted.
- Cost: DP training typically requires 3-5x more data and epochs to achieve the same quality as non-DP training.

```
from opacus import PrivacyEngine
from torch.utils.data import DataLoader
import torch

def train_with_dp(model, dataset, target_epsilon=8.0,
                  target_delta=1e-5, max_grad_norm=1.0):
    optimizer = torch.optim.Adam(
        model.parameters(), lr=2e-5)
    dataloader = DataLoader(dataset,
                            batch_size=64)
    # Attach privacy engine
    privacy_engine = PrivacyEngine()
    model, optimizer, dataloader = \
        privacy_engine.make_private_with_epsilon(
            module=model,
            optimizer=optimizer,
            data_loader=dataloader,
            epochs=3,
            target_epsilon=target_epsilon,
            target_delta=target_delta,
            max_grad_norm=max_grad_norm
        )
```

```
    for epoch in range(3):
        for batch in dataloader:
            optimizer.zero_grad()
            outputs = model(**batch)
            outputs.loss.backward()
            optimizer.step()
        epsilon = privacy_engine.get_epsilon(
            target_delta)
        print(f"Epoch {epoch}: epsilon={epsilon:.2f}")
        if epsilon >= target_epsilon:
            print("Privacy budget exhausted!")
            break
    return model, epsilon
```

Federated Learning: For multi-institution settings (e.g., hospitals), each institution trains a local model on its data and sends only encrypted gradient updates to a central aggregator. The aggregator averages updates and sends back the improved global model. No raw data ever leaves the institution.
Trade-off: $\varepsilon = 8$ provides reasonable privacy while maintaining 90% of non-DP model quality. Lower ε (stronger privacy) requires more data or accepts more quality degradation.

Q3: Design a Dataset Curation and Quality Control System

System Design Focus: **Data Storage & Retrieval** **Observability**
Design a system that manages the lifecycle of NLP training datasets—from collection through annotation, quality filtering, versioning, and access control.

Architecture:
A dataset management platform with automated quality checks, human annotation workflows, and strict lineage tracking.
Components:

- **Data Ingestion:** Sources (web scrape, API, internal data) push to a staging area. Every document gets a SHA-256 hash for deduplication and provenance tracking.
- **Automated Quality Filters:** Language detection, text length, fluency score (perplexity from a small LM), PII scan, toxicity score. Documents outside acceptable ranges are quarantined.

- **Human Annotation:** Label Studio or Prodigy workflow. Annotation tasks assigned based on annotator expertise and language. Inter-annotator agreement tracked in real time.
- **Dataset Versioning:** Each dataset release is an immutable snapshot stored on S3 with a manifest (list of document hashes, annotation versions, quality filter parameters).

```python
import hashlib, json
from datetime import datetime

class DatasetCurationSystem:
    def __init__(self, quality_filters,
                 annotation_platform, storage):
        self.filters = quality_filters
        self.annotation = annotation_platform
        self.storage = storage

    def ingest_document(self, text: str,
                        source: str) -> dict:
        doc_id = hashlib.sha256(
            text.encode()).hexdigest()
        quality = self.assess_quality(text)
        if quality["status"] == "rejected":
            return {"doc_id": doc_id,
                    "status": "quarantined",
                    "reasons": quality["reasons"]}
        doc = {
            "id": doc_id,
            "text": text,
            "source": source,
            "quality_scores": quality["scores"],
            "ingested_at": datetime.utcnow()
                           .isoformat()
        }
        self.storage.save(doc)
        return {"doc_id": doc_id,
                "status": "accepted"}

    def create_dataset_version(self, docs,
                               name, version):
        manifest = {
            "name": name, "version": version,
            "created_at": datetime.utcnow().isoformat(),
```

```
            "document_count": len(docs),
            "document_hashes": [
                d["id"] for d in docs],
            "filter_params": self.filters.config,
            "statistics": self.compute_stats(docs)
        }
        version_path = f"datasets/{name}/{version}"
        self.storage.save_manifest(
            version_path, manifest)
        return version_path

    def assess_quality(self, text: str) -> dict:
        scores = {}
        reasons = []
        scores["length"] = len(text.split())
        if scores["length"] < 10:
            reasons.append("too_short")
        scores["perplexity"] = \
            self.filters.fluency_model.score(text)
        if scores["perplexity"] > 5000:
            reasons.append("low_fluency")
        scores["pii_detected"] = \
            self.filters.pii_detector.has_pii(text)
        if scores["pii_detected"]:
            reasons.append("contains_pii")
        return {"status": "rejected"
                if reasons else "accepted",
                "scores": scores,
                "reasons": reasons}
```

Observability: Real-time dashboard showing ingestion rate, rejection rate by reason, annotation throughput, and inter-annotator agreement. Weekly dataset quality report sent to data owners.

Q4: Design a Human-in-the-Loop NLP System for High-Stakes Decisions

System Design Focus: **High Availability** **Communication Between Services**
Design a loan processing system that uses NLP to parse and analyze financial documents, with mandatory human review for all decisions above a risk threshold.

Architecture:
A hybrid automation system: NLP handles extraction and preliminary analysis; humans review and approve decisions that meet a risk threshold.
Decision Routing:

- **Low-risk (confidence $>$ 0.9, amount $<$ $10K):** Fully automated decision. NLP extracts income, debt, credit history. Risk model outputs recommendation. Decision logged.
- **Medium-risk (confidence 0.7-0.9 or amount $10K-$100K):** Automated draft decision with NLP explanation. Human loan officer reviews the draft, can approve or override with reason.
- **High-risk (confidence $<$ 0.7 or amount $>$ $100K):** NLP extracts information only. Human makes the decision from scratch, with NLP analysis as support material.

```python
from enum import Enum

class RiskTier(Enum):
    LOW = "automated"
    MEDIUM = "human_review"
    HIGH = "human_decision"

class LoanProcessingSystem:
    def __init__(self, nlp_extractor,
                 risk_model, review_queue,
                 decision_logger):
        self.nlp = nlp_extractor
        self.risk = risk_model
        self.queue = review_queue
        self.logger = decision_logger

    def process_application(self, app_id: str,
                            documents: list) -> dict:
        # NLP extraction
        extracted = self.nlp.extract_financials(
            documents)
        # Risk assessment
        risk_score = self.risk.score(extracted)
        amount = extracted.get("loan_amount", 0)
        confidence = risk_score["confidence"]
        # Route by risk tier
        if confidence > 0.9 and amount < 10000:
            tier = RiskTier.LOW
```

```
            decision = risk_score["recommendation"]
            self.logger.log(app_id, decision,
                            tier, extracted)
            return {"decision": decision,
                    "tier": tier.value,
                    "explanation": extracted}

        elif confidence >= 0.7 or amount <= 100000:
            tier = RiskTier.MEDIUM
            draft = {"recommendation":
                     risk_score["recommendation"],
                     "reasoning": extracted,
                     "confidence": confidence}
            review_id = self.queue.submit(
                app_id, draft,
                priority="normal",
                sla_hours=4)
            return {"tier": tier.value,
                    "review_id": review_id,
                    "status": "pending_review"}

        else:
            tier = RiskTier.HIGH
            support_material = {
                "extracted_data": extracted,
                "risk_signals": risk_score,
                "documents": documents
            }
            review_id = self.queue.submit(
                app_id, support_material,
                priority="high", sla_hours=24)
            return {"tier": tier.value,
                    "review_id": review_id}
```

Communication: Review tasks published to a Kafka topic consumed by the loan officer portal. Reviewers see the NLP analysis as a sidebar, not the primary interface—to minimize automation bias. All reviewer decisions are logged with their reasoning.

Q5: Design an Explainable NLP System for Regulated Industries

System Design Focus: Observability Security
Design an NLP system for healthcare that produces not just predictions but human-readable explanations, enabling clinicians to understand and audit model decisions.

Architecture:
An explainability layer wraps every NLP model. Each prediction comes with an explanation in three forms: feature importance, attention visualization, and a natural language rationale.
Explanation Methods:

- **LIME (Local Interpretable Model-agnostic Explanations):** Perturb the input, observe output changes, fit a local linear model. Identifies which words most influenced the prediction.
- **SHAP (SHapley Additive exPlanations):** Game-theoretic approach to feature attribution. More principled than LIME but slower.
- **Attention Weights:** Visualize which tokens the model attended to. Useful as a sanity check, though attention $\neq$ explanation.
- **LLM-Generated Rationale:** A secondary LLM generates a natural language explanation of why the primary model made its decision, citing evidence from the text.

```
import lime.lime_text
import shap
import numpy as np

class ExplainableNLPModel:
    def __init__(self, model, tokenizer,
                 class_names):
        self.model = model
        self.tokenizer = tokenizer
        self.class_names = class_names
        self.explainer = lime.lime_text\
            .LimeTextExplainer(
                class_names=class_names)

    def predict_proba(self, texts):
        """Wrapper for explanation frameworks."""
        results = []
```

```
        for text in texts:
            inputs = self.tokenizer(
                text, return_tensors="pt")
            import torch
            with torch.no_grad():
                logits = self.model(**inputs).logits
            probs = torch.softmax(
                logits, dim=-1).numpy()[0]
            results.append(probs)
        return np.array(results)

    def explain(self, text: str,
                num_features: int = 10) -> dict:
        # LIME explanation
        exp = self.explainer.explain_instance(
            text,
            self.predict_proba,
            num_features=num_features,
            num_samples=500)
        features = exp.as_list()
        prediction = self.predict_proba([text])[0]
        return {
            "prediction": self.class_names[
                prediction.argmax()],
            "confidence": float(prediction.max()),
            "explanation": [
                {"word": f[0], "impact": round(f[1],4)}
                for f in features],
            "positive_words": [
                f[0] for f in features if f[1] > 0],
            "negative_words": [
                f[0] for f in features if f[1] < 0]
        }
```

Regulatory: In healthcare (FDA), financial services (FCRA), and hiring (EEOC), models must be explainable for audit purposes. Every prediction and its explanation are stored immutably (7-year retention for healthcare). Clinicians can submit disagreements, which feed back into model improvement.

Q6: Design a Scalable Model Serving System Using Hugging Face Inference Endpoints

System Design Focus: **Scalability** **Cost Management**
Design a production NLP serving system using Hugging Face's ecosystem (Transformers, Inference API, Optimum) for a company processing 100M NLP requests per day.

Architecture:
A multi-tier serving setup using Hugging Face Transformers locally deployed on managed infrastructure.
Components:

- **Model Registry:** Models versioned and stored in Hugging Face Hub (private). Each model version tagged with performance benchmarks.
- **Optimized Inference:** Models compiled with Hugging Face Optimum (ONNX Runtime or TensorRT backend). Provides 2-3x speedup over vanilla PyTorch.
- **Serving Layer:** Hugging Face Text Generation Inference (TGI) for generative models. FastAPI endpoints for classification/extraction tasks.
- **Caching:** Semantic response cache (Chapter 7 pattern) reduces LLM calls by 30-40%.

```python
from optimum.onnxruntime import (
    ORTModelForSequenceClassification)
from transformers import pipeline, AutoTokenizer

class OptimizedHFServingStack:
    def __init__(self, model_id: str):
        self.model_id = model_id
        # Load ONNX-optimized model
        self.tokenizer = AutoTokenizer\
            .from_pretrained(model_id)
        self.model = ORTModelForSequenceClassification\
            .from_pretrained(
                model_id,
                export=True,
                provider="CUDAExecutionProvider")
        self.classifier = pipeline(
            "text-classification",
            model=self.model,
            tokenizer=self.tokenizer,
```

```
            device=0)

    def classify_batch(self,
                       texts: list[str]) -> list:
        # Batched inference with ONNX Runtime
        results = self.classifier(
            texts,
            batch_size=64,
            truncation=True,
            max_length=512)
        return [{"label": r["label"],
                 "score": round(r["score"], 4)}
                for r in results]

    def benchmark(self, sample_texts):
        import time
        t0 = time.perf_counter()
        self.classify_batch(sample_texts * 10)
        elapsed = time.perf_counter() - t0
        throughput = (len(sample_texts) * 10)\
                     / elapsed
        print(f"Throughput: {throughput:.0f} texts/sec")
```

Cost at Scale: 100M requests/day ≈ 1160 requests/sec average (peak: 5000 RPS). With ONNX optimization (3x speedup) and batching (batch size 64), each A10 GPU handles 5000 classifications/sec. 1 GPU covers average load; 5 GPUs for peak. Monthly GPU cost: $2,000 vs. $40,000 without optimization.

Q7: Design a Fairness-Aware NLP Model Training Pipeline

System Design Focus: Data Consistency Observability
Design a training pipeline that incorporates fairness constraints during model training, ensuring demographic parity across protected groups.

Architecture:
A constrained optimization training loop that jointly minimizes prediction loss and demographic parity violation.
Fairness Constraints:

- **Demographic Parity:** P(positive prediction — group A) = P(positive pre-

diction — group B). The model should not predict positive outcomes more often for one demographic group.

- **Equal Opportunity:** True positive rates should be equal across groups. When the model predicts positive, it should be correct equally often for all groups.
- **Implementation:** Add a fairness regularization term to the loss function: $\mathcal{L}_{total} = \mathcal{L}_{task} + \lambda \cdot \mathcal{L}_{fairness}$.

```python
import torch
import torch.nn as nn
import numpy as np

class FairnessAwareTrainer:
    def __init__(self, model, fairness_lambda=0.1):
        self.model = model
        self.lambda_fair = fairness_lambda

    def compute_fairness_loss(self, predictions,
                              group_labels):
        """Demographic parity constraint."""
        groups = group_labels.unique()
        if len(groups) < 2:
            return torch.tensor(0.0)
        # Positive prediction rate per group
        rates = []
        for g in groups:
            mask = (group_labels == g)
            group_preds = predictions[mask]
            rates.append(group_preds.mean())
        # Penalize variance across group rates
        rates_tensor = torch.stack(rates)
        return rates_tensor.var()

    def training_step(self, batch):
        inputs = batch["input_ids"]
        labels = batch["labels"]
        groups = batch["demographic_group"]

        logits = self.model(inputs).logits
        probs = torch.sigmoid(logits.squeeze())

        # Task loss
        task_loss = nn.BCELoss()(probs, labels.float())
        # Fairness loss
```

```
        fair_loss = self.compute_fairness_loss(
            probs.detach(), groups)
        # Combined loss
        total_loss = task_loss + \
                     self.lambda_fair * fair_loss
        return total_loss, {
            "task_loss": task_loss.item(),
            "fairness_loss": fair_loss.item()
        }
```

Monitoring: Track fairness metrics (demographic parity difference, equal opportunity difference) per model version. A fairness dashboard shows trends across protected attributes. Alert when any metric exceeds threshold.
Data: Fairness-aware training requires demographic labels, which may not always be available. Alternatives: proxy attributes (name, location), or fairness constraints without demographic labels (adversarial debiasing).

Q8: Design an NLP System for Regulatory Compliance Monitoring

System Design Focus: **Security** **High Availability**
Design a system that continuously monitors all text communications within a financial institution for regulatory compliance violations, with real-time alerts and immutable audit logs.

Architecture:
A real-time stream processing pipeline that applies NLP classifiers to internal communications (emails, chat, documents) to detect potential compliance violations.
Detection Categories:

- **Market Manipulation:** Language indicating insider trading, pump-and-dump schemes, front-running.
- **Inappropriate Promises:** "Guaranteed returns," "risk-free investment," misleading performance claims.
- **Regulatory Keywords:** Mentions of competitors, regulatory bodies, or sensitive topics in improper context.
- **Data Exfiltration:** Detection of account numbers, client data, or proprietary information in external communications.

```python
import hashlib
from datetime import datetime

class ComplianceMonitor:
    def __init__(self, nlp_classifiers,
                 alert_service, audit_logger):
        self.classifiers = nlp_classifiers
        self.alerts = alert_service
        self.audit = audit_logger

    def analyze_communication(self,
                              comm_id: str,
                              text: str,
                              metadata: dict) -> dict:
        findings = []
        # Run all compliance classifiers
        for category, clf in \
                self.classifiers.items():
            result = clf.predict(text)
            if result["score"] > 0.7:
                findings.append({
                    "category": category,
                    "score": result["score"],
                    "evidence": result.get(
                        "highlighted_text", "")
                })

        # Immutable audit log (every communication)
        audit_record = {
            "comm_id": comm_id,
            "content_hash": hashlib.sha256(
                text.encode()).hexdigest(),
            "metadata": metadata,
            "findings": findings,
            "analyzed_at": datetime.utcnow().isoformat(),
            "retention_years": 7
        }
        self.audit.write_immutable(audit_record)

        # Alert on high-severity findings
        high_sev = [f for f in findings
                    if f["score"] > 0.85]
        if high_sev:
```

```
            self.alerts.trigger(
                severity="high",
                comm_id=comm_id,
                findings=high_sev,
                escalate_to="compliance_team")

        return {"comm_id": comm_id,
                "findings": findings,
                "requires_review": len(findings) > 0}
```

Security: All communications are encrypted at rest (AES-256) and in transit (TLS 1.3). Audit logs are write-once (WORM storage). Compliance officer access is role-based and itself audited. False positive rate tracked; excessive false positives reduce trust and create compliance fatigue.

Q9: Design a Responsible Data Deletion System for NLP Models

System Design Focus: Data Consistency Security
Design a system that handles user data deletion requests (GDPR "right to be forgotten") for NLP models that were trained on user data.

Architecture:
A multi-layer deletion system addressing data at rest, in the training pipeline, and within trained model weights.
Deletion Layers:

- **Layer 1 – Raw Data:** Delete user data from all storage (S3, databases, backups). Automated with a deletion service that propagates to all replicas and backups within 30 days (GDPR deadline).
- **Layer 2 – Training Datasets:** Remove user data from all training dataset versions. Mark datasets as "containing deleted data." Trigger retraining before those datasets are used again.
- **Layer 3 – Model Weights:** This is the hardest part. Neural networks may memorize training data. Options: (a) Machine Unlearning: update model weights to "forget" the deleted data without full retraining. (b) Full Retraining: most reliable but expensive.

```
import hashlib
```

```python
from datetime import datetime, timedelta

class DataDeletionService:
    def __init__(self, storage, dataset_registry,
                 model_registry, unlearning_engine):
        self.storage = storage
        self.datasets = dataset_registry
        self.models = model_registry
        self.unlearner = unlearning_engine

    def process_deletion_request(self,
                                 user_id: str,
                                 request_id: str):
        deadline = (datetime.utcnow() +
                    timedelta(days=30)).isoformat()
        # Step 1: Delete raw data
        raw_records = self.storage.find_by_user(
            user_id)
        for record in raw_records:
            self.storage.delete(record["id"])

        # Step 2: Mark affected datasets
        affected_datasets = \
            self.datasets.find_containing_user(user_id)
        for ds in affected_datasets:
            self.datasets.mark_stale(
                ds["id"],
                reason=f"user_{user_id}_deleted")

        # Step 3: Queue model unlearning
        affected_models = \
            self.models.find_trained_on(affected_datasets)
        for model in affected_models:
            self.unlearner.queue_unlearning(
                model_id=model["id"],
                user_id=user_id,
                priority="compliance")

        # Audit trail
        return {
            "request_id": request_id,
            "user_id": user_id,
            "deadline": deadline,
```

```
            "raw_records_deleted": len(raw_records),
            "datasets_affected": len(
                affected_datasets),
            "models_queued_for_unlearning": len(
                affected_models)
        }
```

Machine Unlearning: An emerging field with no perfect solution yet. SISA training (Sharded, Isolated, Sliced, and Aggregated) is one approach: partition training data into shards, train model components on each shard, enabling removal of a shard (and retraining only that component) when data is deleted. Reduces retraining cost by 10-100x.

Q10: Design a Responsible AI Monitoring Dashboard for NLP Systems

System Design Focus: **Observability** **High Availability**
Design a real-time monitoring system that tracks model quality, fairness, bias, cost, and usage across all NLP models in an organization's portfolio.

Architecture:
A centralized observability platform aggregating metrics from all NLP services into a unified dashboard with role-specific views.
Metric Categories:

- **Performance:** Latency (P50/P95/P99), throughput, error rate, availability. Per-model, per-endpoint.
- **Quality:** Automatic metric scores (BLEU, F1), human eval scores, user satisfaction (CSAT, thumbs up/down).
- **Fairness:** Demographic parity, equal opportunity, calibration across subgroups.
- **Safety:** Content policy violation rate, prompt injection detection rate, hallucination rate (for generative models).
- **Cost:** GPU utilization, cost per prediction, monthly cost by team.
- **Drift:** Input distribution drift (PSI), output distribution shift, embedding space drift.

```
class ResponsibleAIMonitor:
    def __init__(self, metrics_db,
```

```
                alert_manager, dashboard):
        self.db = metrics_db
        self.alerts = alert_manager
        self.dashboard = dashboard

    def ingest_model_metrics(self, model_id,
                             version, metrics):
        self.db.write(model_id, version, metrics)
        self.check_thresholds(model_id,
                             version, metrics)

    def check_thresholds(self, model_id,
                         version, metrics):
        thresholds = {
            "latency_p99_ms": 500,
            "error_rate": 0.01,
            "gender_bias_score": 0.05,
            "hallucination_rate": 0.10,
            "input_psi": 0.20
        }
        violations = []
        for metric, threshold in \
                thresholds.items():
            if metric in metrics and \
               metrics[metric] > threshold:
                violations.append({
                    "metric": metric,
                    "value": metrics[metric],
                    "threshold": threshold
                })
        if violations:
            self.alerts.fire(
                model_id=model_id,
                severity="warning",
                violations=violations)

    def generate_weekly_report(self,
                               team: str) -> dict:
        models = self.db.get_team_models(team)
        report = {}
        for model_id in models:
            report[model_id] = {
                "quality": self.db.avg(
```

```
                    model_id, "f1", days=7),
                "fairness": self.db.avg(
                    model_id, "gender_bias", days=7),
                "cost_usd": self.db.sum(
                    model_id, "cost_usd", days=7),
                "availability": self.db.avg(
                    model_id, "availability", days=7)
            }
        return report
```

Role-Specific Views:

- **Engineers:** Technical performance, latency, error rates, GPU utilization.
- **Data Scientists:** Model quality metrics, drift detection, evaluation results.
- **Ethics & Policy:** Fairness metrics, bias scores, safety violation rates.
- **Leadership:** Cost by team, portfolio health, regulatory compliance status.

High Availability: The monitoring system is the most critical service—if it goes down, teams lose visibility into production. Multi-region deployment, separate from the models being monitored. Data retention: 90 days hot (InfluxDB), 2 years cold (S3 + Parquet).

11 Bonus: Top 35 MAANG NLP System Design Questions

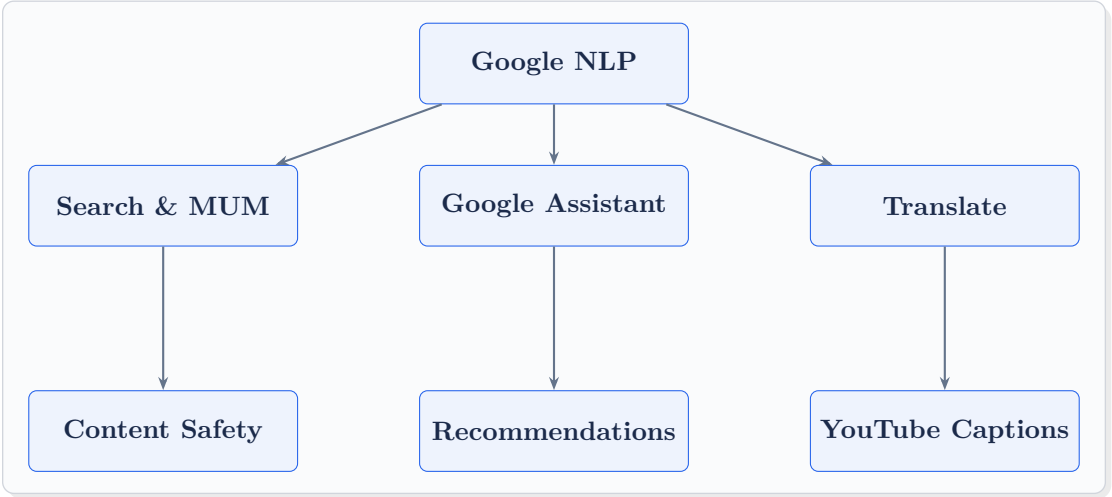
linkedin.com/in/lamhotsiagian

This bonus chapter presents the most frequently asked NLP system design questions at Meta, Amazon, Apple, Netflix, and Google—seven per company—sourced from interview reports, engineering blogs, and first-hand accounts. Each answer is structured for the real interview room: lead with scale requirements, state trade-offs clearly, and ground every decision in measurable outcomes.

How MAANG Interviews Differ

Each company has a distinct engineering culture that shapes how they ask system design questions. Google prizes theoretical depth and elegant abstractions. Meta values shipping fast and measuring impact. Amazon demands explicit cost-benefit analysis via the Leadership Principles. Apple prioritizes privacy-by-design. Netflix obsesses over personalization and reliability. Knowing this shapes not just what you build, but how you present it.

11.1 Google — Top 7 NLP System Design Questions



G-Q1: Design Google Search's Query Understanding Pipeline (MUM/BERT at Scale)

System Design Focus: Scalability Latency
How would you design the query understanding layer for Google Search—covering intent detection, entity recognition, and semantic matching—to serve billions of queries per day at sub-100ms latency?

Architecture: A two-tier system: a lightweight BERT-mini model for real-time query tagging (intent, entities, language) running in under 5ms, and a heavier MUM/T5-based model for complex multi-step queries triggered only when the lightweight model signals low confidence.

Serving Infrastructure:

- **Model Serving:** TensorFlow Serving clusters with hardware-accelerated TPU inference. Each query goes through a DAG: language detection → tokenization → lightweight classifier → (conditional) deep model.
- **Caching:** Query embedding cache (Redis) keyed on normalized query hash. Cache hit rate for popular queries exceeds 40%, reducing model calls significantly.
- **Fallback:** If deep model exceeds 80ms, fall back to lightweight result. SLA: p99 $<$ 100ms.

```python
import torch
from transformers import BertTokenizerFast, BertForSequenceClassification

class QueryUnderstandingService:
    def __init__(self, light_model_path, heavy_model_path, threshold=0.85):
        self.tokenizer = BertTokenizerFast.from_pretrained(light_model_path)
        self.light_model = BertForSequenceClassification.from_pretrained(light_model_path).half().cuda().eval()
        self.threshold = threshold
        self.cache = {}  # In prod: Redis

    @torch.no_grad()
    def understand(self, query: str) -> dict:
        key = hash(query.lower().strip())
        if key in self.cache:
            return self.cache[key]

        inputs = self.tokenizer(query, return_tensors="pt", max_length=64, truncation=True).to("cuda")
        logits = self.light_model(**inputs).logits
        confidence = torch.softmax(logits, dim=-1).max().item()

        if confidence < self.threshold:
            # Route to heavy model (async in production)
            result = self._heavy_model_inference(query)
        else:
            result = {"intent": logits.argmax().item(), "confidence": confidence, "model": "light"}
        self.cache[key] = result
        return result

    def _heavy_model_inference(self, query):
        # MUM/T5 call with timeout guard
        return {"intent": 0, "confidence": 1.0, "model": "heavy"}
```

Scalability: Horizontal scaling behind a load balancer. Each shard owns a partition of the cache. Query volume spikes handled via autoscaling with a 30-second warm-up buffer for new replicas.

G-Q2: Design Google Translate's Neural Machine Translation Serving System

System Design Focus: **High Availability** **Latency**
Design the serving infrastructure for Google Translate supporting 100+ language pairs, billions of daily requests, and offline mobile translation.

Multi-Tier Architecture: (1) Cloud serving for connected users using large Transformer models; (2) On-device models (quantized to 8-bit) for offline use. A routing layer detects connectivity and selects the appropriate inference path.
Model Management: Language pairs are grouped into multilingual models (one model handles 50 language pairs via language tokens). Rare language pairs share a low-resource multilingual checkpoint. This reduces the fleet from 10,000 specialized models to 20 multilingual ones.

```python
from transformers import MarianMTModel, MarianTokenizer
import torch

class TranslationRouter:
    def __init__(self, cloud_endpoint, on_device_models: dict):
        self.cloud_endpoint = cloud_endpoint
        self.on_device = on_device_models  # {lang_pair: model}

    def translate(self, text: str, src: str, tgt: str,
                  offline: bool = False) -> str:
        lang_pair = f"{src}-{tgt}"
        if offline or not self._cloud_healthy():
            return self._on_device_translate(text, lang_pair)
        return self._cloud_translate(text, src, tgt)

    def _on_device_translate(self, text: str, lang_pair: str) -> str:
```

```
        model, tokenizer = self.on_device[lang_pair]
        inputs = tokenizer(text, return_tensors="pt",
            truncation=True)
        with torch.no_grad():
            outputs = model.generate(**inputs,
                max_new_tokens=256)
        return tokenizer.decode(outputs[0],
            skip_special_tokens=True)

    def _cloud_healthy(self) -> bool:
        return True  # In prod: health check with circuit
            breaker
```

Availability: Multi-region active-active deployment. Each region can serve all language pairs independently. Cross-region failover with DNS-based routing. SLA: 99.99% uptime.

G-Q3: Design a Real-Time Content Safety System for YouTube Using NLP

System Design Focus: High Availability Observability
Design an NLP pipeline to detect harmful content (hate speech, misinformation, spam) in YouTube comments at 5M comments/hour with sub-second flagging.

Pipeline: Multi-stage cascade. Stage 1: fast regex/keyword blocklist filter (handles 60% of obvious violations in microseconds). Stage 2: lightweight BERT binary classifier (latency 8ms). Stage 3: multi-label fine-grained classifier for borderline cases. Stage 4: human review queue.

```
import asyncio
from enum import Enum

class SafetyLabel(Enum):
    SAFE = "safe"
    BORDERLINE = "borderline"
    VIOLATION = "violation"

class ContentSafetyPipeline:
    def __init__(self, blocklist, fast_model, deep_model):
```

```
        self.blocklist = blocklist
        self.fast_model = fast_model
        self.deep_model = deep_model

    async def classify(self, comment: str) -> dict:
        # Stage 1: blocklist
        if any(term in comment.lower() for term in
            self.blocklist):
            return {"label": SafetyLabel.VIOLATION,
                "stage": 1,
                    "action": "auto_remove"}

        # Stage 2: fast model
        score = await self.fast_model.score(comment)
        if score < 0.1:
            return {"label": SafetyLabel.SAFE, "stage": 2}
        if score > 0.85:
            return {"label": SafetyLabel.VIOLATION,
                "stage": 2,
                    "action": "auto_remove"}

        # Stage 3: deep model for borderline
        detail = await self.deep_model.classify(comment)
        return {"label": SafetyLabel.BORDERLINE, "stage":
            3,
                "detail": detail, "action":
                    "human_review"}
```

Observability: Each stage emits latency, throughput, and false-positive rate metrics to Cloud Monitoring. A/B test new models against production traffic before full rollout. Alert if false-positive rate exceeds 0.5%.

G-Q4: Design Google Assistant's Natural Language Understanding Service

System Design Focus: Latency Service Communication

How would you design the NLU backend for Google Assistant to handle intent classification, slot filling, and dialogue state management at sub-50ms end-to-end latency?

Architecture: A gRPC microservice pipeline: (1) ASR output arrives, (2) NLU service performs intent + slot extraction in parallel, (3) Dialogue Manager reads/writes state from a low-latency state store (Spanner or Bigtable), (4) fulfillment API called, (5) TTS response.

```python
import grpc
from dataclasses import dataclass
from typing import List, Dict

@dataclass
class NLUResult:
    intent: str
    slots: Dict[str, str]
    confidence: float
    dialogue_state: dict

class NLUService:
    def __init__(self, intent_model, slot_model, state_store):
        self.intent_model = intent_model
        self.slot_model = slot_model
        self.state_store = state_store

    async def process(self, utterance: str, session_id: str) -> NLUResult:
        # Parallel intent and slot extraction
        import asyncio
        intent_task = asyncio.create_task(
            self.intent_model.predict(utterance))
        slot_task = asyncio.create_task(
            self.slot_model.extract(utterance))

        intent, slots = await asyncio.gather(intent_task, slot_task)

        # Read dialogue state
        state = await self.state_store.get(session_id) or {}

        # Update state with new turn
        state["last_intent"] = intent.label
        state["turn_count"] = state.get("turn_count", 0) + 1
```

```
        await self.state_store.set(session_id, state,
            ttl=3600)

        return NLUResult(intent=intent.label, slots=slots,
                         confidence=intent.score,
                            dialogue_state=state)
```

Latency Budget: ASR 20ms + NLU 15ms (parallel) + State read 3ms + Fulfillment 10ms = 48ms total. P99 guardrail enforced with a 50ms timeout; fallback to a cached "I didn't understand" response.

G-Q5: Design a Semantic Search System for Google Workspace (Docs, Drive, Gmail)

System Design Focus: Database Bottlenecks Data Storage & Retrieval
Design a cross-product semantic search system that lets users search their personal documents, emails, and calendar events using natural language across Google Workspace.

Indexing Pipeline: Documents ingested via Pub/Sub → embedding service (BERT/E5 model) → vector index (ScaNN — Google's own ANN library). Each user's documents are embedded at creation/modification time and stored in a per-user vector shard.

```
from google.cloud import pubsub_v1
import numpy as np

class WorkspaceIndexer:
    def __init__(self, embed_model, vector_store,
        text_store):
        self.embed_model = embed_model
        self.vector_store = vector_store  # ScaNN / ANN
            index
        self.text_store = text_store      # Bigtable for
            raw text

    def index_document(self, doc_id: str, user_id: str,
                       content: str, doc_type: str):
```

```
        # Chunk long documents (max 512 tokens per chunk)
        chunks = self._chunk(content, max_tokens=512)
        embeddings = self.embed_model.encode(chunks,
                                             batch_size=32,
                                             normalize=True)
        # Store embeddings with metadata
        for i, (chunk, emb) in enumerate(zip(chunks,
            embeddings)):
            chunk_id = f"{doc_id}_{i}"
            self.vector_store.upsert(
                user_id=user_id,
                vector_id=chunk_id,
                vector=emb.tolist(),
                metadata={"doc_type": doc_type, "doc_id":
                    doc_id}
            )
            self.text_store.put(chunk_id, chunk)

    def _chunk(self, text: str, max_tokens: int) -> list:
        words = text.split()
        return [" ".join(words[i:i+max_tokens])
                for i in range(0, len(words), max_tokens)]
```

Retrieval: User query → embedding → ANN search in user's shard (top-k=20) → cross-encoder re-ranking → top-5 results. Strict per-user isolation ensures no data leakage across Workspace accounts.

G-Q6: Design an AutoComplete and Query Suggestion System for Google Search

System Design Focus: **Latency** **Scalability**

Design the query suggestion system that powers Google Search's autocomplete, supporting 10B+ daily interactions with sub-30ms p99 latency.

Architecture: A precomputed prefix trie stored in distributed memory (Redis Cluster) combined with a neural re-ranker for personalization. The trie covers the top 10M queries; long-tail queries are handled by an n-gram language model fallback.

```python
import redis
import json
from typing import List

class AutoCompleteService:
    def __init__(self, redis_client, personalizer,
        lm_fallback):
        self.redis = redis_client
        self.personalizer = personalizer
        self.lm_fallback = lm_fallback

    def suggest(self, prefix: str, user_id: str,
                top_k: int = 8) -> List[str]:
        # Step 1: Trie lookup (sub-1ms from Redis)
        key = f"ac:{prefix.lower()[:20]}"
        cached = self.redis.get(key)

        if cached:
            candidates = json.loads(cached)[:20]
        else:
            # Fallback: n-gram LM generates candidates
            candidates =
                self.lm_fallback.generate(prefix, n=20)

        # Step 2: Personalized re-ranking
        scored = self.personalizer.rerank(candidates,
            user_id)
        return [c["query"] for c in sorted(
            scored, key=lambda x: -x["score"])][:top_k]
```

Freshness: Trending queries (last 15 minutes, from stream processing) are injected directly into Redis bypassing the nightly trie rebuild. This keeps autocomplete current during breaking news events.

G-Q7: Design Google's Large-Scale Multilingual Document Classification System

System Design Focus: **Scalability** **Cost Management**

Design a system to classify billions of web documents daily across 100+ categories and 50+ languages to power Google's topic-based content organization.

Architecture: Batch-first approach. Daily Dataflow pipeline reads raw crawl data → language detection → multilingual BERT classifier (mBERT or XLM-R) → results written to Bigtable. Real-time path (for newly indexed content) uses a Pub/Sub triggered classifier.

```python
import apache_beam as beam
from transformers import pipeline

class ClassifyDocsDoFn(beam.DoFn):
    def setup(self):
        # Loaded once per worker
        self.classifier = pipeline(
            "text-classification",
            model="xlm-roberta-base",
            device=0,  # GPU
            truncation=True,
            max_length=512
        )

    def process(self, doc: dict):
        result = self.classifier(doc["text"][:2000])
        yield {
            "doc_id": doc["doc_id"],
            "language": doc.get("lang", "unknown"),
            "category": result[0]["label"],
            "score": result[0]["score"],
        }

def run_pipeline(input_table, output_table):
    with beam.Pipeline() as p:
        (p
         | "ReadDocs" >>
             beam.io.ReadFromBigQuery(table=input_table)
         | "Classify" >> beam.ParDo(ClassifyDocsDoFn())
         | "WriteToBQ" >>
             beam.io.WriteToBigQuery(table=output_table))
```

Cost Management: Use distilled models (6-layer MobileBERT equivalent) for 95% of documents. Only documents in low-confidence or high-value categories are re-routed to the full XLM-R model. This reduces compute cost by 4× while maintaining accuracy on the long tail.

11.2 Meta — Top 7 NLP System Design Questions

Meta's NLP Focus

Meta's NLP challenges are driven by scale (3B+ users), multimodality (text + image + video), and safety (content moderation at unprecedented volume). Meta prizes real-time systems, tight feedback loops, and measurable engagement impact.

M-Q1: Design Meta's News Feed Ranking NLP Signal Extraction System

System Design Focus: **Scalability** **Latency**
Design the NLP component of Meta's Feed ranking system that extracts semantic signals (topic, sentiment, entity) from 500M+ posts per day to inform the ranking model.

Architecture: Asynchronous offline enrichment pipeline. Posts published → Kafka topic → NLP workers (Flink jobs) extract signals → signals stored in TAO (Meta's distributed graph store) → Feed ranking model reads signals at inference time.

```python
from dataclasses import dataclass, field
from typing import List, Dict
import json

@dataclass
class PostSignals:
    post_id: str
    topics: List[str] = field(default_factory=list)
    sentiment: float = 0.0  # -1 to 1
    entities: List[str] = field(default_factory=list)
    toxicity_score: float = 0.0
    language: str = "en"

class FeedNLPWorker:
    def __init__(self, topic_model, sentiment_model,
                 entity_model, toxicity_model,
                    tao_client):
        self.topic_model = topic_model
        self.sentiment_model = sentiment_model
```

```
        self.entity_model = entity_model
        self.toxicity_model = toxicity_model
        self.tao = tao_client

    def process_post(self, post: dict) -> PostSignals:
        text = post.get("text", "")[:512]

        # Parallel inference (in prod: separate
            microservices)
        signals = PostSignals(post_id=post["id"])
        signals.topics = self.topic_model.predict(text)
        signals.sentiment =
            self.sentiment_model.score(text)
        signals.entities = self.entity_model.extract(text)
        signals.toxicity_score =
            self.toxicity_model.score(text)
        signals.language = post.get("lang", "en")

        # Write to TAO graph
        self.tao.write(f"post:{post['id']}:nlp_signals",
                       signals.__dict__)
        return signals
```

Scalability: Kafka consumer groups scaled to handle 6K posts/second peak. Signal freshness SLA: within 2 seconds of post creation. Hot posts (viral) are prioritized in the processing queue.

M-Q2: Design Meta's Multilingual Content Moderation NLP System

System Design Focus: **High Availability** **Observability**
Design a content moderation system capable of detecting hate speech, violence, and policy violations across 100+ languages at Meta's scale.

Multi-Model Architecture: Language detection → route to language-specialized or multilingual model → policy-specific classifiers (hate speech, violence, nudity, spam) → severity scoring → action (auto-remove, demote, human review).

```
from enum import Enum
```

```python
from typing import Tuple

class ModerationAction(Enum):
    ALLOW = "allow"
    DEMOTE = "demote"
    HUMAN_REVIEW = "human_review"
    AUTO_REMOVE = "auto_remove"

class ModerationPipeline:
    def __init__(self, lang_detector, model_registry,
                 policy_engine, review_queue):
        self.lang_detector = lang_detector
        self.models = model_registry  # {lang: model}
        self.policy = policy_engine
        self.review_queue = review_queue

    def moderate(self, content: dict) ->
        Tuple[ModerationAction, dict]:
        text = content.get("text", "")
        lang = self.lang_detector.detect(text)

        # Select model (specialized > multilingual
            fallback)
        model = self.models.get(lang,
            self.models["multilingual"])
        scores = model.score_all_policies(text)

        # Policy engine maps scores to actions
        action, reason = self.policy.evaluate(scores,
            lang)

        if action == ModerationAction.HUMAN_REVIEW:
            self.review_queue.enqueue(content, scores,
                priority=
                scores.get("severity", 0.5))

        return action, {"scores": scores, "lang": lang,
                        "reason": reason}
```

Observability: Per-language, per-policy precision/recall tracked daily. Human review accuracy feeds back as labels for weekly model retraining. Alert thresholds trigger on-call if false-positive rate exceeds 1% for any language.

M-Q3: Design WhatsApp's On-Device NLP for Smart Reply Suggestions

System Design Focus: **Security** **Latency**
Design a smart reply system for WhatsApp that generates contextual reply suggestions entirely on-device, without sending message content to Meta's servers.

Privacy-First Design: The entire NLP pipeline (intent classification, reply generation) runs locally on device using a quantized model. No message text is transmitted; only aggregate, differential-privacy-protected usage signals are federated back for model improvement.

```python
import torch
from transformers import AutoModelForSeq2SeqLM, AutoTokenizer

class OnDeviceSmartReply:
    def __init__(self, model_path: str, max_suggestions: int = 3):
        # INT8 quantized model for mobile (< 50MB)
        self.tokenizer = AutoTokenizer.from_pretrained(model_path)
        self.model = torch.quantization.quantize_dynamic(
            AutoModelForSeq2SeqLM.from_pretrained(model_path),
            {torch.nn.Linear}, dtype=torch.qint8
        )
        self.max_suggestions = max_suggestions

    def suggest(self, last_message: str,
                context: list = None) -> list:
        # Context window: last 3 messages only
        ctx = (context or [])[-2:] + [last_message]
        prompt = " | ".join(ctx)

        inputs = self.tokenizer(prompt, return_tensors="pt",
                                max_length=128, truncation=True)
        with torch.no_grad():
            outputs = self.model.generate(
                **inputs,
```

```
                num_return_sequences=self.max_suggestions,
                num_beams=self.max_suggestions,
                max_new_tokens=20,
                early_stopping=True
            )
        suggestions = self.tokenizer.batch_decode(
            outputs, skip_special_tokens=True)
        return list(dict.fromkeys(suggestions))  # deduplicate
```

Security: End-to-end encryption is maintained. The model runs in a sandboxed process with no network access. Federated learning updates are clipped and noised with Gaussian mechanism ($\varepsilon = 8$) before aggregation.

M-Q4: Design Meta AI's Retrieval-Augmented Generation System for Llama Integration

System Design Focus: **Data Storage & Retrieval** **Latency**
Design the RAG architecture that powers Meta AI assistant's ability to answer questions about current events using Llama models with real-time web retrieval.

Architecture: User query → query rewriter (Llama-3 8B) → dual retrieval (dense ANN vector search + BM25 sparse) → cross-encoder re-ranker → context assembly → Llama-3 70B generation.

```
from dataclasses import dataclass
from typing import List

@dataclass
class RetrievedDoc:
    doc_id: str
    text: str
    score: float
    source_url: str

class MetaAI_RAG:
    def __init__(self, query_rewriter, dense_retriever,
                 sparse_retriever, reranker, llm):
        self.rewriter = query_rewriter
```

```
        self.dense = dense_retriever
        self.sparse = sparse_retriever
        self.reranker = reranker
        self.llm = llm

    def answer(self, user_query: str,
               max_docs: int = 5) -> dict:
        # Step 1: Rewrite for better retrieval
        search_query = self.rewriter.rewrite(user_query)

        # Step 2: Dual retrieval (parallel)
        import asyncio
        dense_docs = self.dense.search(search_query,
            top_k=20)
        sparse_docs = self.sparse.search(search_query,
            top_k=20)

        # Step 3: Merge and re-rank
        candidates = list({d.doc_id: d
                           for d in dense_docs +
                               sparse_docs}.values())
        ranked = self.reranker.rank(search_query,
            candidates)
        context_docs = ranked[:max_docs]

        # Step 4: Generate with Llama
        context = "\n\n".join(
            f"[{i+1}] {d.text}" for i, d in
                enumerate(context_docs))
        prompt = (f"Answer based on the following
            sources:\n{context}"
                  f"\n\nQuestion: {user_query}\nAnswer:")
        response = self.llm.generate(prompt,
            max_tokens=512,
                                     temperature=0.1)
        return {"answer": response,
                "sources": [d.source_url for d in
                    context_docs]}
```

Latency Budget: Query rewrite 80ms + Retrieval 40ms (parallel) + Re-ranking 20ms + LLM generation 500ms = 640ms end-to-end. Streaming output reduces perceived latency; first token arrives in ∼100ms.

M-Q5: Design Instagram's Hashtag and Caption Understanding System

System Design Focus: **Scalability** **Data Consistency**
Design an NLP system to understand Instagram captions and hashtags to power content discovery, topic tagging, and recommendation at 100M+ posts/day.

Architecture: Async pipeline. New post → Kafka → NLP enrichment (entity extraction, topic tagging, sentiment, hashtag normalization) → search index (Elasticsearch) + recommendation feature store (Redis).

```python
from typing import List, Dict
import hashlib

class InstagramNLPEnricher:
    def __init__(self, entity_extractor, topic_tagger,
                 hashtag_normalizer, feature_store):
        self.entity_extractor = entity_extractor
        self.topic_tagger = topic_tagger
        self.hashtag_normalizer = hashtag_normalizer
        self.store = feature_store

    def enrich(self, post: dict) -> dict:
        caption = post.get("caption", "")
        hashtags = post.get("hashtags", [])

        # NLP extraction
        entities = self.entity_extractor.extract(caption)
        topics = self.topic_tagger.predict(caption + " " +
                                           "
                                              ".join(hashtags))
        norm_hashtags =
           [self.hashtag_normalizer.normalize(h)
                        for h in hashtags]

        enriched = {
            "post_id": post["id"],
            "entities": entities,
            "topics": topics,
            "normalized_hashtags": norm_hashtags,
            "content_language": self._detect_lang(caption)
```

```
        }

        # Write to feature store (for recommendations)
        self.store.hset(f"post:{post['id']}:features",
                        mapping=enriched)
        return enriched

    def _detect_lang(self, text: str) -> str:
        return "en"  # Simplified; use langdetect in prod
```

Data Consistency: Eventual consistency is acceptable—search index and recommendations update within 5 seconds of post creation. Hashtag normalization is idempotent so duplicate Kafka messages are safe to reprocess.

M-Q6: Design Meta's Automated Translation System for Cross-Language Social Interaction

System Design Focus: High Availability Cost Management
Design the system that automatically translates posts and comments across languages on Facebook, serving 3B+ users with real-time translation at minimal cost.

Architecture: Translation-as-a-Service behind a CDN cache. Identical content (same post, same language pair) served from cache without re-inference. Cache key: SHA256(source_text) + lang_pair.

```
import hashlib
import redis
from typing import Optional

class TranslationService:
    def __init__(self, translation_model, redis_client,
                 cache_ttl: int = 86400):
        self.model = translation_model
        self.cache = redis_client
        self.cache_ttl = cache_ttl

    def translate(self, text: str, src_lang: str,
                  tgt_lang: str) -> str:
```

```
        if src_lang == tgt_lang:
            return text

        # Cache lookup
        cache_key = self._cache_key(text, src_lang,
            tgt_lang)
        cached = self.cache.get(cache_key)
        if cached:
            return cached.decode()

        # Inference
        translation = self.model.translate(text,
            src_lang, tgt_lang)

        # Cache result (posts are immutable, so TTL can
            be long)
        self.cache.setex(cache_key, self.cache_ttl,
                         translation.encode())
        return translation

    def _cache_key(self, text: str, src: str, tgt: str)
        -> str:
        h = hashlib.sha256(text.encode()).hexdigest()[:16]
        return f"trans:{src}:{tgt}:{h}"
```

Cost Management: Cache hit rate of 70%+ for popular posts (virality drives repetition). Batch translation for low-priority content (historical posts). Low-resource language pairs handled by a pivot translation through English when direct model isn't available.

M-Q7: Design a Real-Time Hate Speech Detection System for Live Video on Facebook

System Design Focus: **Latency** **High Availability**
Design a system to detect hate speech in real-time audio/text streams from Facebook Live, with under 5-second detection latency and 99.9% uptime.

Architecture: Audio stream → ASR (streaming Whisper-style model) → rolling 30-second transcript window → hate speech classifier → action (mute stream,

notify reviewer, soft block). The text and audio pipelines run in parallel with the ASR latency as the dominant factor.

```python
import asyncio
from collections import deque

class LiveStreamModerator:
    def __init__(self, asr_streamer, hate_classifier,
                 action_engine, window_secs=30):
        self.asr = asr_streamer
        self.classifier = hate_classifier
        self.action_engine = action_engine
        self.window = deque(maxlen=window_secs * 10)  # ~10 words/sec

    async def monitor_stream(self, stream_id: str,
                             audio_chunks):
        async for chunk in audio_chunks:
            # ASR transcription (streaming, partial results)
            transcript_piece = await self.asr.transcribe_chunk(chunk)
            if not transcript_piece:
                continue

            self.window.extend(transcript_piece.split())
            rolling_text = " ".join(self.window)

            # Score every 2 seconds to limit compute
            score = await self.classifier.score_async(rolling_text)
            if score > 0.75:
                await self.action_engine.take_action(
                    stream_id=stream_id,
                    score=score,
                    evidence=rolling_text[-200:]  # Last 200 chars
                )
```

High Availability: Stateless classifier pods behind an L7 load balancer. Stream processing state (rolling window) is held in-process; failure restarts the window but does not miss future content. Redundant ASR workers per stream ensure no single point of failure.

11.3 Amazon — Top 7 NLP System Design Questions

Amazon's NLP Focus

Amazon's NLP spans three ecosystems: AWS services (Comprehend, Lex, Polly), Alexa voice assistant, and e-commerce (product search, reviews, recommendations). Amazon's Leadership Principles strongly influence interview expectations—especially Customer Obsession, Frugality, and Dive Deep.

A-Q1: Design Alexa's Intent Recognition and Slot Filling System at Scale

System Design Focus: Latency Scalability
Design Alexa's NLU backend to process voice commands from 100M+ devices with intent classification and slot filling at sub-100ms latency.

Architecture: On-device wake word detection → audio streamed to Alexa cloud → ASR → NLU (intent + slots) → skill routing → fulfillment. NLU uses a multi-task BERT-based model: intent classification head + slot-filling token classification head trained jointly.

```python
import torch
import torch.nn as nn
from transformers import BertModel

class AlexaNLUModel(nn.Module):
    def __init__(self, bert_path, num_intents,
        num_slot_labels):
        super().__init__()
        self.bert = BertModel.from_pretrained(bert_path)
        hidden = self.bert.config.hidden_size
        self.intent_head = nn.Linear(hidden, num_intents)
        self.slot_head = nn.Linear(hidden,
            num_slot_labels)

    def forward(self, input_ids, attention_mask):
        out = self.bert(input_ids=input_ids,
                        attention_mask=attention_mask)
        # Intent: use [CLS] token
        intent_logits =
```

```
            self.intent_head(out.last_hidden_state[:, 0])
        # Slots: use all token representations
        slot_logits =
            self.slot_head(out.last_hidden_state)
        return intent_logits, slot_logits

class AlexaNLU:
    def __init__(self, model, tokenizer, intent_labels,
        slot_labels):
        self.model = model.half().cuda().eval()
        self.tokenizer = tokenizer
        self.intent_labels = intent_labels
        self.slot_labels = slot_labels

    @torch.no_grad()
    def predict(self, utterance: str) -> dict:
        enc = self.tokenizer(utterance,
            return_tensors="pt",
                             max_length=64,
                                truncation=True)
        enc = {k: v.cuda() for k, v in enc.items()}
        intent_logits, slot_logits = self.model(**enc)
        intent =
            self.intent_labels[intent_logits.argmax()]
        slot_ids = slot_logits.argmax(dim=-1)[0].tolist()
        tokens = self.tokenizer.convert_ids_to_tokens(
            enc["input_ids"][0])
        slots = {self.slot_labels[s]: t
                 for t, s in zip(tokens, slot_ids)
                 if self.slot_labels[s] != "O"}
        return {"intent": intent, "slots": slots}
```

Scalability: Auto-scaling EC2 Inf1 (Inferentia) instances. Load varies by time of day and device wake patterns. Predictive scaling pre-warms capacity before morning peak (6–9 AM per timezone).

A-Q2: Design Amazon Product Search's Semantic Understanding Layer

System Design Focus: **Data Storage & Retrieval** **Scalability**
Design the NLP component of Amazon's product search to understand natural

language queries and match them semantically to a catalog of 350M+ products.

Bi-Encoder Architecture: Product embeddings precomputed offline and stored in FAISS; query embeddings computed at search time. Re-ranking by a cross-encoder for the top-100 candidates.

```python
import faiss
import numpy as np

class ProductSearchNLP:
    def __init__(self, query_encoder, product_index: faiss.Index,
                 reranker, product_db):
        self.encoder = query_encoder
        self.index = product_index
        self.reranker = reranker
        self.db = product_db

    def search(self, query: str, top_k: int = 20) -> list:
        # Encode query
        q_emb = self.encoder.encode(query, normalize=True)
        q_emb = np.array([q_emb], dtype=np.float32)

        # ANN search
        distances, indices = self.index.search(q_emb, 100)
        candidates = [self.db.get(i) for i in indices[0]
            if i >= 0]

        # Cross-encoder re-ranking
        pairs = [(query, p["title"] + " " +
            p["description"][:200])
                 for p in candidates]
        scores = self.reranker.score(pairs)
        ranked = sorted(zip(candidates, scores),
                        key=lambda x: -x[1])
        return [p for p, _ in ranked[:top_k]]
```

Index Freshness: Full FAISS rebuild nightly (overnight batch job). Intraday product additions handled via a delta index merged at query time. A/B tested against keyword BM25 baseline with click-through rate as the primary metric.

A-Q3: Design AWS Comprehend's Multi-Tenant NLP API

System Design Focus: **Security** **Scalability**
Design the multi-tenant architecture for AWS Comprehend that serves NLP tasks (sentiment, entities, key phrases) to millions of customer accounts with strict isolation.

Multi-Tenant Architecture: Shared model pool with per-customer request routing. Each API request carries an IAM-verified customer ID; requests are queued in per-customer FIFO queues to prevent noisy-neighbor problems. Custom models (Comprehend Custom) are isolated in per-customer inference containers.

```
from collections import defaultdict
import asyncio

class ComprehendService:
    def __init__(self, shared_model,
        custom_model_registry,
                 iam_verifier, rate_limiter):
        self.model = shared_model
        self.custom_models = custom_model_registry
        self.iam = iam_verifier
        self.limiter = rate_limiter
        self.queues = defaultdict(asyncio.Queue)

    async def analyze(self, request: dict,
                      auth_token: str) -> dict:
        # Verify identity
        account_id = await self.iam.verify(auth_token)

        # Rate limiting per account
        if not await self.limiter.allow(account_id):
            raise Exception("ThrottlingException: quota
                exceeded")

        # Route: custom model or shared
        if account_id in self.custom_models:
            model = self.custom_models[account_id]
        else:
            model = self.model

        text = request["Text"][:5000]  # Max chars per
```

```
            request
        task = request.get("AnalysisType", "SENTIMENT")

        if task == "SENTIMENT":
            return model.detect_sentiment(text)
        elif task == "ENTITIES":
            return model.detect_entities(text)
        else:
            raise ValueError(f"Unknown task: {task}")
```

Security: Customer data never crosses account boundaries. Custom model artifacts stored in customer-owned S3 buckets with KMS encryption. VPC endpoints ensure traffic stays on AWS backbone.

A-Q4: Design Amazon Review Sentiment Aggregation System

System Design Focus: **Data Storage & Retrieval** **Scalability**
Design a system to analyze sentiment of billions of Amazon product reviews and surface aspect-level insights (e.g., "battery life: positive", "shipping: negative") to shoppers and sellers.

Architecture: Batch processing pipeline: S3 (raw reviews) → EMR Spark job with ABSA (Aspect-Based Sentiment Analysis) → DynamoDB (aggregated results) → real-time API for product pages.

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import udf, col
from pyspark.sql.types import ArrayType, StructType,
    StringType, FloatType

def run_absa_pipeline(spark: SparkSession,
                      model_broadcast, input_path,
                          output_path):
    reviews_df = spark.read.parquet(input_path)

    @udf(returnType=ArrayType(StructType([
        ("aspect", StringType()),
        ("sentiment", StringType()),
        ("score", FloatType())
```

```
    ])))
    def extract_aspects(text: str):
        if not text:
            return []
        model = model_broadcast.value
        return model.analyze(text[:512])

    result_df = (reviews_df
        .filter(col("text").isNotNull())
        .withColumn("aspects",
           extract_aspects(col("text")))
        .groupby("product_id")
        .agg({"aspects": "collect_list"})
    )
    result_df.write.mode("overwrite").parquet(output_path)
```

Data Freshness: New reviews trigger near-real-time recomputation via DynamoDB Streams + Lambda, updating product-level sentiment summaries incrementally without full reprocessing.

A-Q5: Design Amazon Lex's Conversational AI Backend

System Design Focus: **High Availability** **Service Communication**
Design the dialogue management and NLU backend for Amazon Lex to power customer service bots with multi-turn conversation state management.

Architecture: Stateless NLU microservice + stateful Session Manager (DynamoDB). Each turn: request arrives → NLU extracts intent/slots → Dialogue Manager reads session state → selects next action → response assembled → state updated.

```
import boto3
import json
from typing import Optional

class LexDialogueManager:
    def __init__(self, nlu_service, dynamodb_table,
        fulfillment):
        self.nlu = nlu_service
```

```
        self.table = dynamodb_table
        self.fulfillment = fulfillment

    def process_turn(self, session_id: str,
                     utterance: str) -> dict:
        # Load session state
        state = self._load_state(session_id) or {
            "intent": None, "slots": {}, "turn": 0}

        # NLU inference
        nlu_result = self.nlu.predict(utterance)
        new_intent = nlu_result["intent"]
        new_slots = nlu_result["slots"]

        # Update state
        if new_intent and new_intent != "FollowUp":
            state["intent"] = new_intent
        state["slots"].update(new_slots)
        state["turn"] += 1

        # Check if all required slots are filled
        required = self._required_slots(state["intent"])
        missing = [s for s in required if s not in
            state["slots"]]
        if missing:
            response = f"Could you tell me your
                {missing[0]}?"
        else:
            # All slots filled: call fulfillment
            response =
                self.fulfillment.call(state["intent"],
                                      state["slots"])
            state = {}  # Reset after completion

        self._save_state(session_id, state)
        return {"response": response, "session_state":
            state}

    def _load_state(self, session_id: str) ->
        Optional[dict]:
        try:
            item = self.table.get_item(
                Key={"session_id": session_id})
```

```
            return json.loads(item["Item"]["state"])
        except KeyError:
            return None

    def _save_state(self, session_id: str, state: dict):
        self.table.put_item(Item={
            "session_id": session_id,
            "state": json.dumps(state)})

    def _required_slots(self, intent: str) -> list:
        slot_map = {"BookFlight": ["origin",
           "destination", "date"],
                    "OrderPizza": ["size", "toppings",
                       "address"]}
        return slot_map.get(intent, [])
```

High Availability: NLU service deployed across 3 AZs. DynamoDB with global tables for low-latency multi-region session state. Circuit breaker around fulfillment calls with graceful fallback message.

A-Q6: Design Amazon's Query Rewriting System for Sponsored Product Search

System Design Focus: Latency Cost Management

Design an NLP-based query rewriting system that expands and reformulates user search queries to improve Sponsored Product ad relevance while keeping latency under 20ms.

Architecture: Precomputed rewrite index for common queries + a seq2seq model for long-tail queries. Common queries (~80% of traffic) get rewrites from Redis in under 2ms. Long-tail queries go through a distilled T5-small model (50ms).

```
from functools import lru_cache

class QueryRewriter:
    def __init__(self, redis_client, seq2seq_model,
                 tokenizer, coverage_threshold=0.8):
        self.cache = redis_client
        self.model = seq2seq_model
```

```
        self.tokenizer = tokenizer
        self.threshold = coverage_threshold

    def rewrite(self, query: str) -> list:
        normalized = query.lower().strip()

        # Fast path: precomputed rewrites
        cached = self.cache.lrange(f"rw:{normalized}", 0,
            9)
        if cached:
            return [c.decode() for c in cached]

        # Slow path: seq2seq generation
        inputs = self.tokenizer(
            f"rewrite: {normalized}",
            return_tensors="pt", max_length=64,
                truncation=True)
        outputs = self.model.generate(
            **inputs,
            num_beams=4, num_return_sequences=4,
            max_new_tokens=32, early_stopping=True)
        rewrites = self.tokenizer.batch_decode(
            outputs, skip_special_tokens=True)
        unique_rewrites = list(dict.fromkeys(rewrites))

        # Cache for future use (1 week TTL)
        if unique_rewrites:
            self.cache.rpush(f"rw:{normalized}",
                *unique_rewrites)
            self.cache.expire(f"rw:{normalized}", 604800)

        return unique_rewrites
```

Cost Management: Seq2seq model (T5-small, 60M params) rather than T5-large. Async pre-warming: popular queries cached during off-peak hours via a batch job that processes the top 10M queries nightly.

A-Q7: Design an NLP System for Amazon Customer Service Email Triage

System Design Focus: **Observability** **High Availability**
Design an automated email triage system that classifies incoming customer service emails, extracts order/product entities, and routes them to the correct team with an SLA of 2-minute triage time.

Pipeline: Email arrives → SQS queue → Lambda triage service → (1) intent classification, (2) entity extraction (order ID, ASIN, customer ID), (3) urgency scoring → CRM routing API.

```python
import re
from dataclasses import dataclass
from typing import Optional

@dataclass
class EmailTriageResult:
    intent: str
    order_id: Optional[str]
    asin: Optional[str]
    urgency: float  # 0-1
    team: str

class EmailTriageService:
    def __init__(self, classifier, entity_extractor,
                 urgency_scorer, routing_rules):
        self.classifier = classifier
        self.extractor = entity_extractor
        self.urgency = urgency_scorer
        self.rules = routing_rules

    def triage(self, email: dict) -> EmailTriageResult:
        subject = email.get("subject", "")
        body = email.get("body", "")[:1000]  # Truncate
            long emails
        full_text = f"{subject}\n{body}"

        # Classify intent
        intent = self.classifier.predict(full_text)

```

```
        # Entity extraction (regex + NER)
        order_id = self._extract_order_id(full_text)
        asin = self._extract_asin(full_text)
        entities = self.extractor.extract(full_text)

        # Urgency scoring
        urgency_score = self.urgency.score(full_text,
            intent)

        # Routing
        team = self.rules.route(intent, urgency_score,
            entities)

        return EmailTriageResult(
            intent=intent, order_id=order_id,
            asin=asin, urgency=urgency_score, team=team)

    def _extract_order_id(self, text: str) ->
        Optional[str]:
        match = re.search(r'\b(1\d{2}-\d{7}-\d{7})\b',
            text)
        return match.group(1) if match else None

    def _extract_asin(self, text: str) -> Optional[str]:
        match = re.search(r'\b(B0[A-Z0-9]{8})\b', text)
        return match.group(1) if match else None
```

Observability: Each triage decision logged with model version, confidence, and outcome. Weekly dashboard tracks routing accuracy, miss-classification rate by intent, and SLA compliance (p95 triage time).

11.4 Apple — Top 7 NLP System Design Questions

Apple's NLP Focus

Apple's NLP engineering is defined by privacy-first principles. Almost all NLP at Apple runs on-device (Siri, Spotlight, Mail, Autocorrect). System design interviews at Apple heavily weight privacy, efficiency, and on-device model constraints. If you can't explain how your system works without any user data leaving the device, you're missing Apple's core constraint.

Ap-Q1: Design Siri's On-Device Natural Language Understanding System

System Design Focus: **Latency** **Security**
Design the NLU component of Siri that runs entirely on-device using Apple Neural Engine, supporting intent classification and slot filling with under 10ms latency.

Architecture: Quantized BERT-tiny (INT8, ~4MB) compiled for Apple Neural Engine (ANE) via Core ML. The model runs on ANE in parallel with the CPU/GPU, leaving system resources free. Input: ASR transcript, output: intent + slots + confidence.

```python
import coremltools as ct
import numpy as np

class SiriNLUCoreML:
    def __init__(self, model_path: str):
        self.model = ct.models.MLModel(model_path)
        self.intent_labels = self._load_labels("intents.json")
        self.slot_labels = self._load_labels("slots.json")

    def predict(self, transcript: str) -> dict:
        # Tokenize (BPE, vocab size 16k for small footprint)
        token_ids = self._tokenize(transcript, max_len=64)
        input_dict = {"input_ids": np.array([token_ids], dtype=np.int32)}
        out = self.model.predict(input_dict)
        intent_idx = int(np.argmax(out["intent_logits"]))
        slot_ids = np.argmax(out["slot_logits"], axis=-1)[0]
        return {
            "intent": self.intent_labels[intent_idx],
            "confidence": float(np.max(
                ct.models.utils.softmax(out["intent_logits"]))),
            "slots": self._decode_slots(token_ids, slot_ids)
        }

    def _tokenize(self, text: str, max_len: int) -> list:
```

```
        return [0] * max_len  # Placeholder for actual
            BPE tokenizer

    def _decode_slots(self, tokens, slot_ids) -> dict:
        slots = {}
        for token, slot_id in zip(tokens, slot_ids):
            label = self.slot_labels[slot_id]
            if label != "O":
                slots[label] = slots.get(label, "") + f"
                    {token}"
        return {k: v.strip() for k, v in slots.items()}

    def _load_labels(self, path: str) -> list:
        import json
        with open(path) as f:
            return json.load(f)
```

Privacy: Transcripts never leave the device for NLU processing. Only anonymized, differentially private aggregate statistics (intent distribution, not content) are optionally shared for model improvement with user consent.

Ap-Q2: Design Apple's Mail Smart Reply System with On-Device Privacy

System Design Focus: **Security** **Latency**
Design Mail's smart reply suggestions that read email context and generate contextual responses, entirely on-device with no email content sent to Apple servers.

Architecture: Lightweight seq2seq model (GPT-2 small, quantized to 4-bit) compiled for ANE. A retrieval module scans a local vector index of user-approved reply templates before falling back to generation. This makes replies both personalized and fast.

```
import numpy as np
from typing import List

class MailSmartReply:
    def __init__(self, template_index, gen_model,
```

```
        tokenizer,
                 max_templates: int = 3):
        self.index = template_index  # Local FAISS index
        self.gen_model = gen_model   # Quantized GPT-2 (CoreML)
        self.tokenizer = tokenizer
        self.max_templates = max_templates

    def suggest(self, email_body: str,
                context_turns: list = None) -> List[str]:
        # Step 1: Template retrieval (sub-5ms, local)
        query_emb = self._embed(email_body[:256])
        template_scores, template_ids = self.index.search(
            np.array([query_emb], dtype=np.float32), self.max_templates)
        templates = [self.index.get_text(i)
                     for i in template_ids[0] if i >= 0]

        # Step 2: Generation for variety (if templates are poor match)
        if template_scores[0][0] < 0.6:
            prompt = f"Reply to: {email_body[:128]}\nSuggestion:"
            tokens = self.tokenizer(prompt, return_tensors="np",
                                    max_length=128, truncation=True)
            gen_ids = self.gen_model.predict(tokens)
            generated = self.tokenizer.decode(
                gen_ids, skip_special_tokens=True)
            templates = [generated] + templates[:2]

        return templates[:self.max_templates]

    def _embed(self, text: str) -> np.ndarray:
        return np.random.randn(384).astype(np.float32)  # Placeholder
```

Differential Privacy: Template usage patterns (which suggestions were accepted, not what the emails said) are aggregated with local DP ($\varepsilon = 4$) before any signal is reported to improve future models.

Ap-Q3: Design Spotlight Search's Natural Language Query System

System Design Focus: **Data Storage & Retrieval** **Latency**
Design Apple Spotlight's ability to understand natural language queries ("photos from last summer in Paris") and retrieve matching content across on-device data.

Architecture: On-device semantic index built at idle time. Each document/photo/message gets embedded by a local model. At query time: query → embedding → ANN search on local HNSW index + structured metadata filter (date range, location) → ranked results.

```python
import sqlite3
from typing import List, Dict

class SpotlightNLSearch:
    def __init__(self, embed_model, hnsw_index, metadata_db: str):
        self.embed = embed_model
        self.index = hnsw_index
        self.db = sqlite3.connect(metadata_db)

    def search(self, query: str, top_k: int = 20) -> List[Dict]:
        # Parse structured constraints
        filters = self._parse_filters(query)
        clean_query = self._strip_filters(query, filters)

        # Semantic search
        q_emb = self.embed.encode(clean_query, normalize=True)
        labels, distances = self.index.knn_query(
            q_emb.reshape(1, -1), k=top_k * 3)

        # Post-filter by metadata
        candidates = []
        for label in labels[0]:
            row = self._get_metadata(int(label))
            if row and self._matches_filters(row, filters):
                candidates.append(row)
            if len(candidates) >= top_k:
```

```
                break
        return candidates[:top_k]

    def _parse_filters(self, query: str) -> dict:
        import re
        filters = {}
        if m := re.search(r'(last|this)\s+(\w+)',
            query.lower()):
            filters["time_hint"] = f"{m.group(1)}
                {m.group(2)}"
        return filters

    def _strip_filters(self, query: str, filters: dict)
        -> str:
        return query  # Simplified

    def _get_metadata(self, doc_id: int) -> dict:
        row = self.db.execute(
            "SELECT * FROM documents WHERE id=?",
                (doc_id,)).fetchone()
        return dict(row) if row else None

    def _matches_filters(self, row: dict, filters: dict)
        -> bool:
        return True  # Simplified; prod checks
            date/location
```

Index Freshness: HNSW index updated incrementally as new content is indexed. Low-battery mode pauses indexing. Full re-index triggered on major OS updates or significant data changes.

Ap-Q4: Design Apple's Autocorrect and Predictive Text System

System Design Focus: Latency Security

Design an on-device autocorrect and next-word prediction system for the iOS keyboard, achieving under 5ms suggestion latency with personalization to user vocabulary.

Architecture: Three-tier: (1) Static language model (compressed n-gram trie, pre-installed, covers 99% of common words); (2) Personal vocabulary trie (user's custom words and frequently used terms, encrypted in local Keychain); (3) Neural prediction (LSTM or small Transformer) for context-aware next-word suggestions.

```python
from collections import Counter
import pickle

class AutoCorrectSystem:
    def __init__(self, base_lm_path, personal_vocab_path,
                 neural_predictor):
        with open(base_lm_path, "rb") as f:
            self.base_lm = pickle.load(f)  # N-gram trie
        with open(personal_vocab_path, "rb") as f:
            self.personal = pickle.load(f)  # Counter
        self.neural = neural_predictor

    def correct(self, word: str, context: list) -> list:
        # Stage 1: exact match (personal vocab first)
        if word in self.personal:
            return [word]

        # Stage 2: edit-distance candidates from base LM
        candidates = self._edit_distance_candidates(word,
            max_dist=2)
        if not candidates:
            return [word]  # Unknown word, no correction

        # Stage 3: rank by context using neural model
        scored = []
        for cand in candidates[:10]:
            ctx_score =
                self.neural.score_sequence(context +
                [cand])
            freq_score = (self.personal.get(cand, 0) *
                0.3 +
                          self.base_lm.get(cand, 0) * 0.7)
            scored.append((cand, ctx_score + freq_score))

        return [c for c, _ in sorted(scored, key=lambda
            x: -x[1])][:3]

    def _edit_distance_candidates(self, word: str,
```

```
                                        max_dist: int) -> list:
        # Simplified BK-tree or trie traversal
        return []  # Placeholder

    def update_personal_vocab(self, accepted_word: str):
        self.personal[accepted_word] += 1
        # Persist encrypted to local storage
```

Privacy: Personal vocabulary never leaves the device. Apple's differential privacy system (used in iOS keyboard telemetry) uses a local randomizer before any aggregate statistics are reported, ensuring Apple learns word popularity without knowing which user typed what.

Ap-Q5: Design Apple's On-Device Federated Learning System for Siri Personalization

System Design Focus: Security Data Consistency
Design the federated learning infrastructure that improves Siri's NLU models using signals from millions of devices without centralizing user data.

Architecture: Devices participate in federated rounds during charging + WiFi. Each device trains on local interaction logs (no audio content, only NLU signal labels generated on-device) for a few gradient steps. Local updates are clipped, noised (Gaussian mechanism), and aggregated by Apple's secure aggregation server.

```
import torch
import torch.nn as nn
from copy import deepcopy

class FederatedSiriClient:
    def __init__(self, global_model: nn.Module,
                 local_lr: float = 1e-4,
                 clip_norm: float = 1.0,
                 noise_multiplier: float = 0.1):
        self.model = deepcopy(global_model)
        self.optimizer = torch.optim.SGD(
            self.model.parameters(), lr=local_lr)
        self.clip_norm = clip_norm
```

```
        self.noise_sigma = noise_multiplier * clip_norm

    def local_train(self, local_data: list,
                    epochs: int = 1) -> dict:
        self.model.train()
        for epoch in range(epochs):
            for batch in local_data:
                loss = self.model.compute_loss(batch)
                self.optimizer.zero_grad()
                loss.backward()
                # Gradient clipping (required for DP)
                nn.utils.clip_grad_norm_(
                    self.model.parameters(),
                        self.clip_norm)
                self.optimizer.step()

        # Compute update = local_model - global_model
        # (passed to server, not raw gradients)
        return self._compute_delta()

    def _compute_delta(self) -> dict:
        # Add Gaussian noise for differential privacy
        delta = {}
        for name, param in self.model.named_parameters():
            noise = torch.randn_like(param) *
                self.noise_sigma
            delta[name] = param.data + noise
        return delta
```

Privacy Guarantee: Central DP with (ε, δ)-DP guarantee. Secure aggregation ensures Apple's server only sees the sum of encrypted updates—no individual device's model update is visible in plaintext.

Ap-Q6: Design Apple's Document Intelligence System for iOS Files App

System Design Focus: **Latency** **Data Storage & Retrieval**
Design the NLP pipeline that powers document understanding in the Files app—extracting dates, contacts, topics, and enabling full-text semantic search—running entirely on-device.

Architecture: Background daemon (runs when device is idle/charging) processes new documents: PDF/DOCX → text extraction → NLP enrichment (dates, contacts, topics) → Core Data index update → local vector index update.

```python
from datetime import datetime
import re

class DocumentIntelligenceDaemon:
    def __init__(self, text_extractor, date_parser,
                 contact_extractor, topic_classifier,
                 embed_model, local_db):
        self.extractor = text_extractor
        self.date_parser = date_parser
        self.contact_extractor = contact_extractor
        self.topic_classifier = topic_classifier
        self.embed = embed_model
        self.db = local_db

    def process_document(self, file_path: str) -> dict:
        # Extract raw text
        text = self.extractor.extract(file_path)
        if not text:
            return {}

        # NLP enrichment
        dates = self.date_parser.parse(text)
        contacts = self.contact_extractor.extract(text)
        topics =
            self.topic_classifier.predict(text[:1000])
        embedding = self.embed.encode(text[:512],
            normalize=True)

        metadata = {
            "file_path": file_path,
            "dates": [str(d) for d in dates],
            "contacts": contacts,
            "topics": topics,
            "indexed_at": datetime.now().isoformat()
        }

        # Persist metadata and embedding
        self.db.upsert_document(file_path, metadata,
            embedding)
```

```
        return metadata
```

Resource Management: Daemon monitors battery and thermal state. Processing pauses if battery $< 20\%$ or CPU temperature is high. Incremental indexing: only new or modified files are processed, tracked via file modification timestamps.

Ap-Q7: Design Apple's Private Cloud Compute NLP Inference Architecture

System Design Focus: **Security** **High Availability**
Design Apple's Private Cloud Compute (PCC) architecture that offloads complex NLP requests from Siri to the cloud while maintaining verifiable privacy guarantees.

Architecture: When on-device model cannot handle a complex request, Siri routes to PCC nodes. PCC nodes run in hardware-attested enclaves (Secure Enclave equivalent at server scale). The request is encrypted end-to-end; the cloud node decrypts, infers, returns response, and discards all data.

```
import hashlib
import os
from typing import Tuple

class PrivateCloudComputeClient:
    def __init__(self, pcc_endpoint: str,
                 device_key: bytes, attestation_verifier):
        self.endpoint = pcc_endpoint
        self.device_key = device_key
        self.verifier = attestation_verifier

    def offload_request(self, encrypted_request: bytes,
                        request_id: str) -> Tuple[bytes,
                           str]:
        # Step 1: Verify PCC node attestation before
           sending data
        attestation =
           self._fetch_attestation(self.endpoint)
        if not self.verifier.verify(attestation):
            raise SecurityError("PCC attestation failed")

```

```
        # Step 2: Send encrypted payload (no plaintext to server)
        response = self._send_encrypted(
            encrypted_request, request_id)

        # Step 3: Verify response includes request_id
        if response.get("request_id") != request_id:
            raise SecurityError("Response mismatch")

        return response["encrypted_result"], response["nonce"]

    def _fetch_attestation(self, endpoint: str) -> dict:
        # Fetch cryptographic attestation of PCC node software
        return {}  # Placeholder

    def _send_encrypted(self, payload: bytes,
                        req_id: str) -> dict:
        # HTTPS + additional application-layer encryption
        return {}  # Placeholder

class SecurityError(Exception):
    pass
```

Privacy Guarantee: Apple's PCC design publishes node software images publicly for independent audit. Nodes cannot write logs containing request content. The "stateless inference" guarantee is enforced at the hardware level—no request data persists after the response is returned.

11.5 Netflix — Top 7 NLP System Design Questions

Netflix's NLP Focus

Netflix uses NLP extensively for content tagging, subtitle/caption generation, multilingual search, and personalization. Netflix's interview culture prizes chaos engineering mentality, A/B testing rigor, and personalization at scale. Every NLP system at Netflix is expected to have a clear A/B test design and measurable engagement metric.

N-Q1: Design Netflix's Multilingual Subtitle Generation and Quality Pipeline

System Design Focus: **Scalability** **High Availability**
Design an automated pipeline to generate and quality-check subtitles for Netflix content in 30+ languages, supporting 15,000+ new titles per year with broadcast-quality accuracy.

Pipeline: Audio → Whisper-based ASR (language-specific fine-tuned) → subtitle timing alignment → neural translation (source subtitles → target languages) → automated QA (timing, length, reading speed checks) → optional human review for premium content.

```python
from dataclasses import dataclass
from typing import List

@dataclass
class SubtitleSegment:
    start_ms: int
    end_ms: int
    text: str
    language: str

class SubtitlePipeline:
    def __init__(self, asr_model, translator,
                 qa_checker, human_review_queue):
        self.asr = asr_model
        self.translator = translator
        self.qa = qa_checker
        self.review_queue = human_review_queue

    def process(self, audio_path: str,
                src_lang: str,
                target_langs: List[str]) -> dict:
        # Step 1: ASR with timing
        segments = self.asr.transcribe_with_timing(
            audio_path, language=src_lang)

        results = {src_lang: segments}

        # Step 2: Translate to target languages
```

```
        for lang in target_langs:
            translated = []
            for seg in segments:
                t_text = self.translator.translate(
                    seg.text, src_lang, lang)
                # Preserve original timing
                translated.append(SubtitleSegment(
                    seg.start_ms, seg.end_ms, t_text,
                        lang))
            results[lang] = translated

        # Step 3: QA checks
        flagged = {}
        for lang, segs in results.items():
            issues = self.qa.check(segs)
            if issues:
                flagged[lang] = issues
                self.review_queue.enqueue(
                    {"lang": lang, "segments": segs,
                        "issues": issues},
                    priority=self.qa.severity(issues))

        return {"subtitles": results, "flagged": flagged}
```

QA Checks: Reading speed (characters per second), minimum display duration (1 second), maximum line length (42 chars), translation round-trip score above threshold. Automated QA catches 90%+ of issues before human review.

N-Q2: Design Netflix's Content Tagging NLP System for Recommendations

System Design Focus: **Scalability** **Data Storage & Retrieval**
Design a system to automatically generate rich content tags (genre, mood, themes, character archetypes, dialogue style) from scripts, synopsis, and audio/video analysis to power Netflix's recommendation engine.

Multi-Modal Pipeline: Script/synopsis → NLP tagger; Audio → ASR → dialogue analyzer; Video frames → scene classifier. Tags from all modalities merged into a structured content vector stored in Netflix's feature store (for

recommendations).

```python
from typing import Dict, List

class ContentTagger:
    def __init__(self, text_tagger, dialogue_analyzer,
                 tag_merger, feature_store):
        self.text_tagger = text_tagger
        self.dialogue = dialogue_analyzer
        self.merger = tag_merger
        self.store = feature_store

    def tag_content(self, content_id: str,
                    synopsis: str,
                    script_path: str = None) -> Dict:
        # Text-based tagging from synopsis
        text_tags = self.text_tagger.predict(synopsis)

        # Dialogue-based tags (if script available)
        dialogue_tags = {}
        if script_path:
            script_text = self._read_script(script_path)
            dialogue_tags =
               self.dialogue.analyze(script_text)

        # Merge and deduplicate
        all_tags = self.merger.merge(text_tags,
           dialogue_tags)

        # Store in feature store
        self.store.upsert(
            f"content:{content_id}:tags",
            all_tags,
            ttl=None  # Content tags are permanent
        )
        return all_tags

    def _read_script(self, path: str) -> str:
        with open(path, "r", encoding="utf-8") as f:
            return f.read()[:50000]  # First 50K chars
```

Tag Quality: Tags validated by A/B testing recommendation engagement (click-through, watch completion) on tagged vs. untagged content. Tag coverage and

consistency monitored via weekly dashboards. Human editorial team reviews tags for new genres/categories.

N-Q3: Design Netflix Search's Semantic Understanding for Mood-Based Queries

System Design Focus: **Latency** **Data Storage & Retrieval**
Design Netflix's search backend to handle natural language and mood-based queries like "something funny for a Friday night" or "thriller like Squid Game", going beyond keyword matching.

Architecture: Query → intent parser (mood, genre, comparable title) → dual-tower model for semantic matching → candidate retrieval from Elasticsearch (keyword) + FAISS (semantic) → personalized re-ranking.

```python
from typing import List, Dict

class NetflixSemanticSearch:
    def __init__(self, intent_parser, query_encoder,
                 content_index, keyword_index,
                    personalized_ranker):
        self.intent = intent_parser
        self.q_encoder = query_encoder
        self.semantic_index = content_index  # FAISS
        self.keyword_index = keyword_index   #
            Elasticsearch
        self.ranker = personalized_ranker

    def search(self, query: str, user_id: str,
               top_k: int = 20) -> List[Dict]:
        # Parse intent
        intent = self.intent.parse(query)
        # e.g., {"mood": "funny", "genre": None,
        #         "similar_to": "Squid Game"}

        # Encode query
        q_emb = self.q_encoder.encode(query,
            normalize=True)

```

```
        # Parallel retrieval
        semantic_hits = self.semantic_index.search(q_emb,
            top_k=50)
        keyword_hits = self.keyword_index.search(
            intent.get("keywords", query), top_k=50)

        # Apply intent filters
        candidates = self._merge_and_filter(
            semantic_hits, keyword_hits, intent)

        # Personalized re-ranking
        ranked = self.ranker.rank(candidates, user_id)
        return ranked[:top_k]

    def _merge_and_filter(self, semantic, keyword,
        intent):
        seen = {}
        for hit in semantic + keyword:
            if hit["id"] not in seen:
                seen[hit["id"]] = hit
        results = list(seen.values())

        if intent.get("mood"):
            results = [r for r in results
                       if intent["mood"] in
                           r.get("moods", [])]
        return results
```

A/B Testing: Semantic search results A/B tested against keyword-only baseline. Primary metric: watch completion rate (not just clicks). Secondary metrics: search abandonment rate, time to first play.

N-Q4: Design a Personalized Synopsis Generation System for Netflix

System Design Focus: Scalability Cost Management

Design a system that generates personalized content synopses—emphasizing different aspects of a show depending on the user's watch history (e.g., emphasizing the romance vs. action elements).

Architecture: User profile analysis → aspect selection (which content elements to emphasize) → LLM-based synopsis generation with aspect emphasis prompt → A/B test personalized vs. generic synopsis on click-through rate.

```python
class PersonalizedSynopsisGenerator:
    def __init__(self, user_profiler, aspect_selector,
                 llm, synopsis_cache):
        self.profiler = user_profiler
        self.selector = aspect_selector
        self.llm = llm
        self.cache = synopsis_cache

    def generate(self, content_id: str,
                 user_id: str, base_synopsis: str) -> str:
        # Check cache
        cache_key = f"synopsis:{content_id}:{user_id}"
        cached = self.cache.get(cache_key)
        if cached:
            return cached

        # Get user profile (preferred genres, themes)
        profile = self.profiler.get_profile(user_id)

        # Select aspects to emphasize
        content_aspects =
           self.selector.analyze(base_synopsis)
        emphasized = self.selector.match_to_profile(
            content_aspects, profile)

        if not emphasized:
            return base_synopsis  # No personalization,
                use default

        # Generate personalized synopsis with LLM
        prompt = (
            f"Rewrite this synopsis in 2-3 sentences, "
            f"emphasizing the following aspects: {',
                '.join(emphasized)}.\n"
            f"Original: {base_synopsis}\nRewritten:"
        )
        personalized = self.llm.generate(
            prompt, max_tokens=150, temperature=0.3)

```

```
        # Cache for 24 hours (user profile changes slowly)
        self.cache.set(cache_key, personalized, ttl=86400)
        return personalized
```

Cost Management: LLM generation only for users whose profile is sufficiently differentiated from the "average" user. The majority of users receive the default synopsis. Estimated 15% of users get personalized synopses, keeping LLM API cost proportional to impact.

N-Q5: Design Netflix's Dialogue-Based Content Safety Review System

System Design Focus: High Availability Observability
Design an automated system to analyze dialogue in Netflix originals for content advisory labeling (violence, language, sexual content) to ensure correct maturity rating before release.

Architecture: Script/subtitle file → dialogue segmentation → multi-label classifier per segment (profanity, violence, drug use, sexual content) → aggregate content report → human reviewer for borderline cases → rating recommendation.

```
from dataclasses import dataclass, field
from typing import List, Dict

@dataclass
class ContentAdvisoryReport:
    title_id: str
    overall_rating: str  # G, PG, PG-13, TV-MA, etc.
    flags: Dict[str, float] = field(default_factory=dict)
    flagged_segments: List[dict] =
       field(default_factory=list)
    needs_human_review: bool = False

class ContentSafetyReviewer:
    def __init__(self, dialogue_classifier, rating_engine,
                 review_queue):
        self.classifier = dialogue_classifier
        self.rating = rating_engine
        self.queue = review_queue
```

```
18
19      def review_content(self, title_id: str,
20                         dialogue_file: str) ->
                               ContentAdvisoryReport:
21          segments = self._parse_dialogue(dialogue_file)
22          report = ContentAdvisoryReport(title_id=title_id)
23
24          for seg in segments:
25              scores = self.classifier.classify(seg["text"])
26
27              # Flag high-confidence violations
28              for category, score in scores.items():
29                  if score > 0.7:
30                      report.flags[category] = max(
31                          report.flags.get(category, 0),
                                score)
32                      report.flagged_segments.append({
33                          "segment": seg,
34                          "category": category,
35                          "score": score
36                      })
37
38          # Determine rating
39          report.overall_rating =
               self.rating.determine(report.flags)
40          report.needs_human_review = any(
41              0.5 < s < 0.7 for s in report.flags.values())
42
43          if report.needs_human_review:
44              self.queue.enqueue(report)
45
46          return report
47
48      def _parse_dialogue(self, path: str) -> List[dict]:
49          lines = []
50          with open(path, "r") as f:
51              for line in f:
52                  line = line.strip()
53                  if line:
54                      lines.append({"text": line})
55          return lines
```

Observability: Rating accuracy tracked by comparing automated recommenda-

tions vs. final human-assigned ratings. Monthly calibration of score thresholds by category. Alert if automated-to-human disagreement rate exceeds 5%.

N-Q6: Design a Cross-Lingual NLP System for Netflix's Global Content Strategy

System Design Focus: **Scalability** **Data Consistency**
Design the multilingual NLP pipeline that helps Netflix's content strategy team analyze viewer comments, reviews, and social media in 30+ languages to inform content investment decisions by region.

Architecture: Multi-source ingestion (Twitter/X, Reddit, internal surveys, Netflix reviews) → language detection → cross-lingual sentiment and topic analysis (XLM-R or mT5) → language-normalized topic aggregation → regional content analytics dashboard.

```python
from collections import defaultdict
from typing import List, Dict

class CrossLingualContentAnalytics:
    def __init__(self, lang_detector, sentiment_model,
                 topic_model, analytics_store):
        self.detector = lang_detector
        self.sentiment = sentiment_model
        self.topics = topic_model
        self.store = analytics_store

    def analyze_batch(self, comments: List[dict]) -> Dict:
        # Group by language
        by_lang = defaultdict(list)
        for comment in comments:
            lang = self.detector.detect(comment["text"])
            by_lang[lang].append(comment)

        global_topics = defaultdict(float)
        regional_sentiment = {}

        for lang, lang_comments in by_lang.items():
            texts = [c["text"][:256] for c in
```

```
                lang_comments]

            # Multilingual inference (XLM-R handles all languages)
            sentiments = self.sentiment.batch_score(texts)
            topic_dist = self.topics.batch_predict(texts)

            avg_sentiment = sum(s["score"] for s in sentiments
                                if s["label"] == "POSITIVE") / len(sentiments)
            regional_sentiment[lang] = avg_sentiment

            # Accumulate topics (normalized labels across languages)
            for dist in topic_dist:
                for topic, score in dist.items():
                    global_topics[topic] += score

        result = {
            "total_comments": len(comments),
            "regional_sentiment": regional_sentiment,
            "global_topics": dict(global_topics),
            "top_topics": sorted(global_topics.items(),
                                 key=lambda x: -x[1])[:10]
        }
        self.store.write("content_analytics", result)
        return result
```

Data Consistency: Topics are mapped to a canonical English-language taxonomy before aggregation (so "Liebesgeschichte" in German and "love story" in English map to the same topic node). Mapping maintained by a controlled vocabulary team and updated quarterly.

N-Q7: Design Netflix's A/B Testing Framework for NLP-Driven Recommendation Changes

System Design Focus: Observability Data Consistency

Design the A/B testing infrastructure that Netflix uses to evaluate NLP-based recommendation changes, ensuring statistical rigor, metric consistency, and no

interference between experiments.

Architecture: Experiment registry → user assignment (hash-based bucketing) → treatment delivery (different NLP signals or model versions) → metric collection (engagement events) → statistical analysis service → experiment dashboard.

```python
import hashlib
from typing import Optional
from dataclasses import dataclass

@dataclass
class Experiment:
    experiment_id: str
    control_model: str
    treatment_model: str
    traffic_fraction: float  # 0.0 to 1.0
    primary_metric: str      # "completion_rate", "ctr"
    min_sample_size: int

class NLPExperimentFramework:
    def __init__(self, experiment_registry,
        model_registry,
                 metrics_store, stats_engine):
        self.registry = experiment_registry
        self.models = model_registry
        self.metrics = metrics_store
        self.stats = stats_engine

    def assign(self, user_id: str,
               experiment_id: str) -> str:
        """Returns 'control' or 'treatment'"""
        exp = self.registry.get(experiment_id)
        if not exp:
            return "control"

        # Deterministic hash-based assignment
        h = hashlib.sha256(
            f"{user_id}:{experiment_id}".encode()).hexdigest()
        bucket = int(h[:8], 16) / 0xFFFFFFFF  # 0.0 to 1.0

        if bucket < exp.traffic_fraction:
```

```
            return "treatment"
        return "control"

    def get_model(self, user_id: str,
                  experiment_id: str) -> object:
        arm = self.assign(user_id, experiment_id)
        exp = self.registry.get(experiment_id)
        model_id = (exp.treatment_model if arm ==
            "treatment"
                    else exp.control_model)
        return self.models.load(model_id)

    def analyze(self, experiment_id: str) -> dict:
        exp = self.registry.get(experiment_id)
        control_metrics = self.metrics.get(
            experiment_id, "control", exp.primary_metric)
        treatment_metrics = self.metrics.get(
            experiment_id, "treatment",
                exp.primary_metric)

        result = self.stats.t_test(
            control_metrics, treatment_metrics)
        return {
            "experiment_id": experiment_id,
            "p_value": result["p_value"],
            "lift": result["lift"],
            "significant": result["p_value"] < 0.05,
            "sample_sizes": {
                "control": len(control_metrics),
                "treatment": len(treatment_metrics)
            }
        }
```

Consistency Guardrails: Novelty effect mitigation by requiring minimum 2-week experiment duration before declaring winner. CUPED variance reduction applied to primary metrics. Automatic experiment collision detection prevents users from being enrolled in conflicting experiments simultaneously.

Interview Debrief: What Distinguishes Top Answers

Across all five companies, the candidates who receive offers share these traits: they state assumptions and constraints within the first 60 seconds; they size the system before designing it; they identify the single most expensive component and address it explicitly; and they proactively bring up failure modes, fallback strategies, and monitoring. The Python code in your answer is a signal of implementation fluency—it does not need to be production-ready, but it must be logically coherent and reflect genuine understanding of the data structures and APIs involved.

11 Final Words

linkedin.com/in/lamhotsiagian

"The best NLP engineers are not just those who know the most algorithms, but those who understand when and how to apply them wisely."

You have now completed a comprehensive tour of NLP system design—from mathematical foundations to ethical deployment. Each chapter addressed a core NLP domain and demonstrated how system design thinking applies at every layer of the stack.

What the Best Interviews Look For:

FAANG-level interviewers are not testing your ability to memorize architectures. They are evaluating your engineering judgment—your ability to make principled trade-offs given real constraints. When you are asked to design an NLP system, they want to see:

- **Clarity of thought:** Can you decompose a complex system into its components?
- **Trade-off awareness:** Do you understand the cost of every architectural choice?
- **Scalability instinct:** Do you think in terms of orders of magnitude—10x, 100x, 1000x?
- **Failure mode thinking:** What happens when things go wrong, and how does your system recover?
- **Metric orientation:** Can you define success quantitatively, and can you measure it?

The Three Questions to Always Answer:

Before you start drawing architecture diagrams in any system design interview, answer these three questions explicitly:

(1) What are the scale requirements? QPS, data volume, latency SLAs. These determine your architecture almost entirely.

(2) What are the consistency requirements? Strong vs. eventual? This determines your database and replication choices.

(3) What are the cost constraints? Real systems have budgets. A solution that requires 1000 A100 GPUs for a startup interview will not impress.

Keep Learning:

NLP moves fast. The transformer was new in 2017. BERT in 2018. GPT-3 in 2020. GPT-4 in 2023. By the time you read this, there will be new architectures and paradigms. But the system design principles in this book—scalability, fault tolerance, consistency, observability—are timeless.

Stay curious, stay humble, and keep building.

Lamhot Siagian

AI Engineering Insider

Connect: linkedin.com/in/lamhotsiagian

Support: buymeacoffee.com/lamhot